

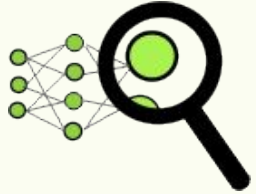


Transformers

(based on materials from Lena Voita)

extra slides: Michael Diskin

Lecture-blog and lots of additional materials are here:
https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html

 Recap: seq2seq attention (last week)

Sequence to Sequence Task

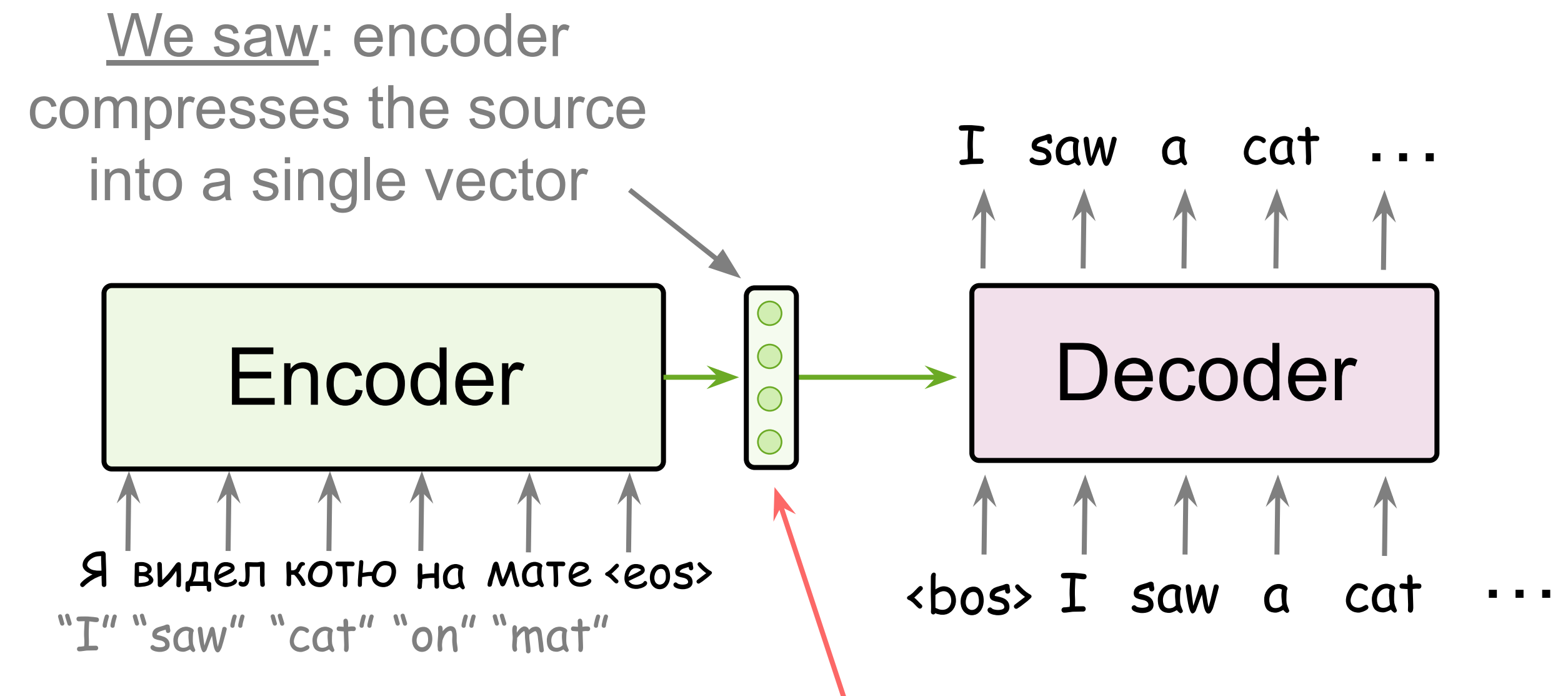
- Translation between natural languages
- More generally, translation between any sequences



The Problem of Fixed Encoder Representation

Fixed source representation is bad:

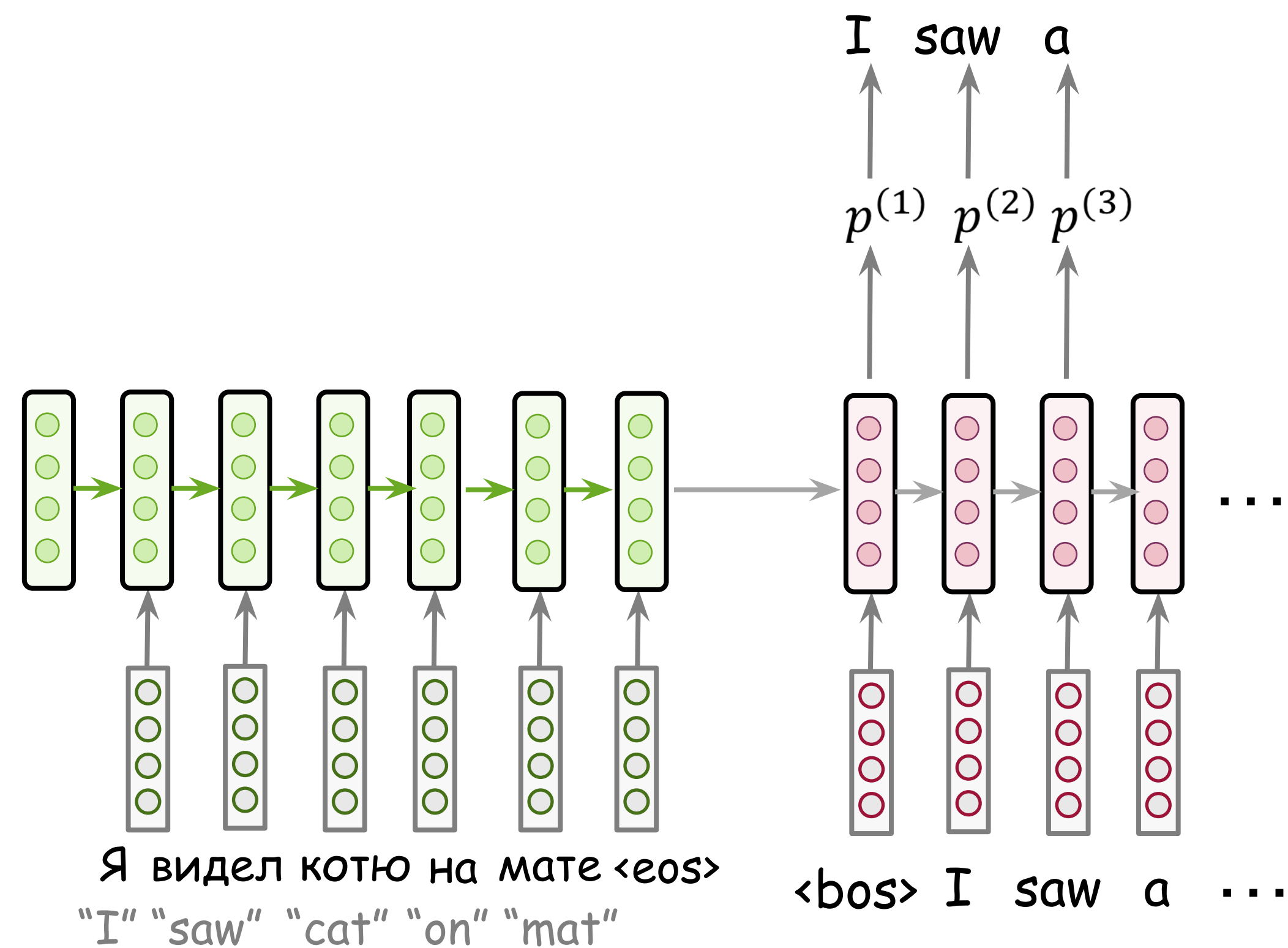
- for **encoder**, it is hard to compress a sentence
- for **decoder**, different information may be needed at different steps



Problem: this is a bottleneck!

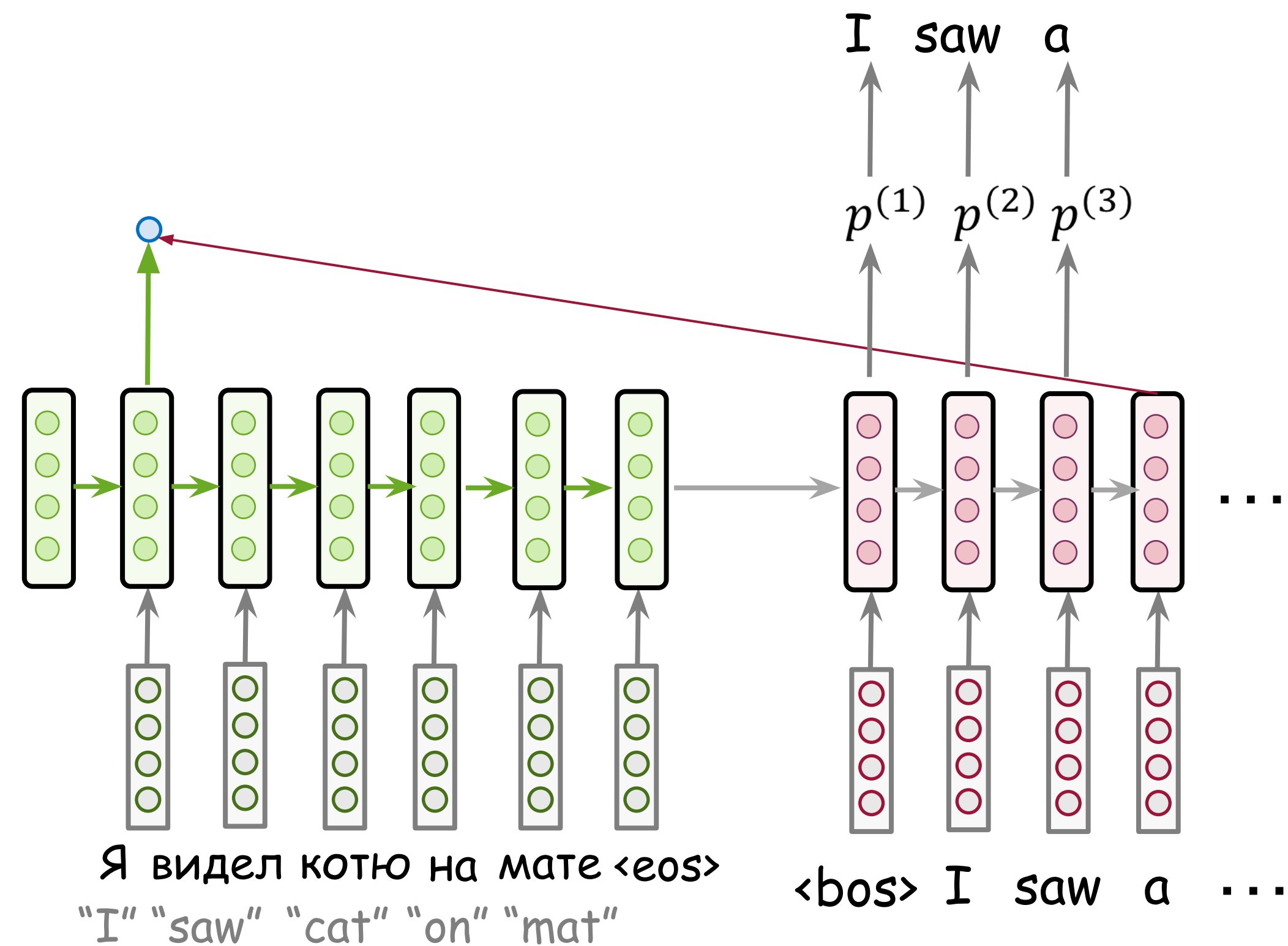
Attention: High-Level View

Attention: At different steps, let a model "focus" on different parts of the input.



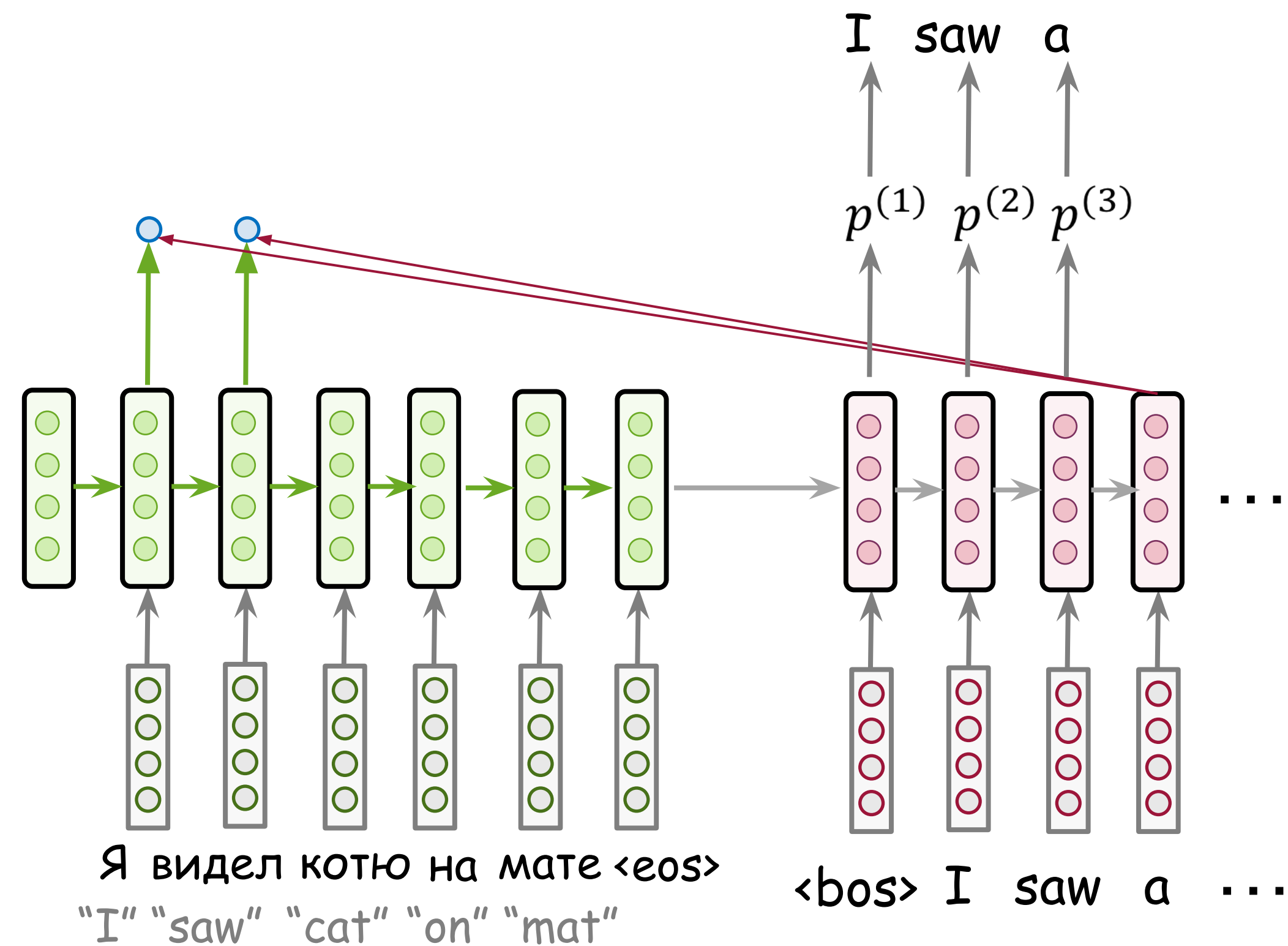
Attention: High-Level View

Attention: At different steps, let a model "focus" on different parts of the input.



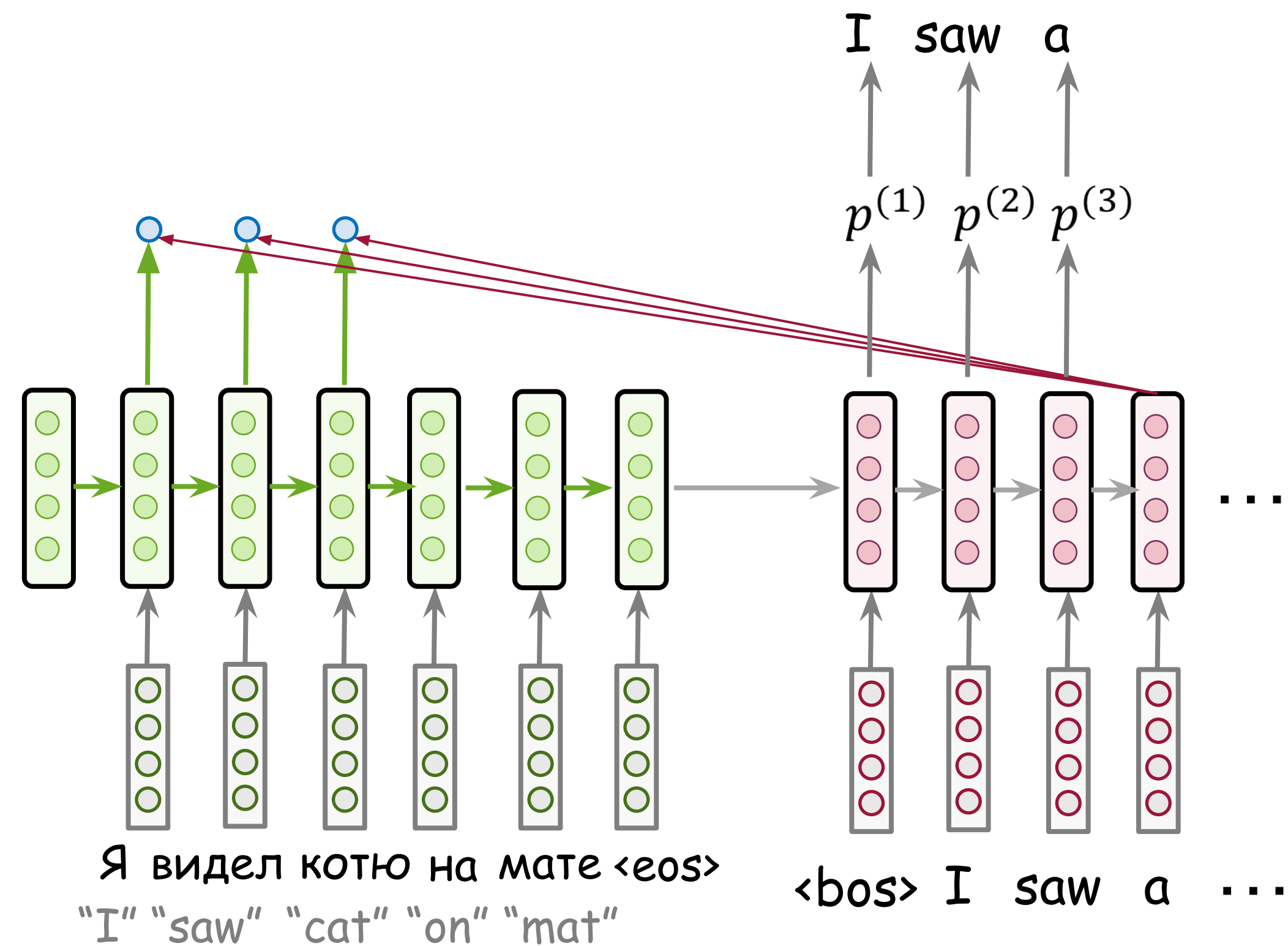
Attention: High-Level View

Attention: At different steps, let a model "focus" on different parts of the input.



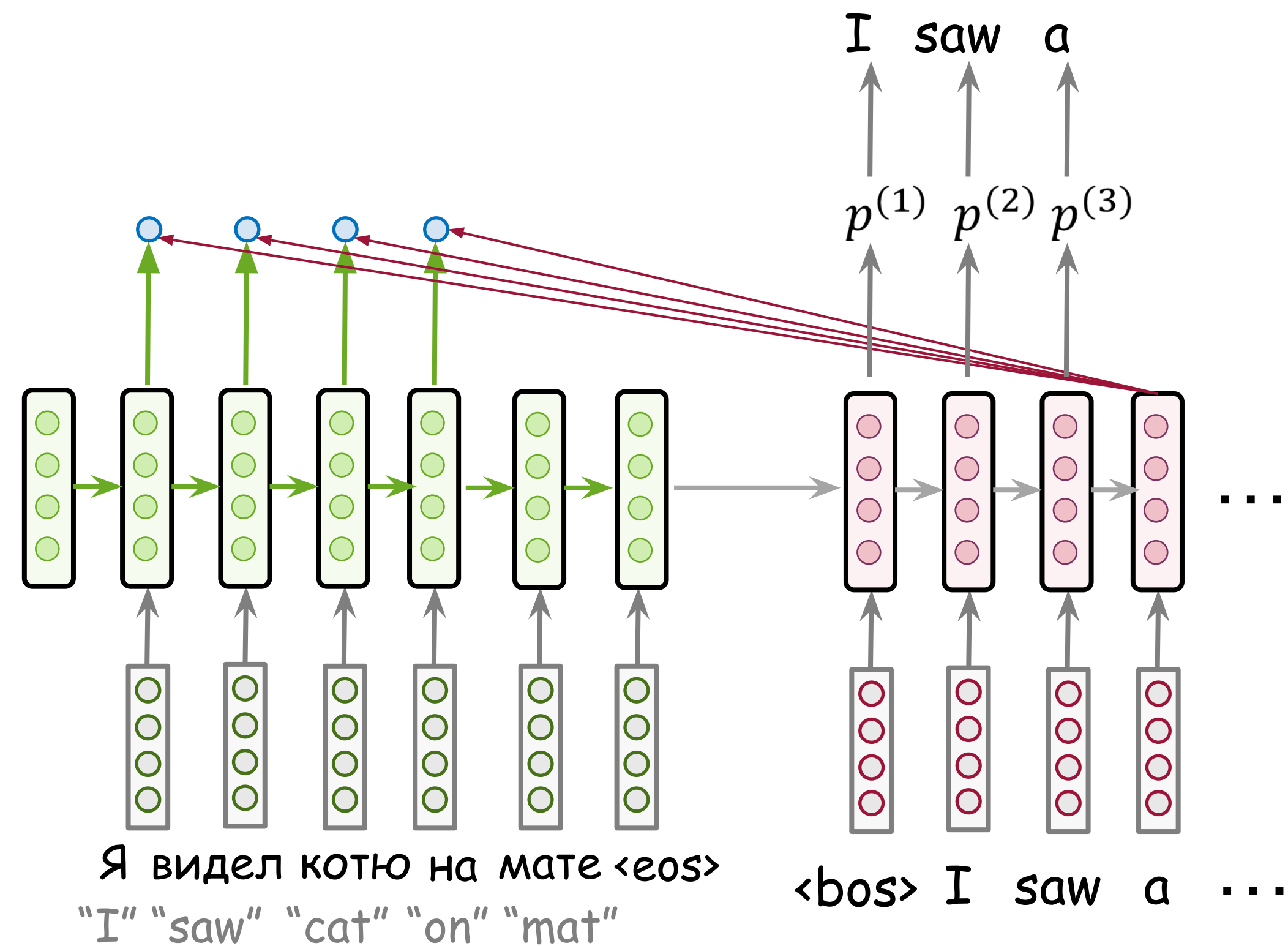
Attention: High-Level View

Attention: At different steps, let a model "focus" on different parts of the input.



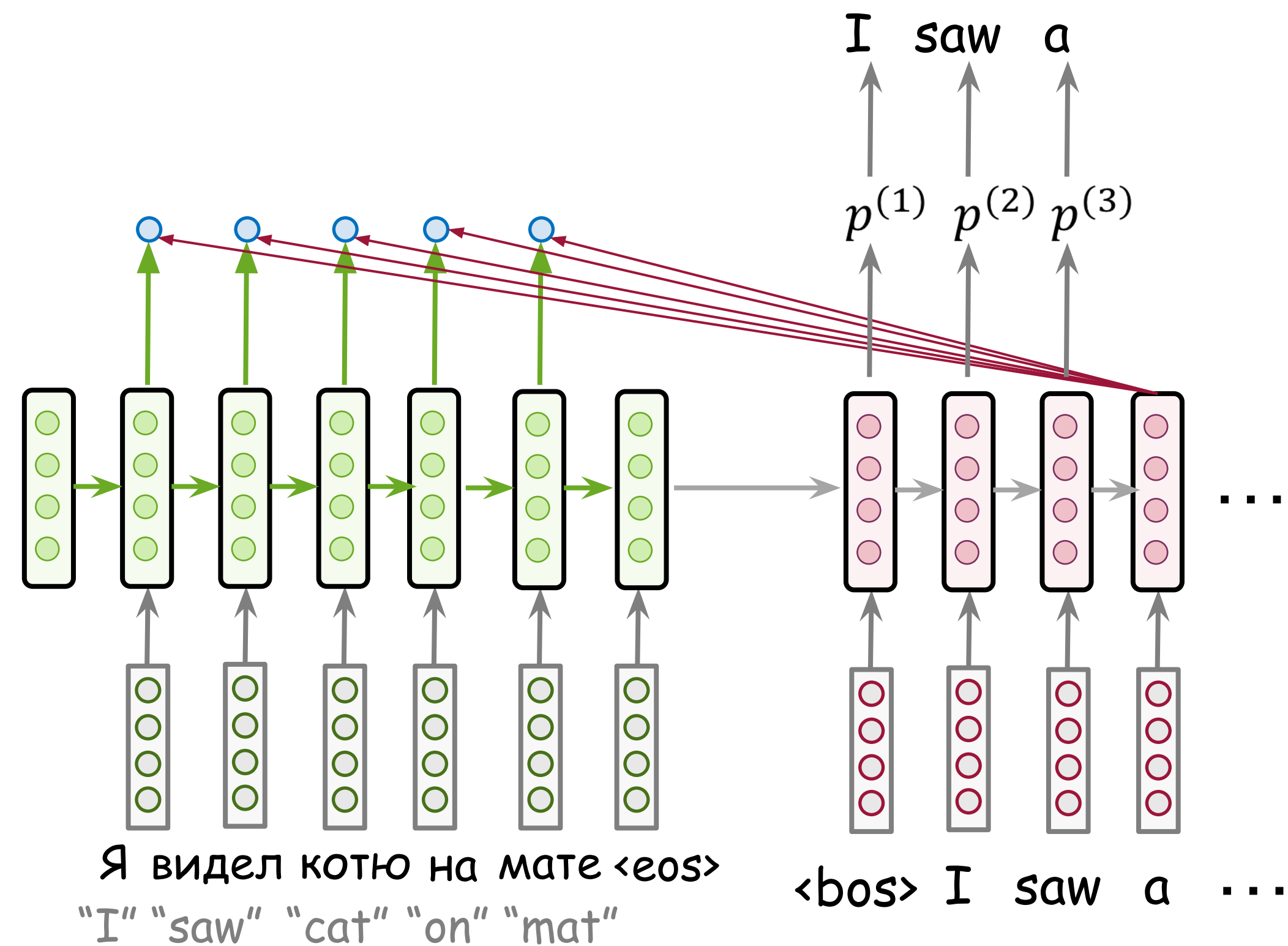
Attention: High-Level View

Attention: At different steps, let a model "focus" on different parts of the input.



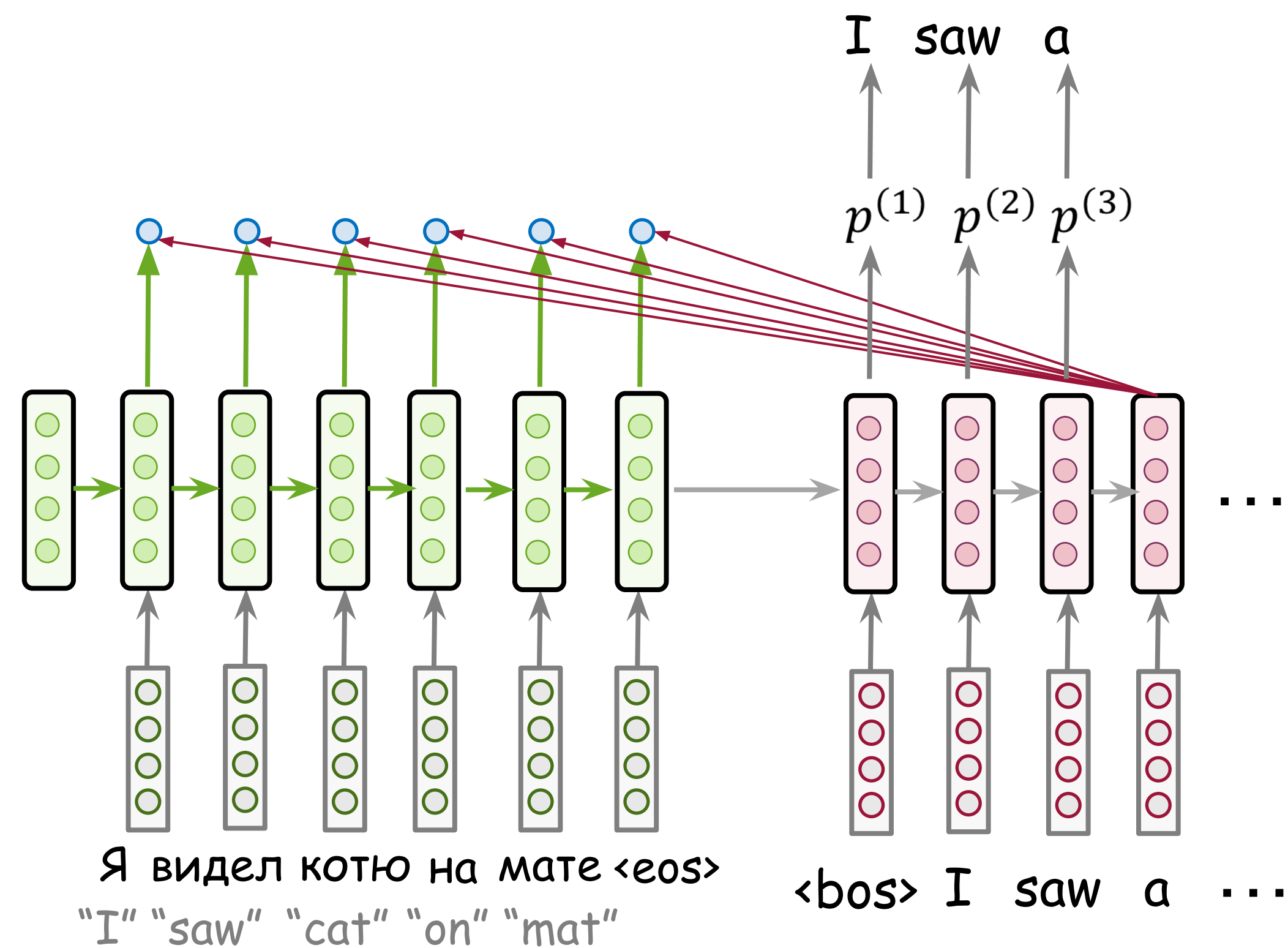
Attention: High-Level View

Attention: At different steps, let a model "focus" on different parts of the input.



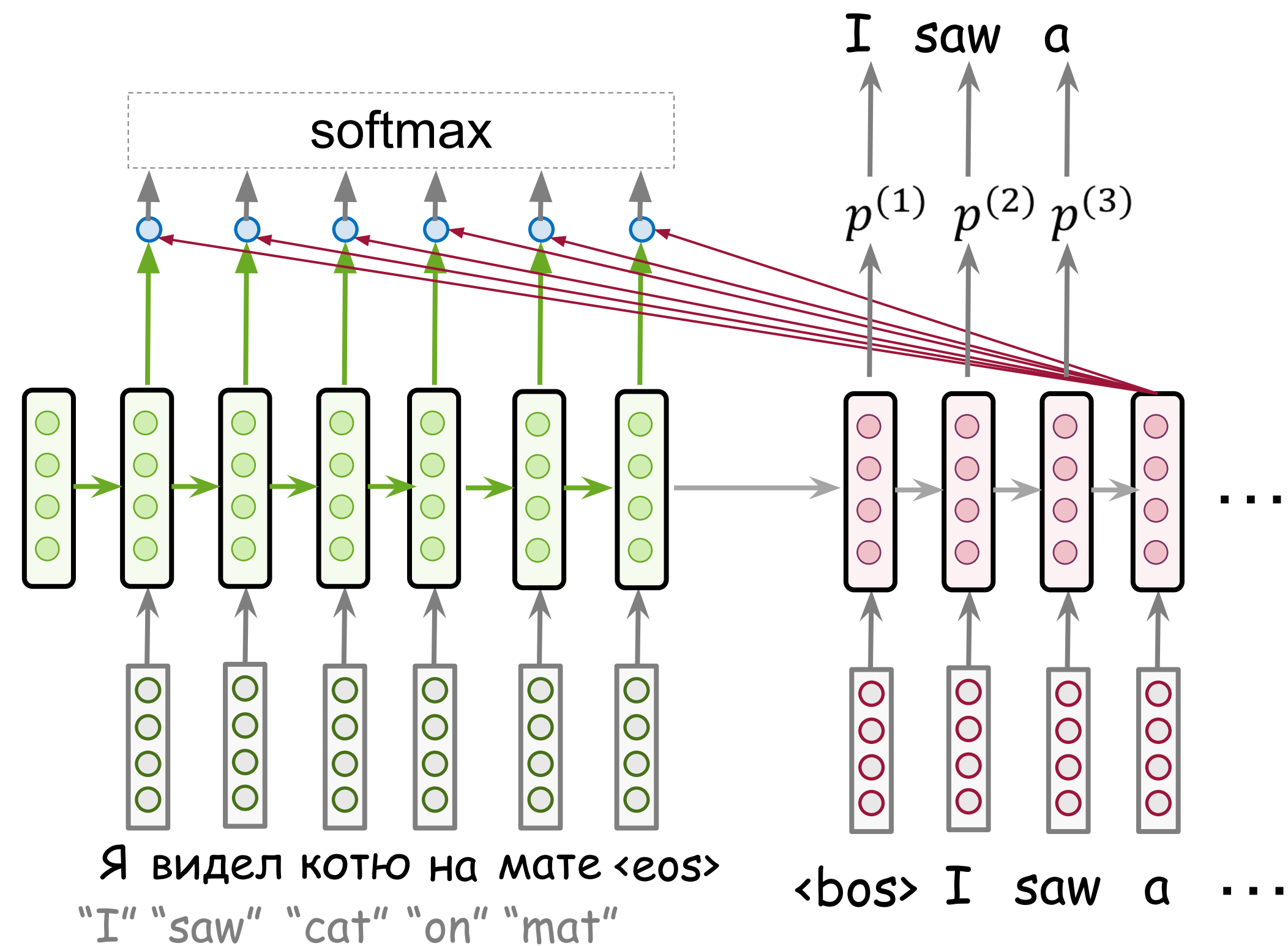
Attention: High-Level View

Attention: At different steps, let a model "focus" on different parts of the input.



Attention: High-Level View

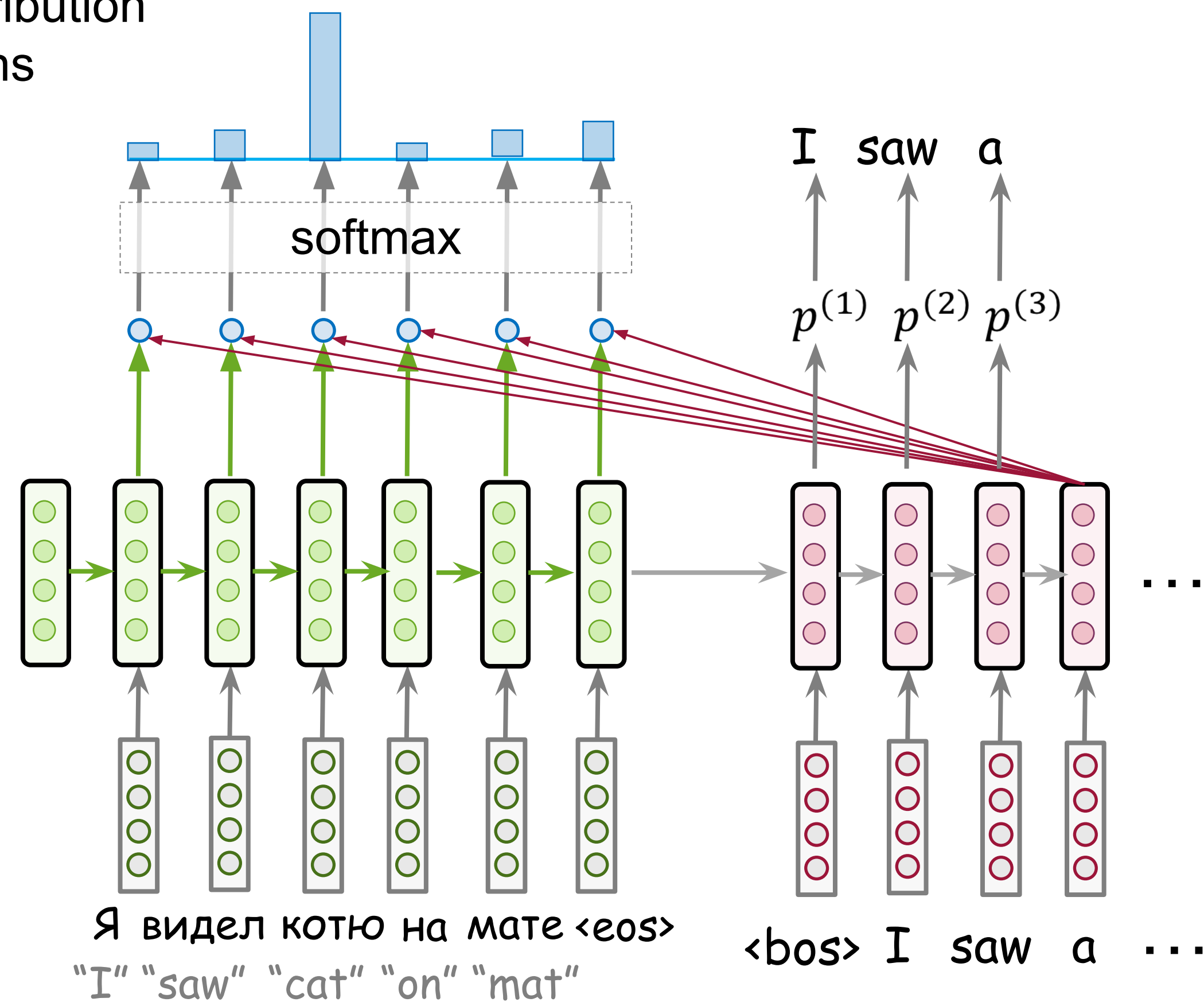
Attention: At different steps, let a model "focus" on different parts of the input.



Attention: High-Level View

Attention: At different steps, let a model "focus" on different parts of the input.

Attention weights: distribution over source tokens

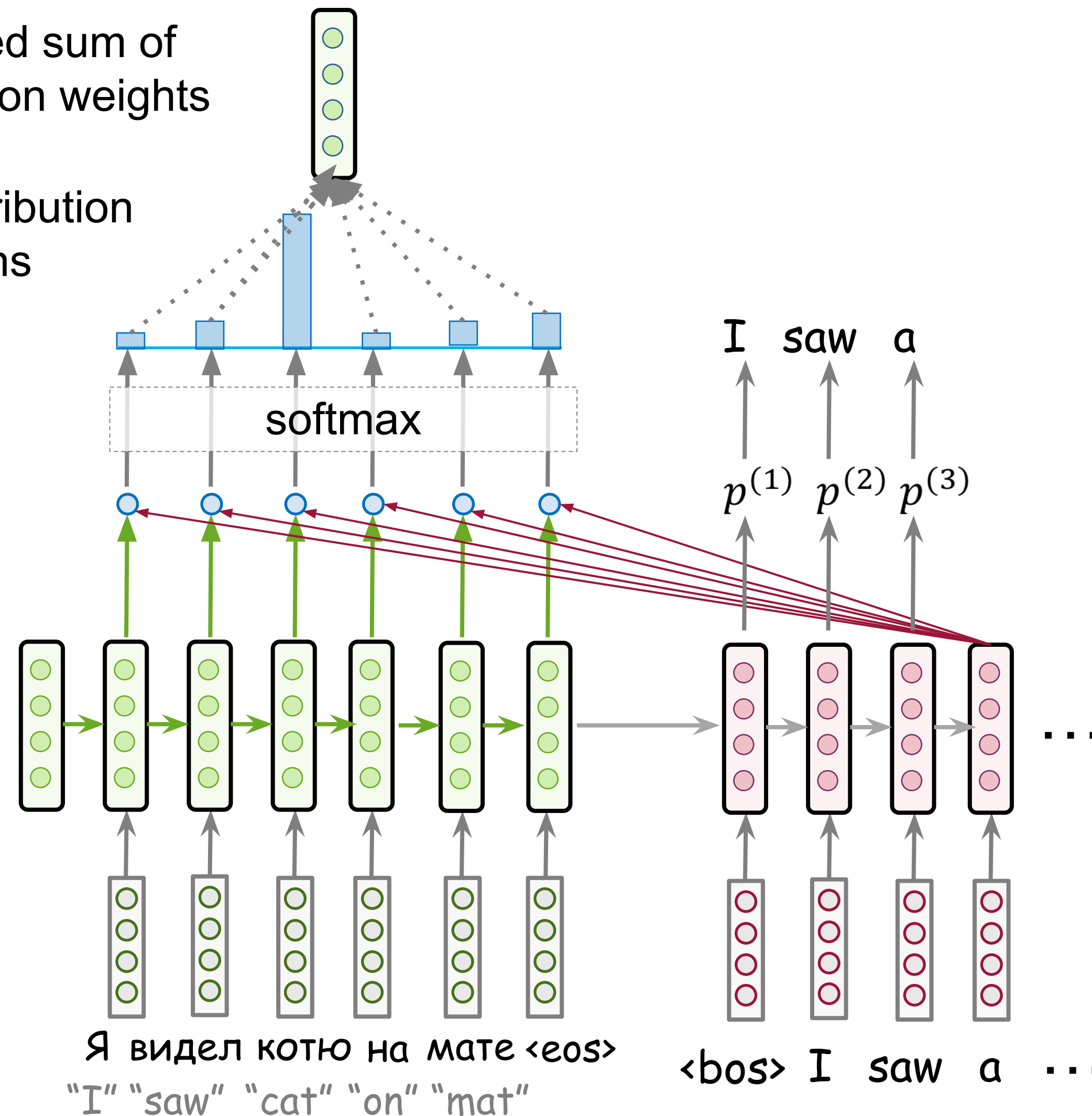


Attention: High-Level View

Attention: At different steps, let a model "focus" on different parts of the input.

Attention output: weighted sum of encoder states with attention weights

Attention weights: distribution over source tokens

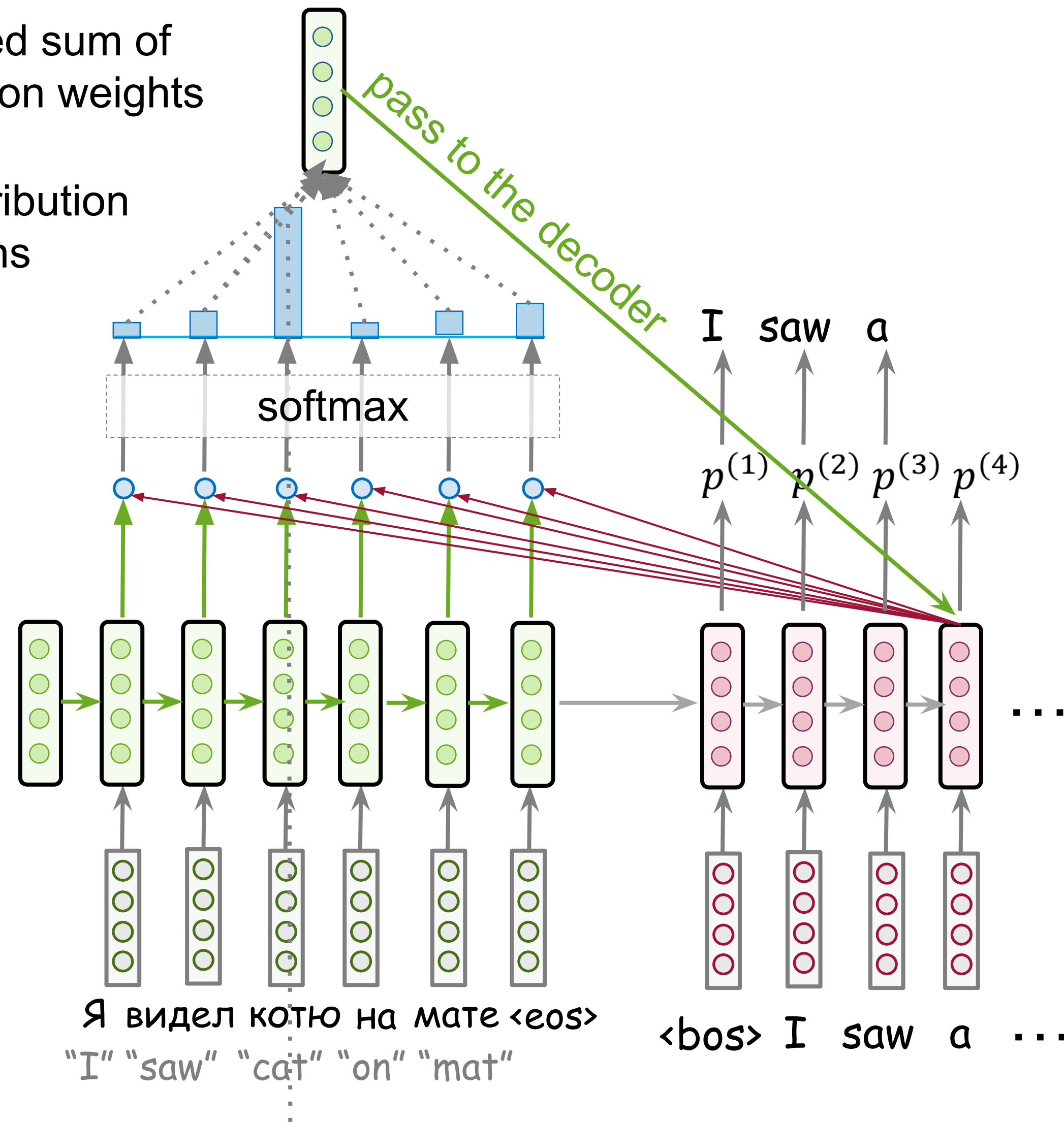


Attention: High-Level View

Attention: At different steps, let a model "focus" on different parts of the input.

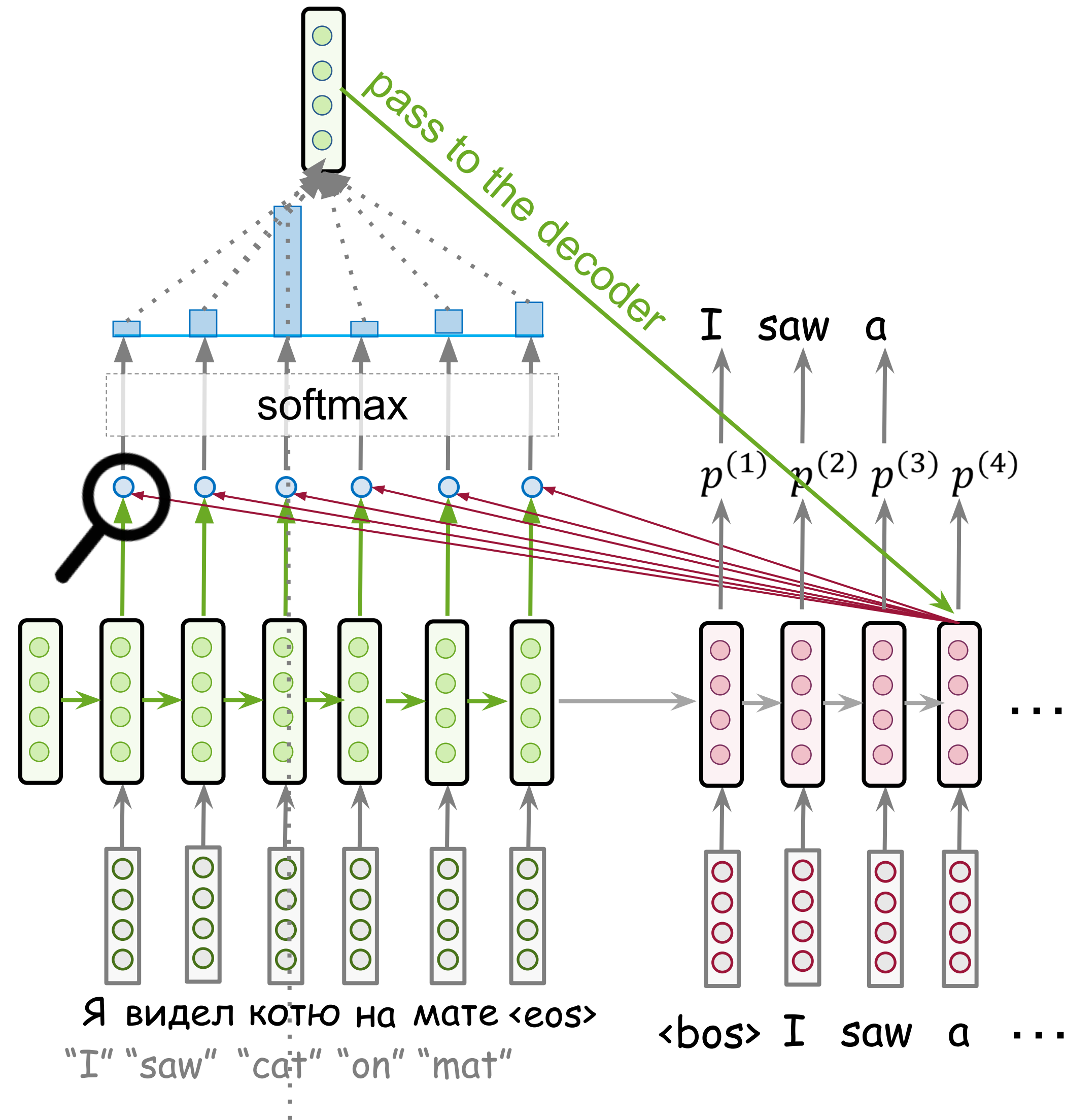
Attention output: weighted sum of encoder states with attention weights

Attention weights: distribution over source tokens



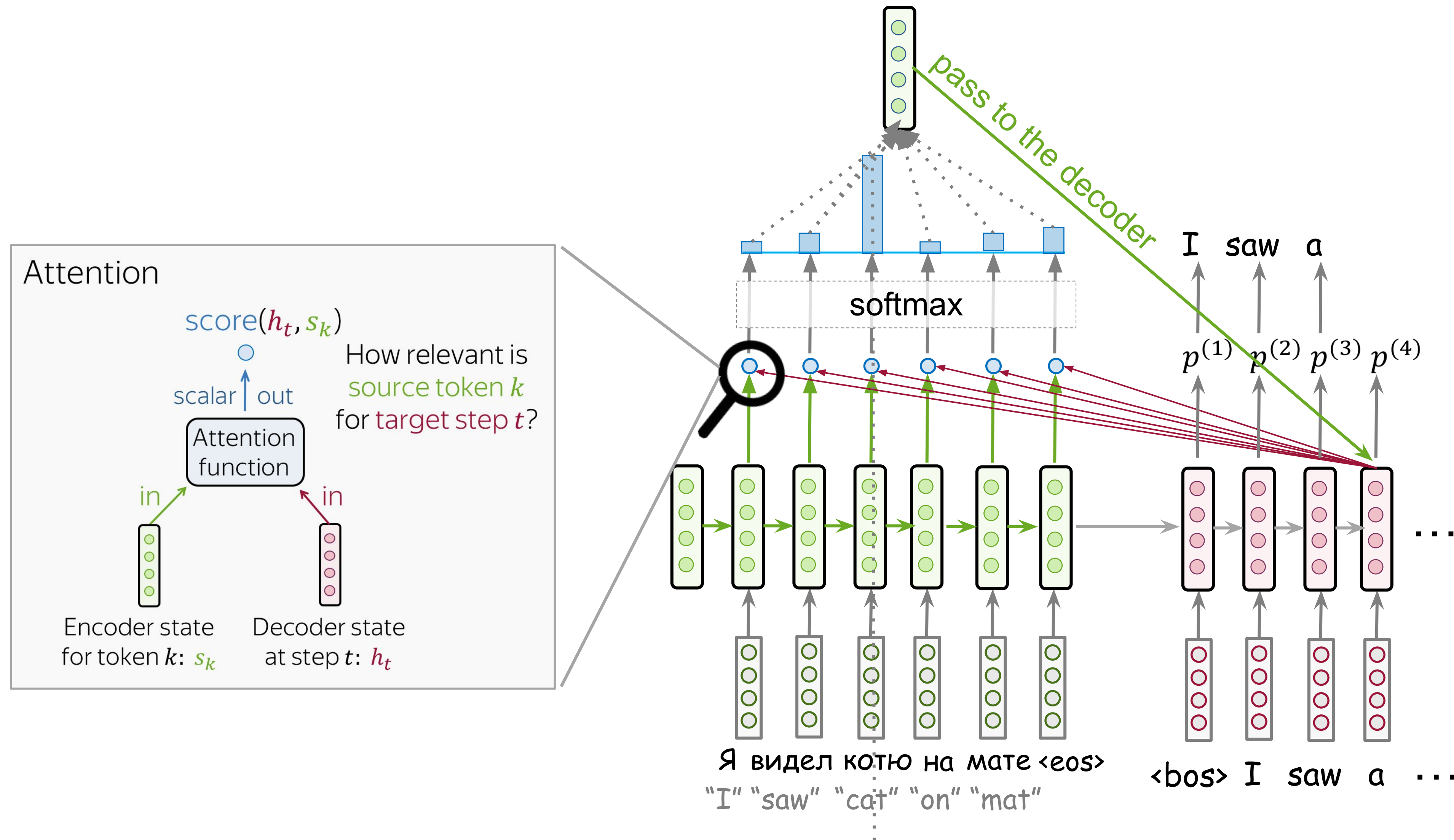
Attention: High-Level View

Attention: At different steps, let a model "focus" on different parts of the input.

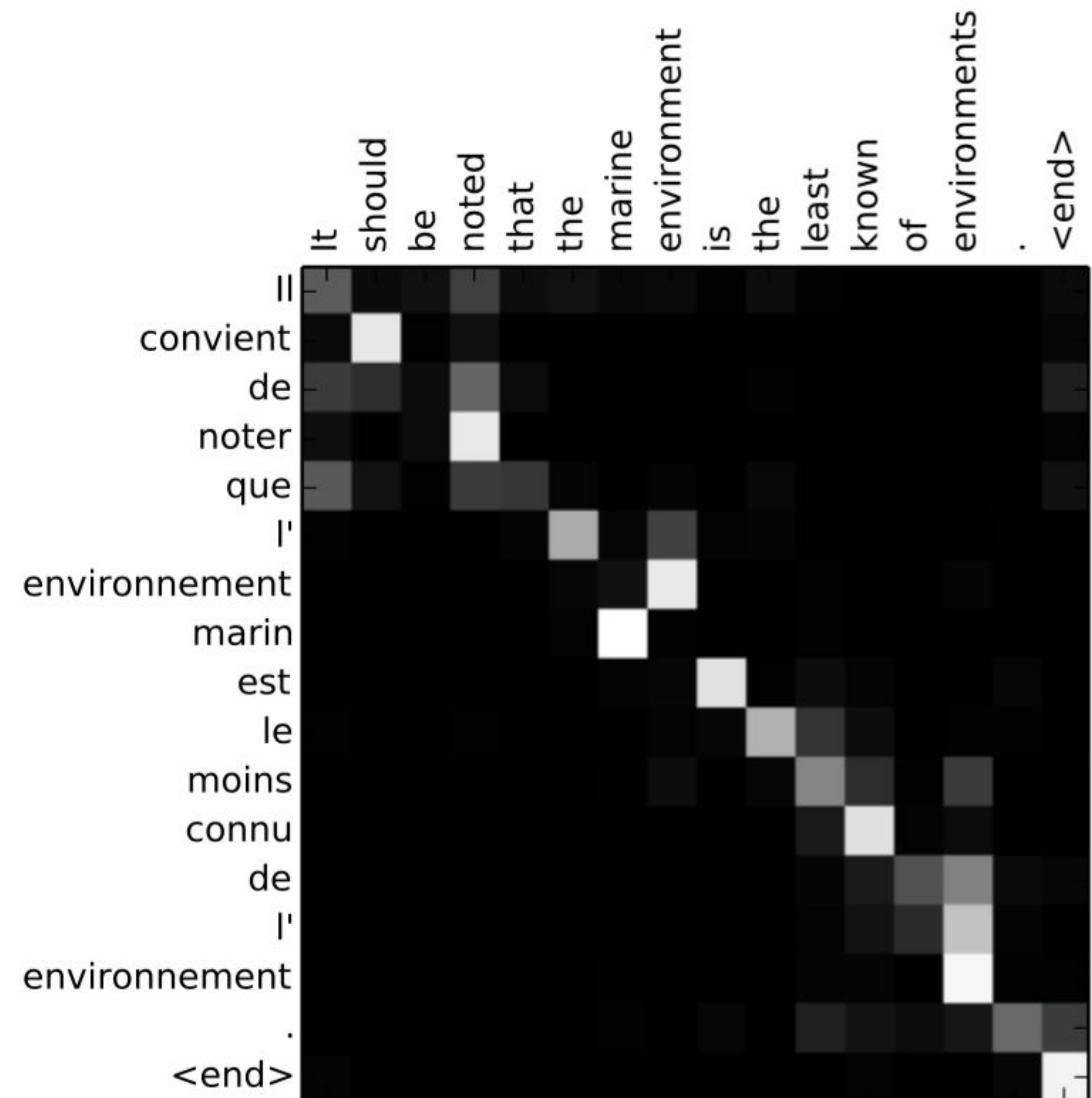
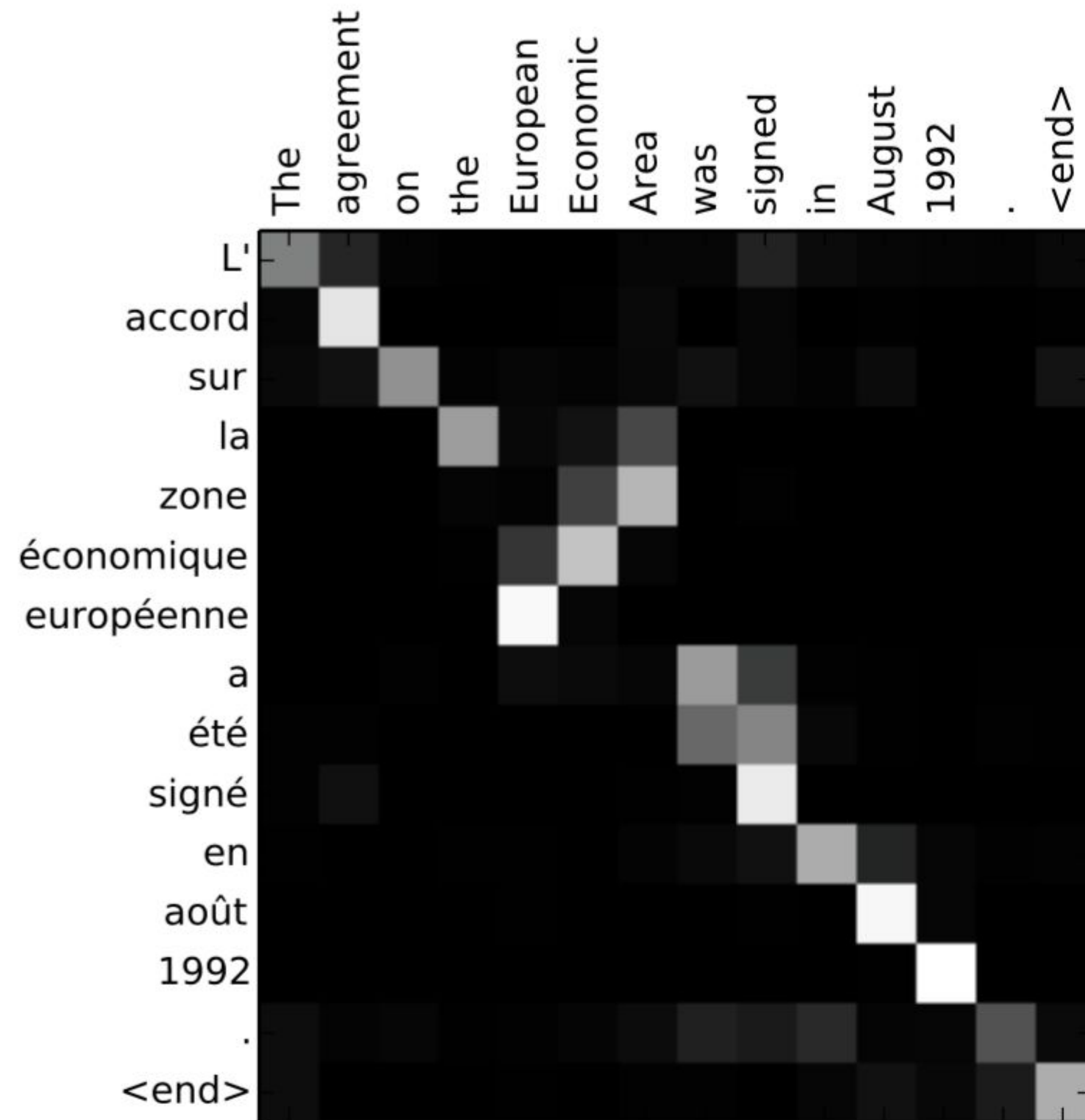


Attention: High-Level View

Attention: At different steps, let a model "focus" on different parts of the input.

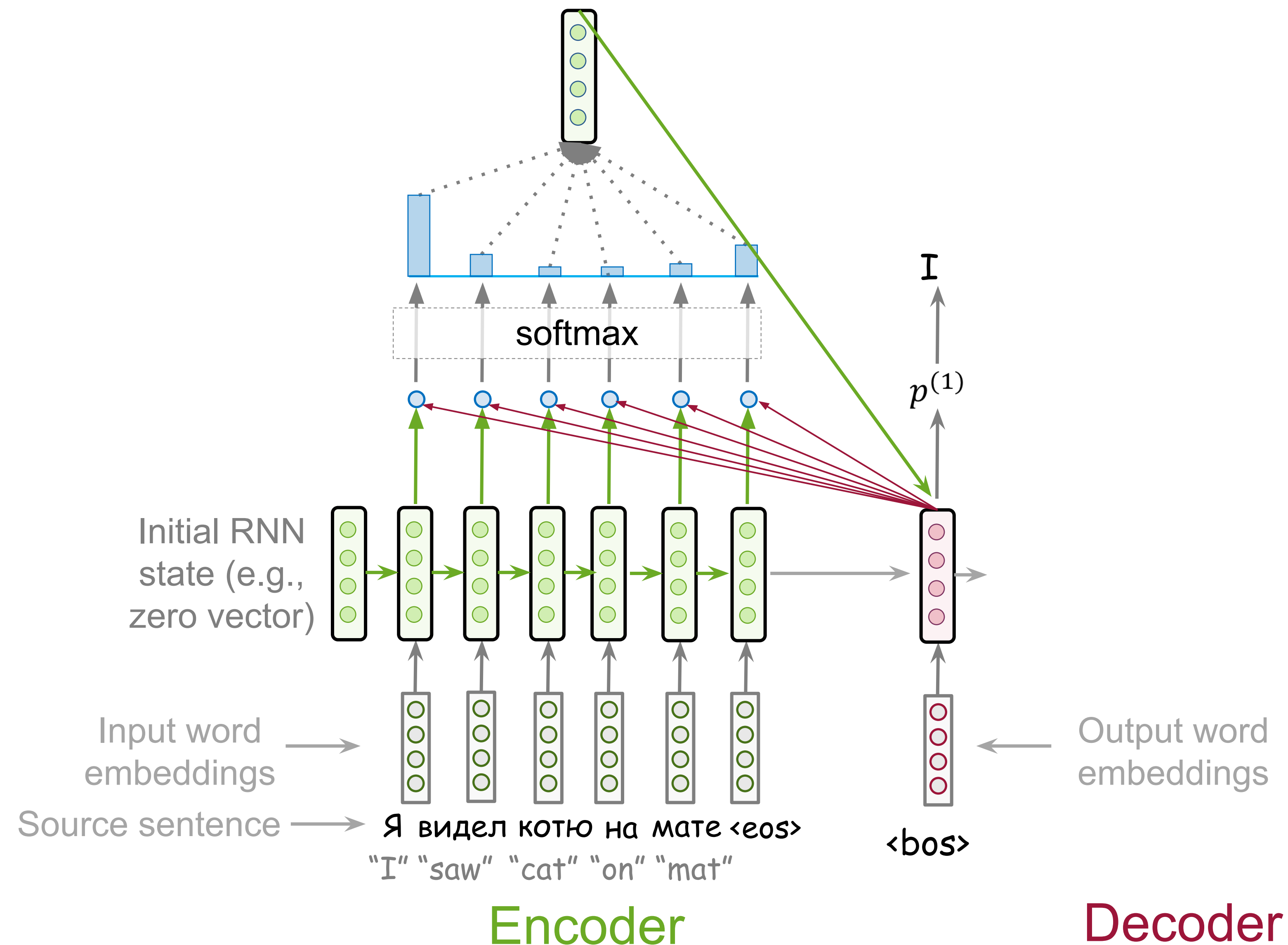


Attention Learned (almost) Alignment

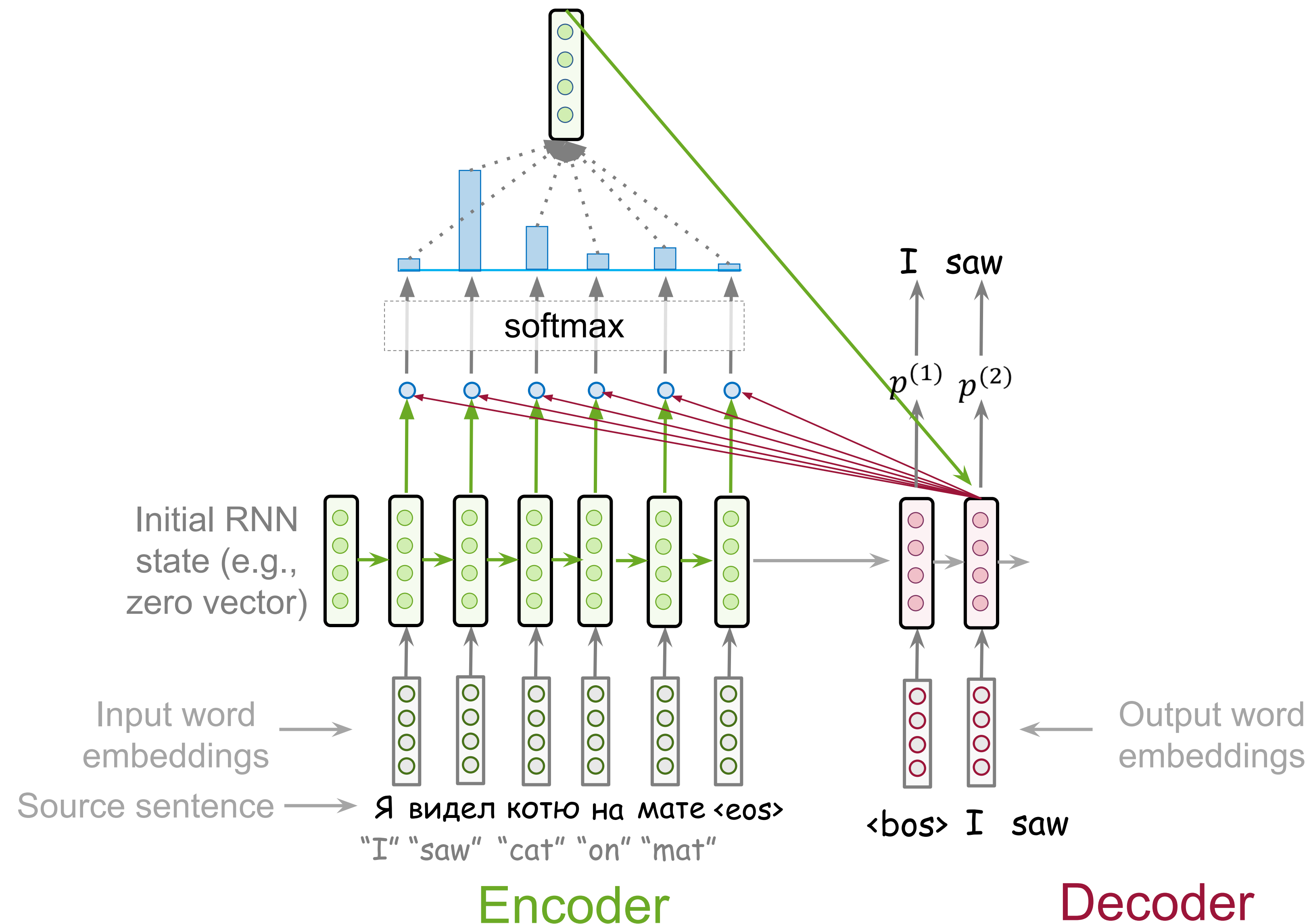


The examples are from the paper [Neural Machine Translation by Jointly Learning to Align and Translate](#).

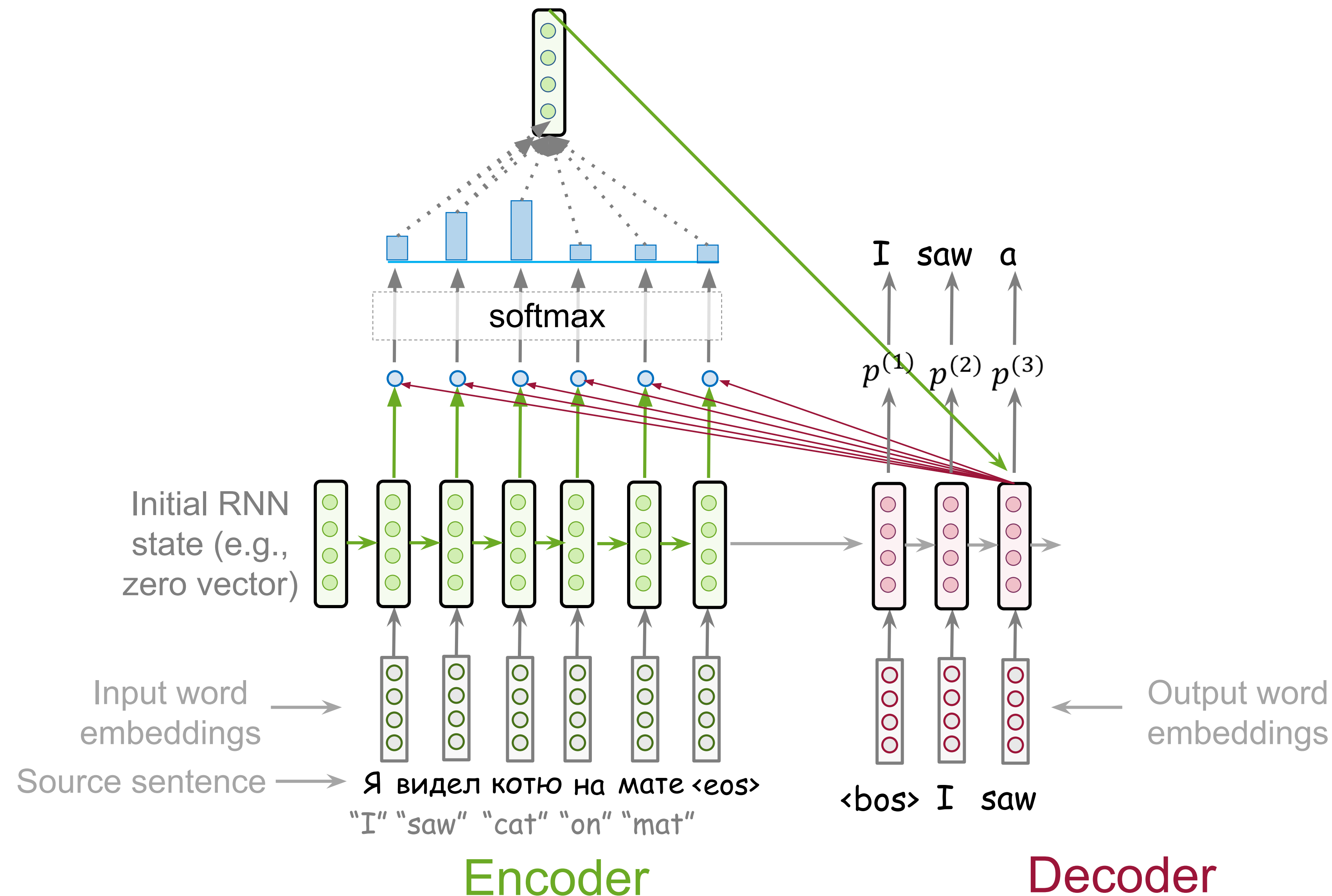
Model Learns to Pick Relevant Tokens



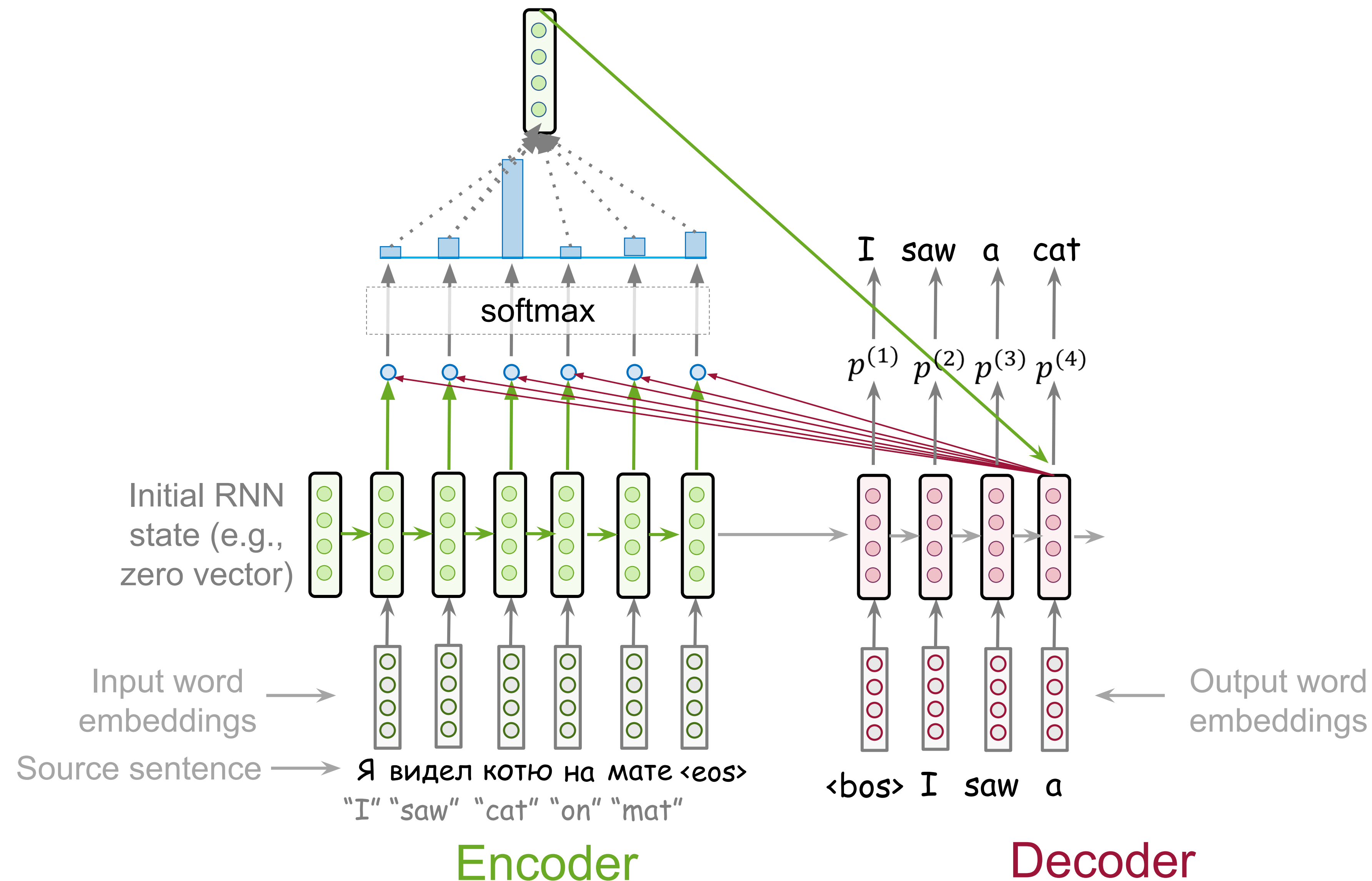
Model Learns to Pick Relevant Tokens



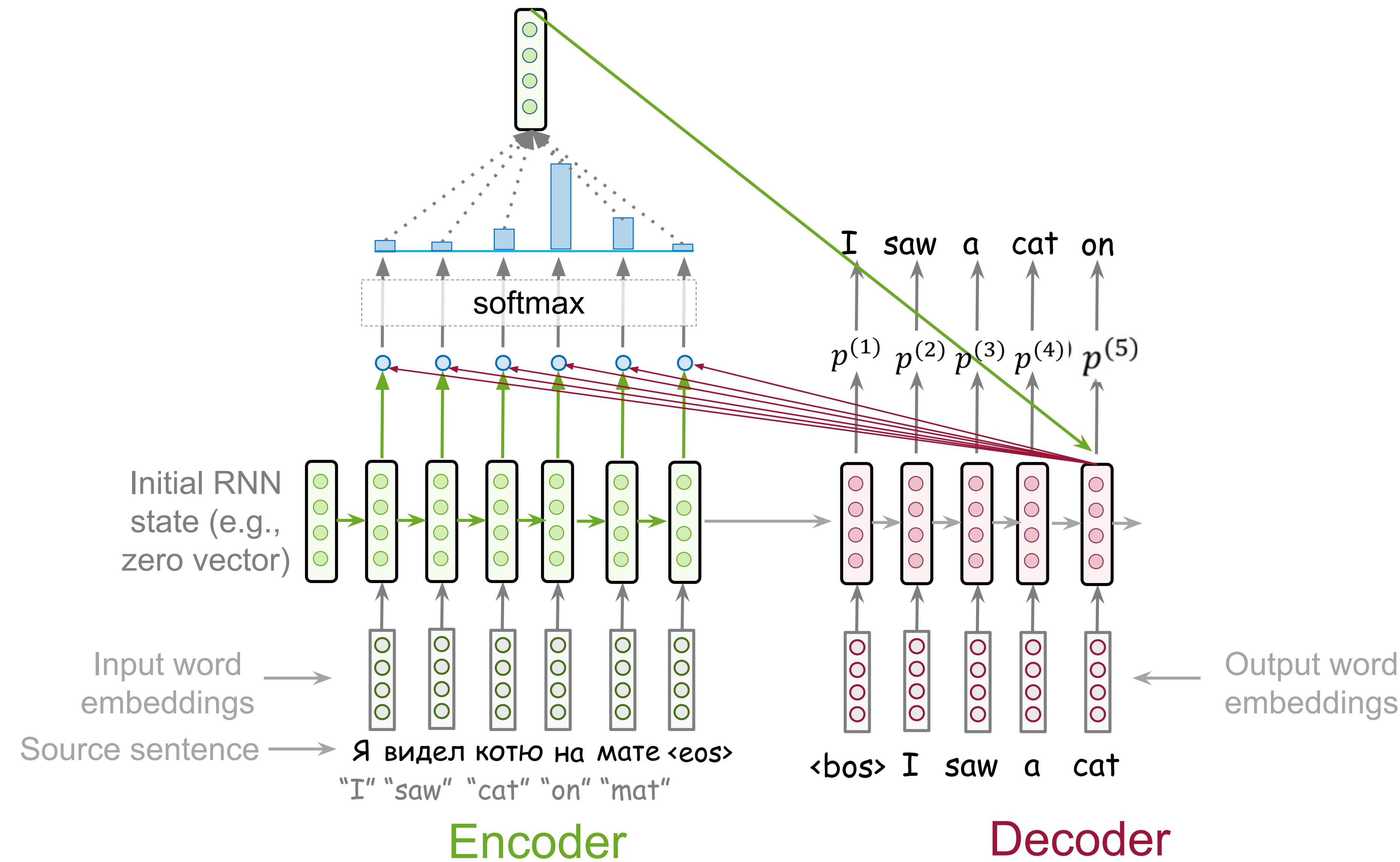
Model Learns to Pick Relevant Tokens



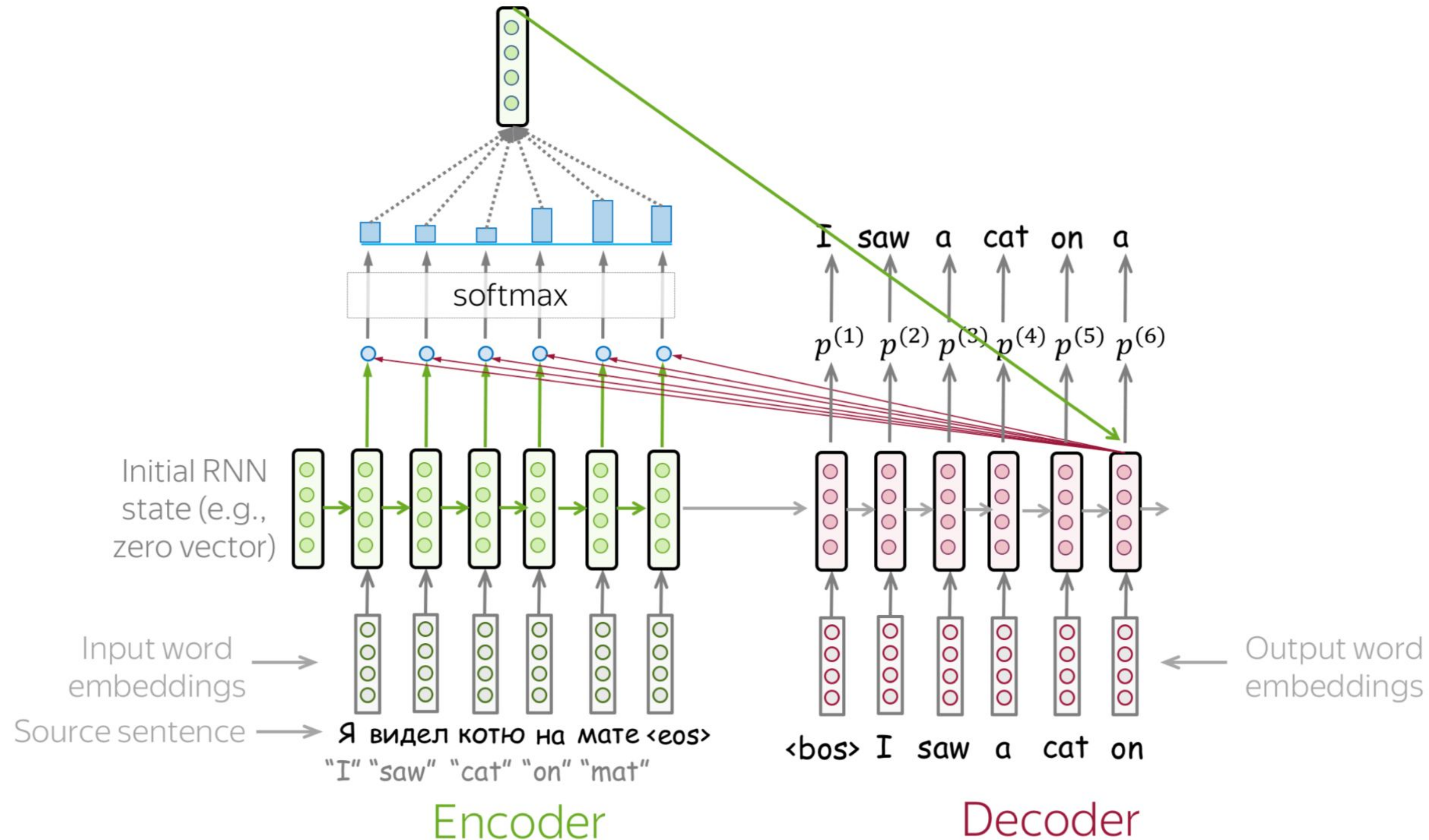
Model Learns to Pick Relevant Tokens



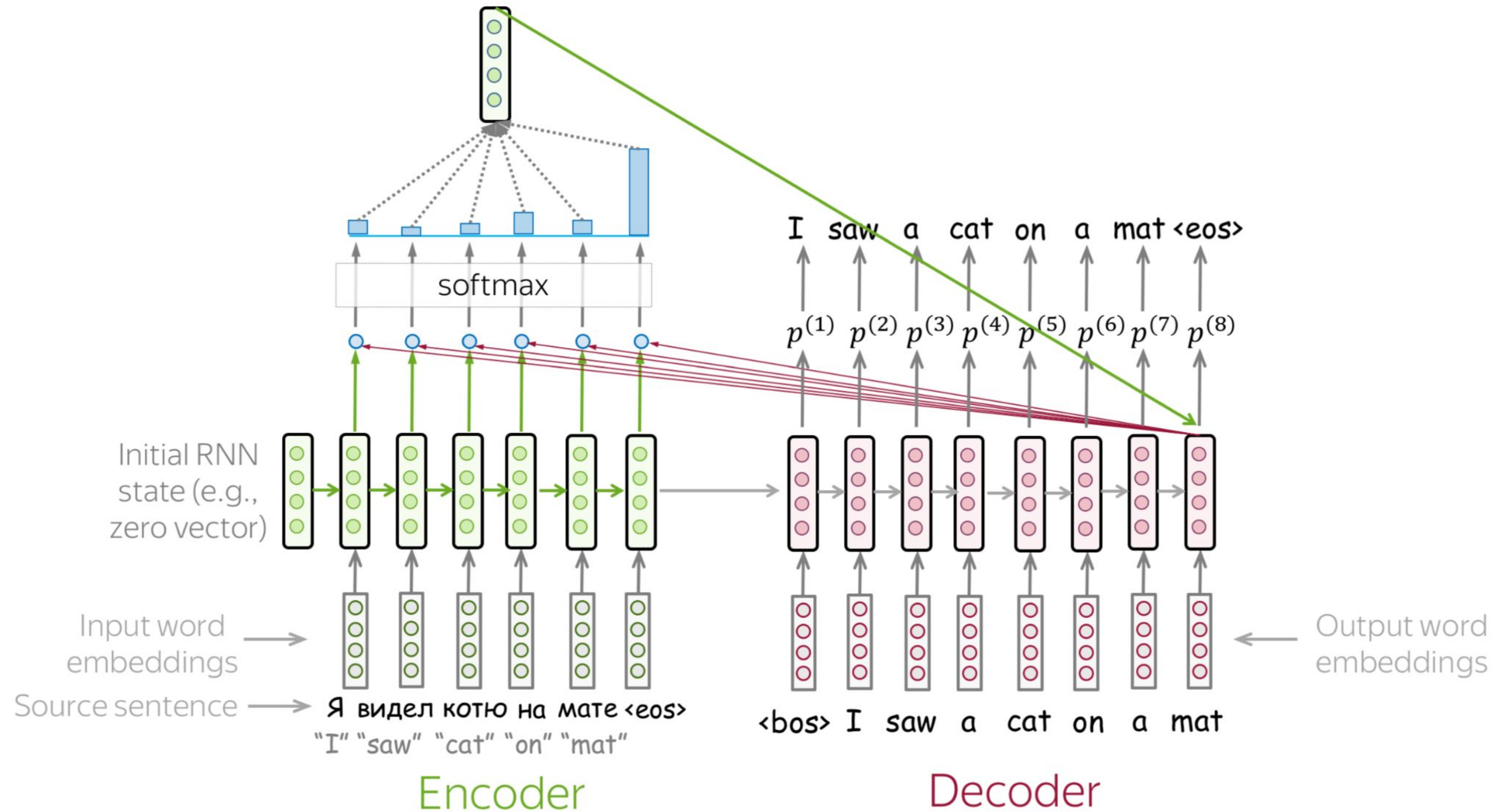
Model Learns to Pick Relevant Tokens



Model Learns to Pick Relevant Tokens



Model Learns to Pick Relevant Tokens



What is going to happen:

- Seq2seq Basics

- Attention

- Transformer



- Transformer LMs

- BERT and family

- Idea

- Self-Attention

- Masked Self-Attention

- Multi-Head Attention

- KV caching

- Model architecture

Idea: Attention is All You Need

	Seq2seq without attention	Seq2seq with attention
processing within encoder	RNN/CNN	RNN/CNN
processing within decoder	RNN/CNN	RNN/CNN
decoder - encoder interaction	static fixed- sized vector	attention

Idea: Attention is All You Need

	Seq2seq without attention	Seq2seq with attention	Transformer
processing within encoder	RNN/CNN	RNN/CNN	attention
processing within decoder	RNN/CNN	RNN/CNN	attention
decoder - encoder interaction	static fixed- sized vector	attention	attention

Idea: Attention is All You Need

The animation is from the [Google AI blog post](#).

Idea: Attention is All You Need

Encoder

Who is doing:

What they are doing:

The animation is from the Google AI blog post.

Idea: Attention is All You Need

Encoder

Who is doing:

- all source tokens

What they are doing:

The animation is from the [Google AI blog post](#).

Idea: Attention is All You Need

Encoder

Who is doing:

- all source tokens

What they are doing:

- look at each other
- update representations

repeat N
times

The animation is from the [Google AI blog post](#).

Idea: Attention is All You Need

Decoder

Who is doing:

What they are doing:

The animation is from the Google AI blog post.

Idea: Attention is All You Need

Decoder

Who is doing:

- target token at the current step

What they are doing:

The animation is from the [Google AI blog post](#).

Idea: Attention is All You Need

Decoder

Who is doing:

- target token at the current step

What they are doing:

- looks at previous target tokens
- looks at source representations
- update representation

repeat N
times

The animation is from the [Google AI blog post](#).

Why can this be better than RNNs?

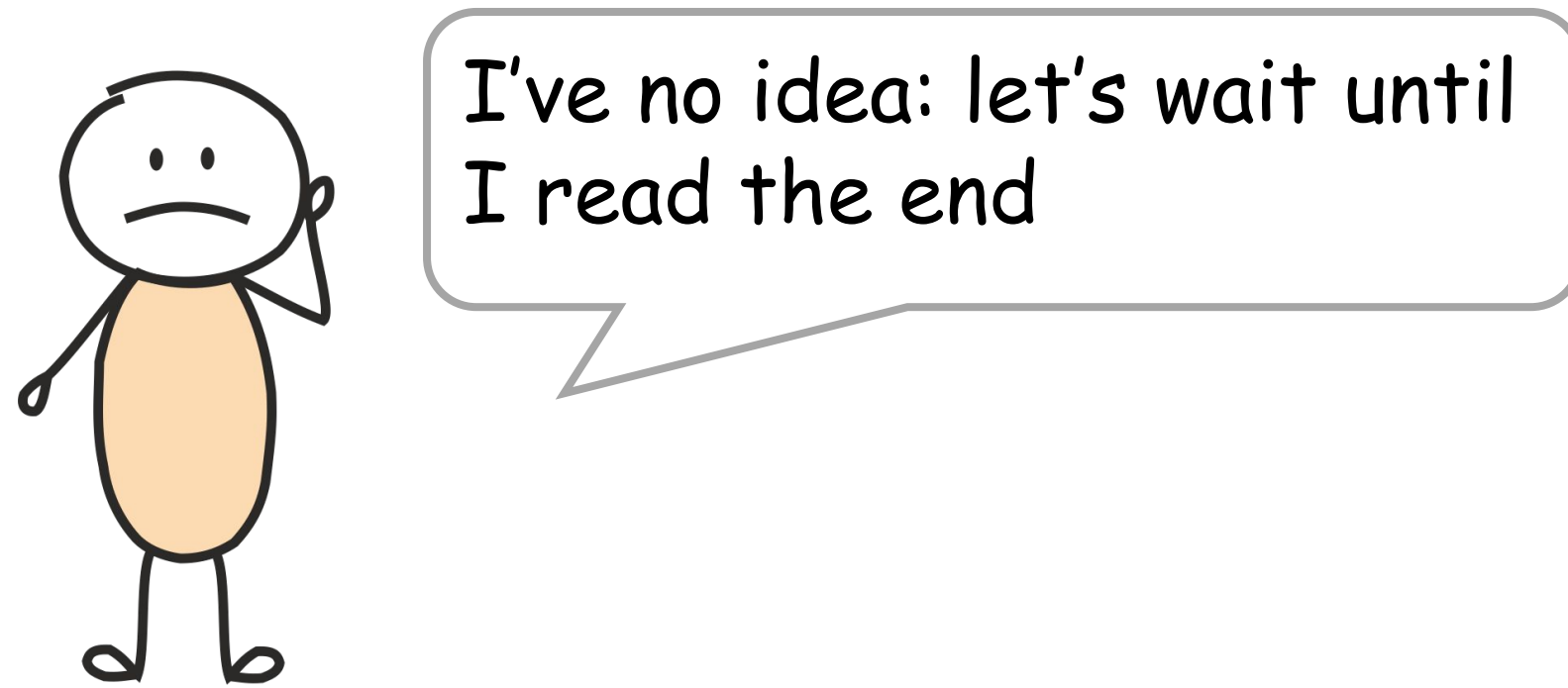
I arrived at the **bank** after crossing thestreet? ...river?

What does bank mean in this sentence?

Why can this be better than RNNs?

I arrived at the **bank** after crossing thestreet? ...river?

What does bank mean in this sentence?



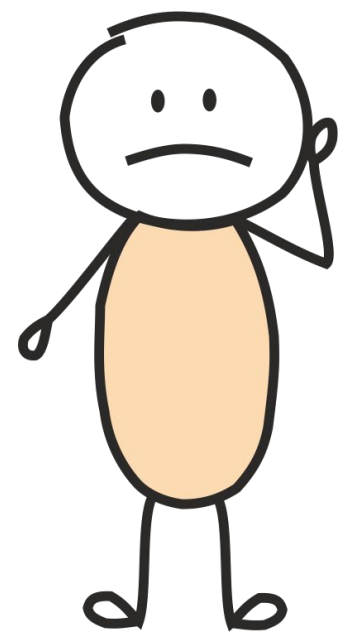
RNNs

$O(N)$ steps to process a
sentence with length N

Why can this be better than RNNs?

I arrived at the **bank** after crossing thestreet? ...river?

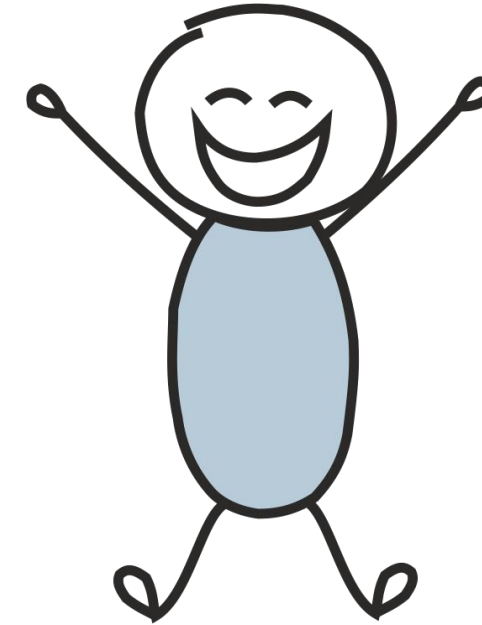
What does bank mean in this sentence?



I've no idea: let's wait until
I read the end

RNNs

$O(N)$ steps to process a
sentence with length N



I don't need to wait - I see
all words at once!

Transformer

Constant number of steps
to process any sentence

What is going to happen:

- Seq2seq Basics
- Attention
- Transformer
- Transformer LMs
- BERT and family



- Idea
- Self-Attention
- Masked Self-Attention
- Multi-Head Attention
- KV caching
- Model architecture

What is going to happen:

- Seq2seq Basics
- Attention
- Transformer
- Transformer LMs
- BERT and family



- Idea
- Self-Attention
- Masked Self-Attention
- Multi-Head Attention
- KV caching
- Model architecture

Self-Attention: Why “Self”?

Decoder-encoder attention is looking

- **from:** one current decoder state
- **at:** all encoder states

Self-Attention: Why “Self”?

Decoder-encoder attention is looking

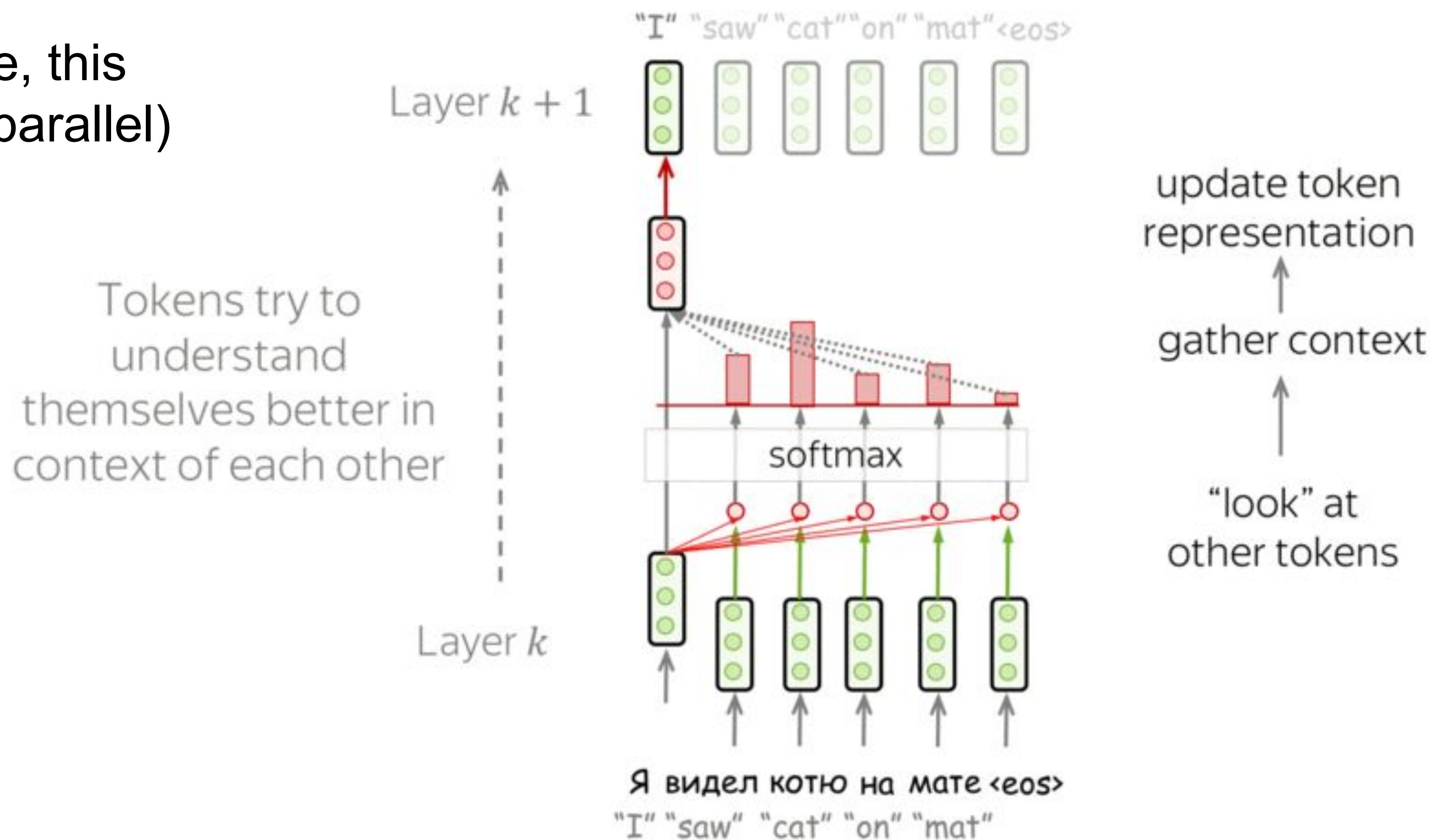
- **from:** one current decoder state
- **at:** all encoder states

Self-attention is looking

- **from:** each state from a set of states
- **at:** all other states in the same set

Self-Attention: The “Look at Each Other” Part

(in practice, this happens in parallel)



Query, Key, Value

Each vector receives three representations (“roles”)

$$\begin{bmatrix} W_Q \end{bmatrix} \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{blue} \\ \text{blue} \\ \text{blue} \end{bmatrix} \quad \text{Query: vector from which the attention is looking}$$

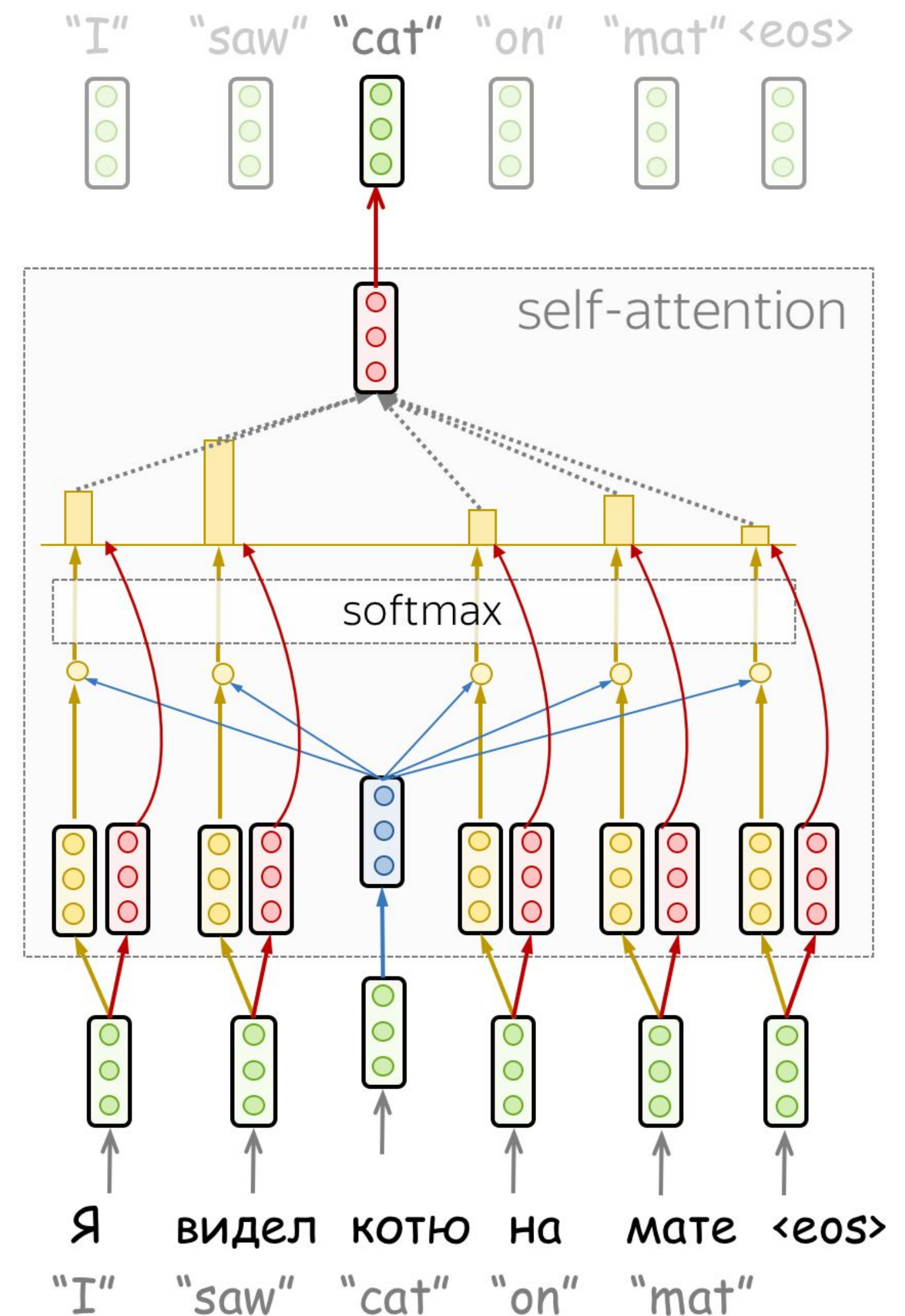
“Hey there, do you have this information?”

$$\begin{bmatrix} W \end{bmatrix} \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{yellow} \\ \text{yellow} \\ \text{yellow} \end{bmatrix} \quad \text{Key: vector at which the query looks to compute weights}$$

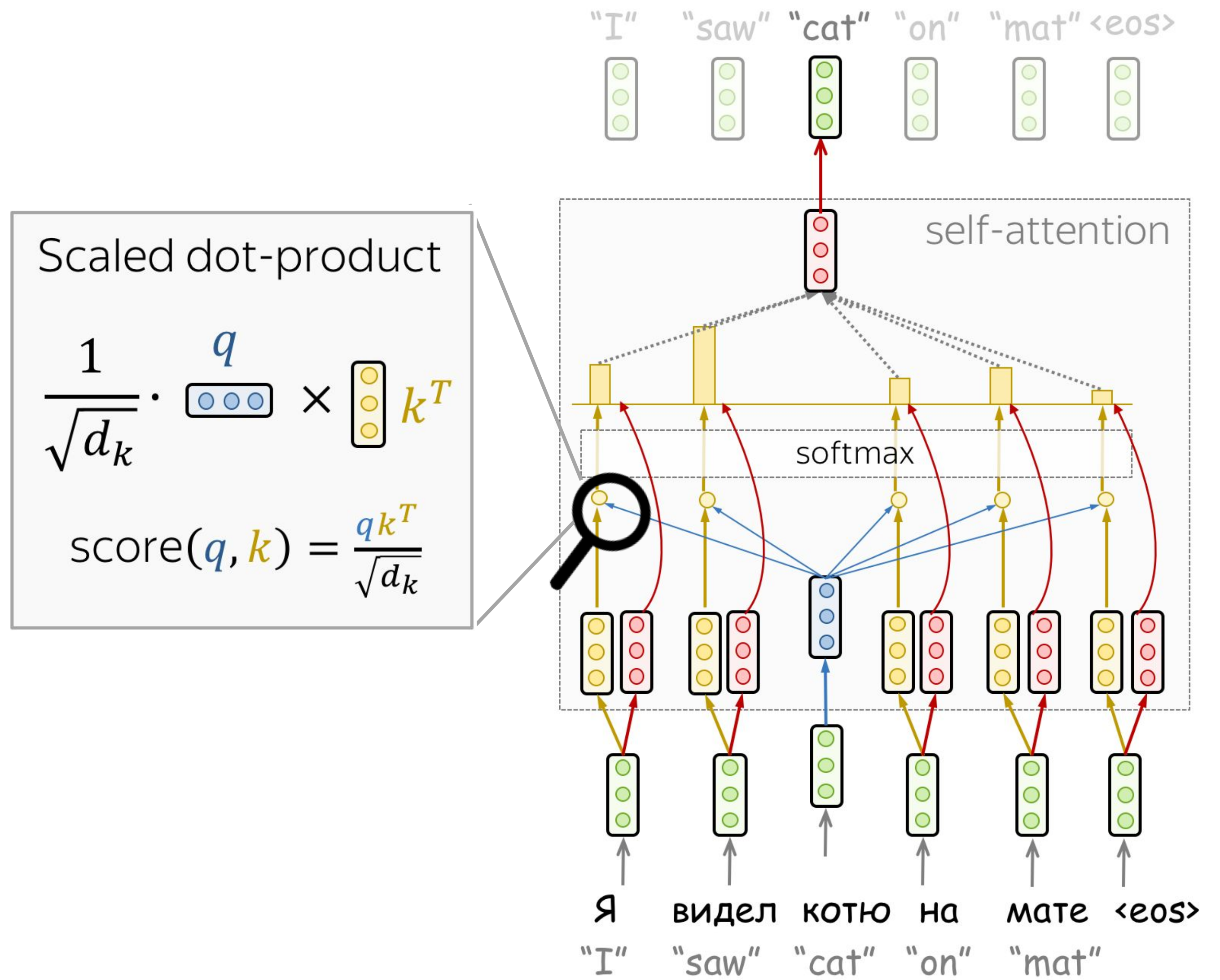
“Hi, I have this information – give me a large weight!”

$$\begin{bmatrix} W \end{bmatrix} \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{red} \\ \text{red} \\ \text{red} \end{bmatrix} \quad \text{Value: their weighted sum is attention output}$$

“Here’s the information I have!”



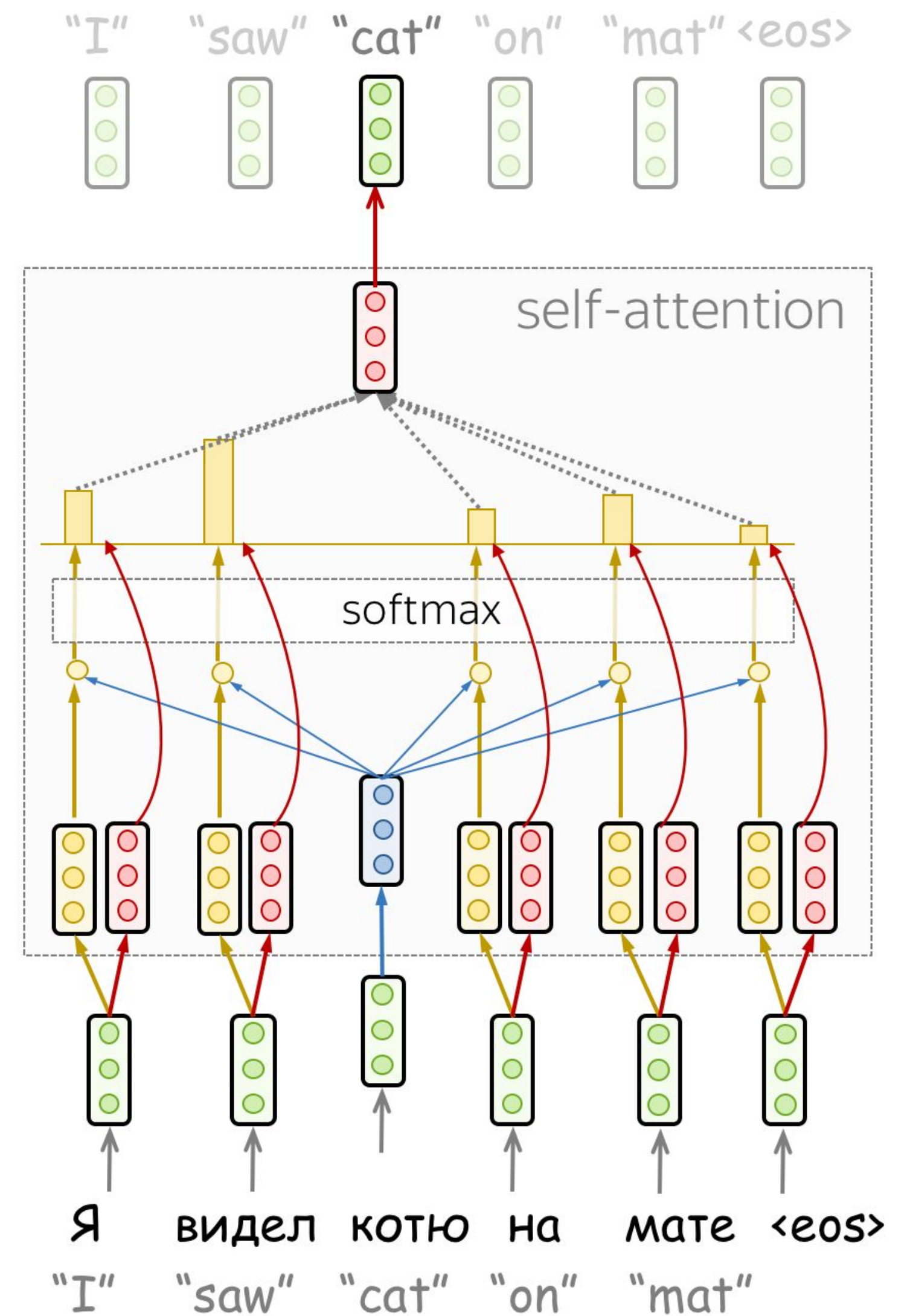
Query, Key, Value



Query, Key, Value

$$Attention(\underset{\substack{\text{from} \\ \text{blue}}}{q}, \underset{\substack{\text{to} \\ \text{red}}}{k}, v) = \overbrace{\text{softmax}\left(\frac{qk^T}{\sqrt{d_k}}\right)}^{\text{Attention weights}} v$$

vector dimensionality of K, V



$\sqrt{d_k}$?

Assumption: $0 = \mathbb{E}[k_i] = \mathbb{E}[q_i]$, $1 = \text{Var}[k_i] = \text{Var}[q_i]$

Can we assume this?

$$\begin{aligned}\text{Var}[XY] &= \mathbb{E}[X^2Y^2] - \mathbb{E}^2[XY] \stackrel{\text{why?}}{=} \mathbb{E}[X^2]\mathbb{E}[Y^2] - \mathbb{E}^2[X]\mathbb{E}^2[Y] \\ &= \text{Var}[X]\text{Var}[Y] + \mathbb{E}[X^2]\mathbb{E}^2[Y] + \mathbb{E}^2[X]\mathbb{E}[Y^2] - 2\mathbb{E}^2[X]\mathbb{E}^2[Y] \\ &= \text{Var}[X]\text{Var}[Y] + \text{Var}[X]\mathbb{E}^2[Y] + \text{Var}[Y]\mathbb{E}^2[X]\end{aligned}$$

Hence

$$\text{Var}[k^T q] = \sum \text{Var}[k_i q_i] = \sum \text{Var}[k_i] \text{Var}[q_i] = d_k$$

What is going to happen:

- Seq2seq Basics
- Attention
- Transformer →
- Subword Segmentation: BPE
-  Analysis and Interpretability

- Idea
- Self-Attention
- Masked Self-Attention
- Multi-Head Attention
- KV caching
- Model architecture

What is going to happen:

- Seq2seq Basics

- Attention

- Transformer



- Subword Segmentation: BPE

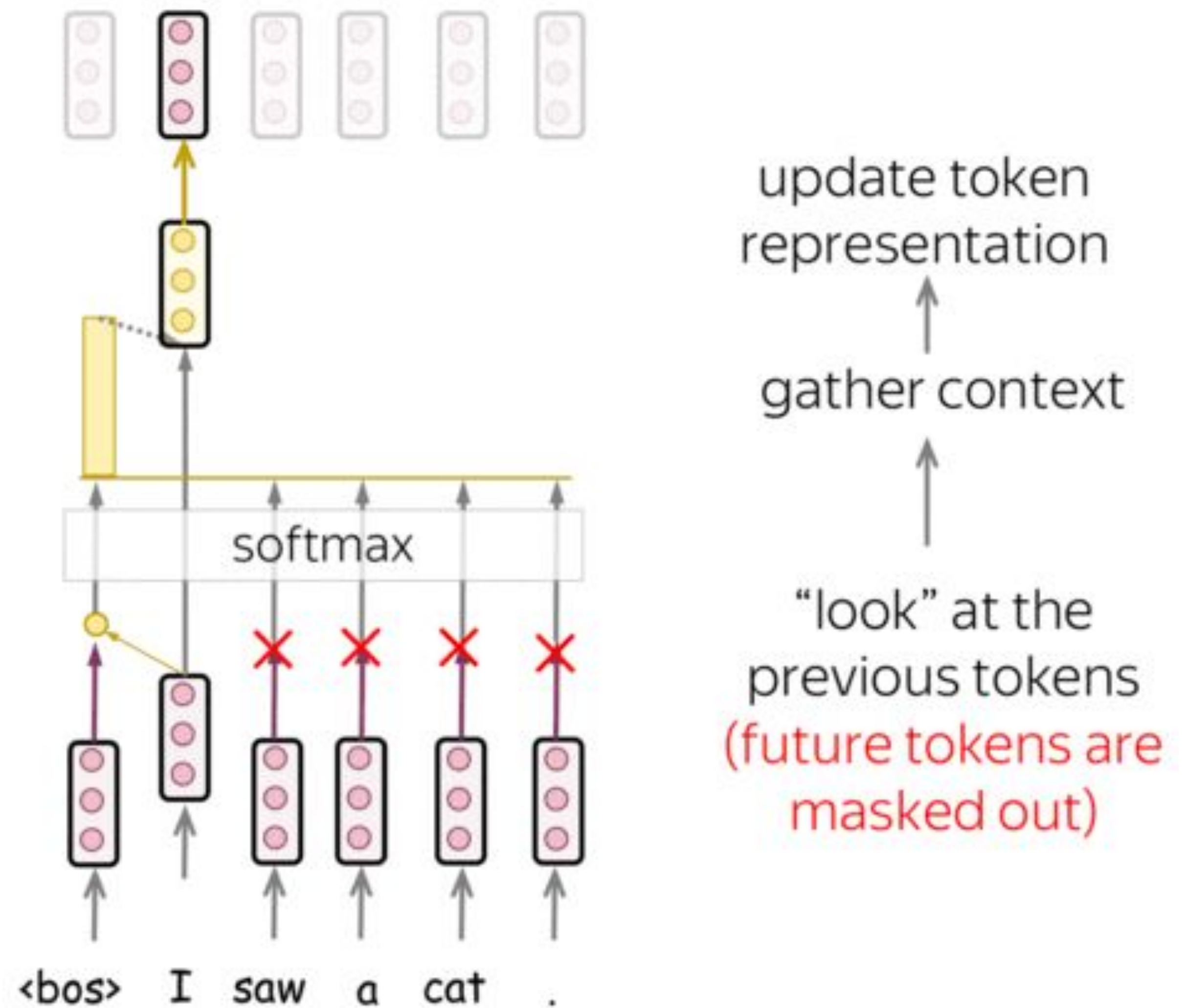
-  Analysis and Interpretability

- Idea
- Self-Attention
- Masked Self-Attention
- Multi-Head Attention
- KV caching
- Model architecture

Masked Self-Attention: “Don’t Look Ahead”

In the decoder, we forbid looking at future tokens – we don’t know them

Note: in training, decoder processes all target tokens at once – without masks, it would see future



What is going to happen:

- Seq2seq Basics

- Attention

- Transformer



- Subword Segmentation: BPE

-  Analysis and Interpretability

- Idea
- Self-Attention
- Masked Self-Attention
- Multi-Head Attention
- KV caching
- Model architecture

What is going to happen:

- Seq2seq Basics

- Attention

- Transformer



- Subword Segmentation: BPE

-  Analysis and Interpretability

- Idea
- Self-Attention
- Masked Self-Attention
- Multi-Head Attention
- KV caching
- Model architecture

Multi-Head Attention

We need to track many different things at once!

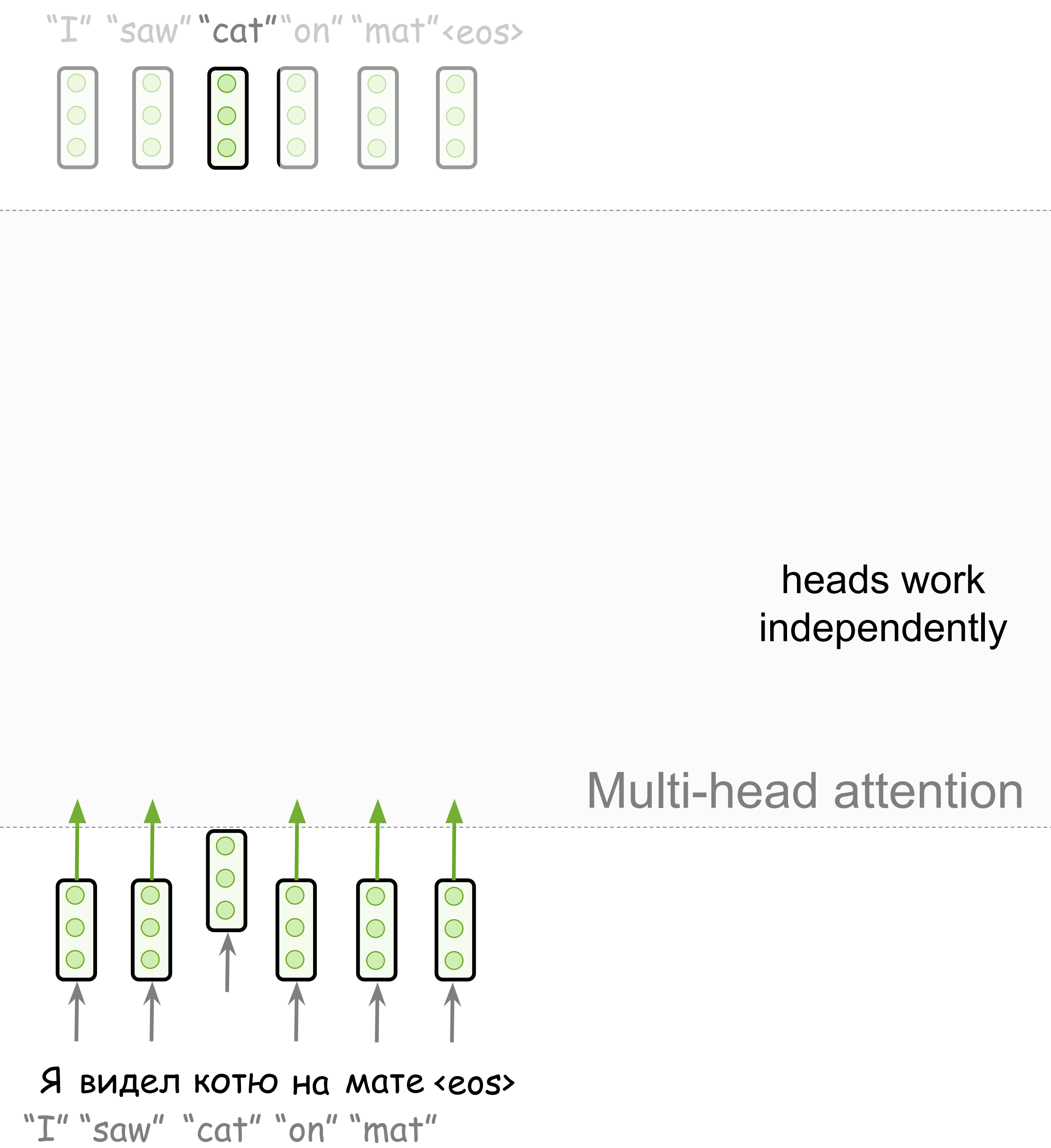


Она руководит НОВЫМ проектом

- Gender agreement
- Case government
- Lexical preferences
- ...

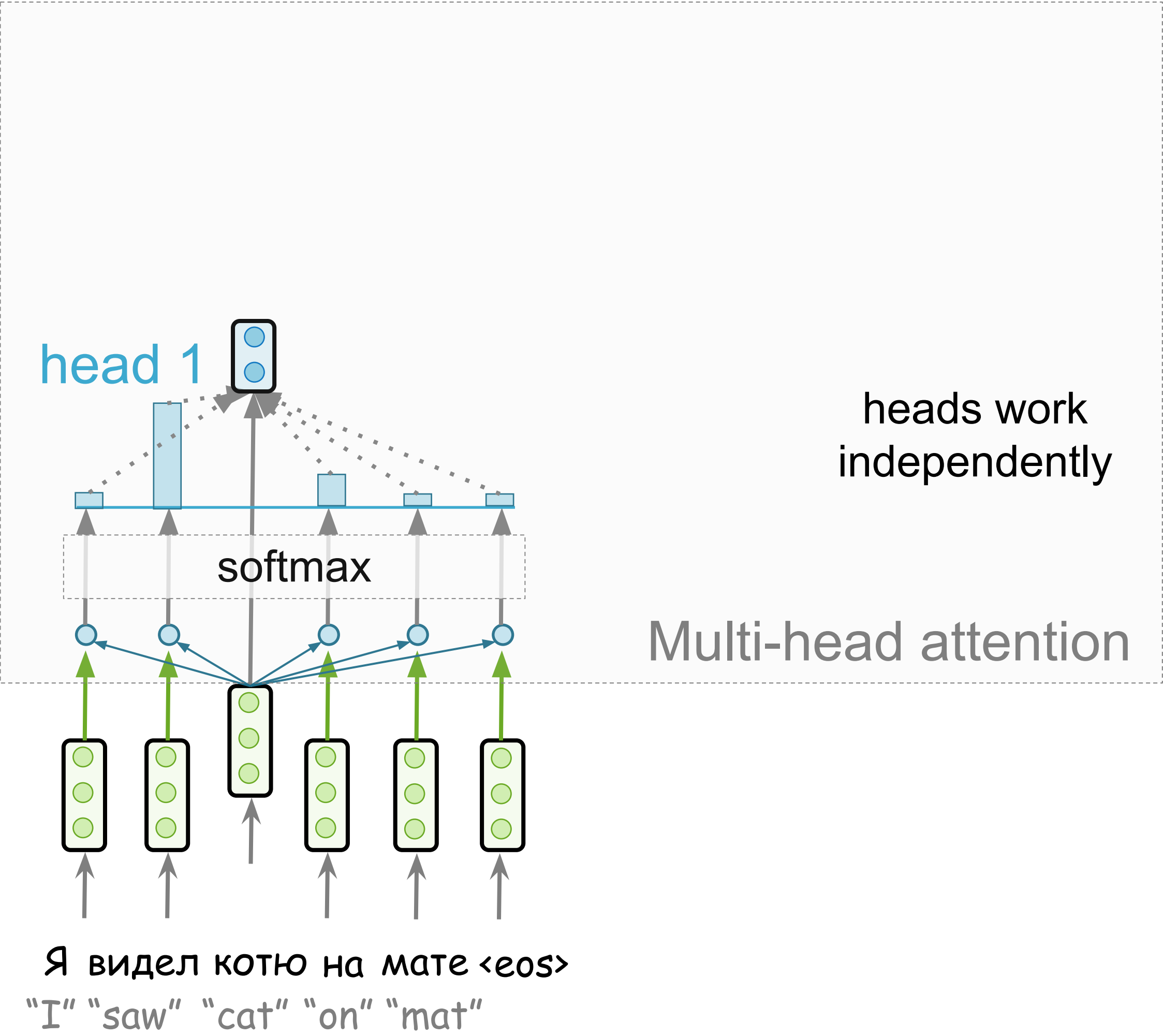
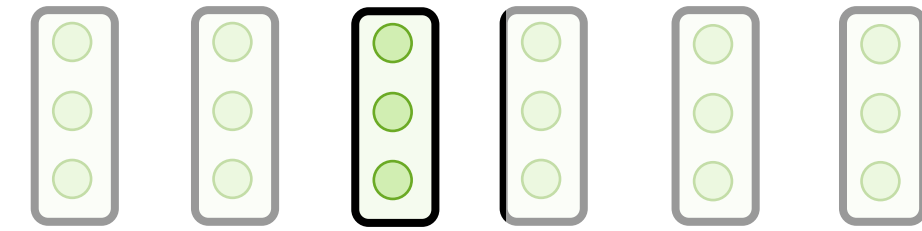
The example is from David Talbot

Multi-Head Attention

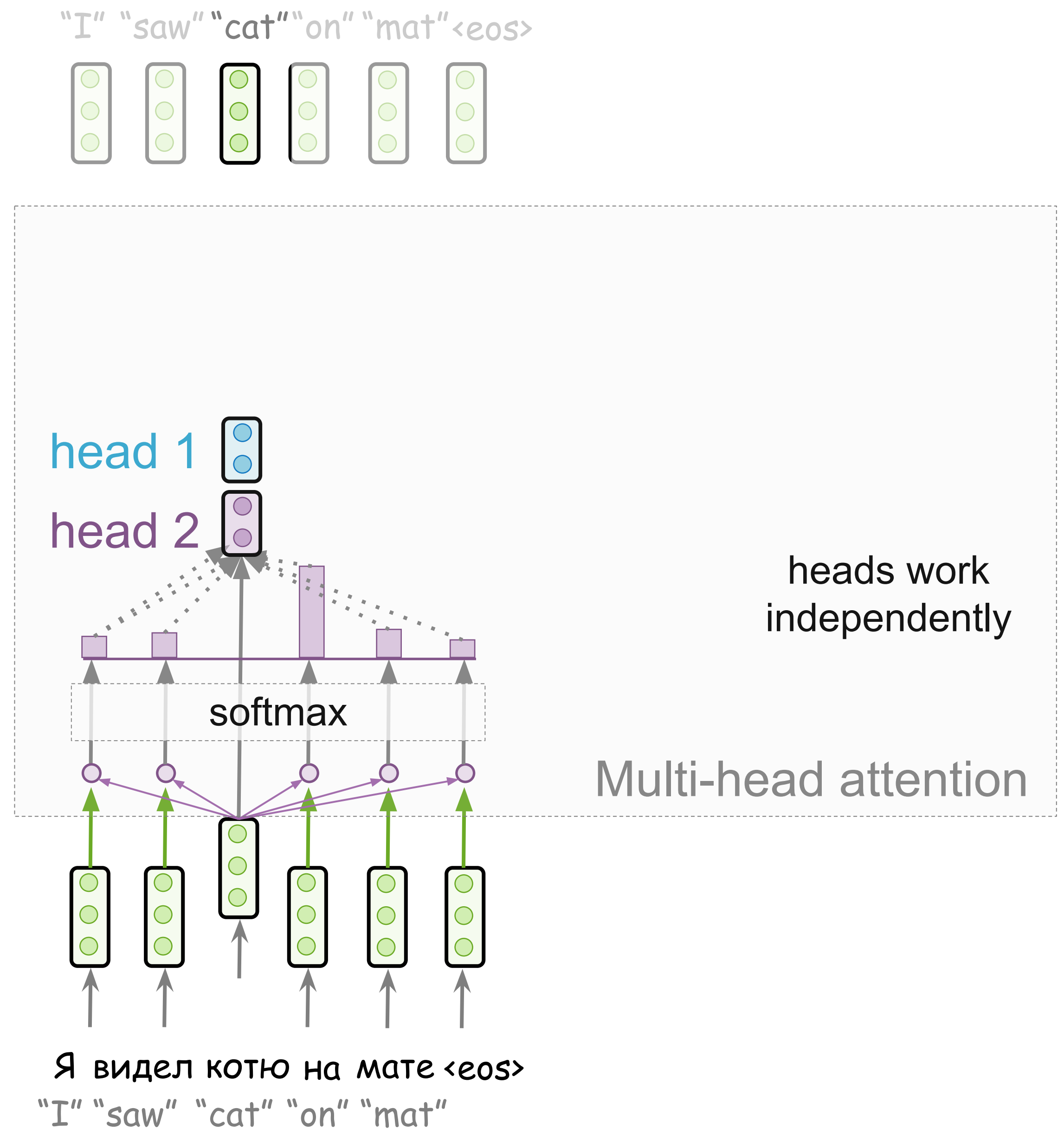


Multi-Head Attention

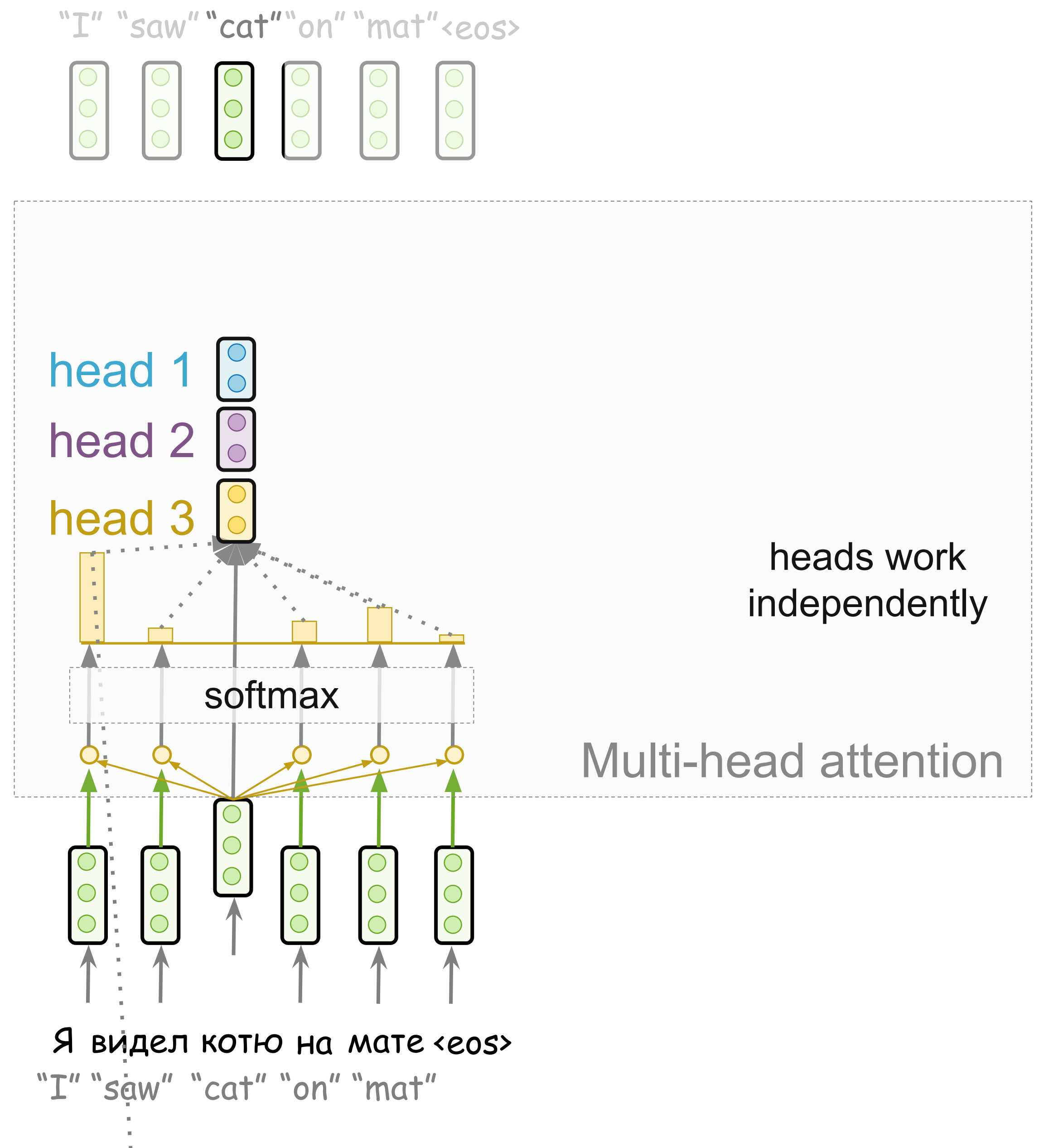
"I" "saw" "cat" "on" "mat" <eos>



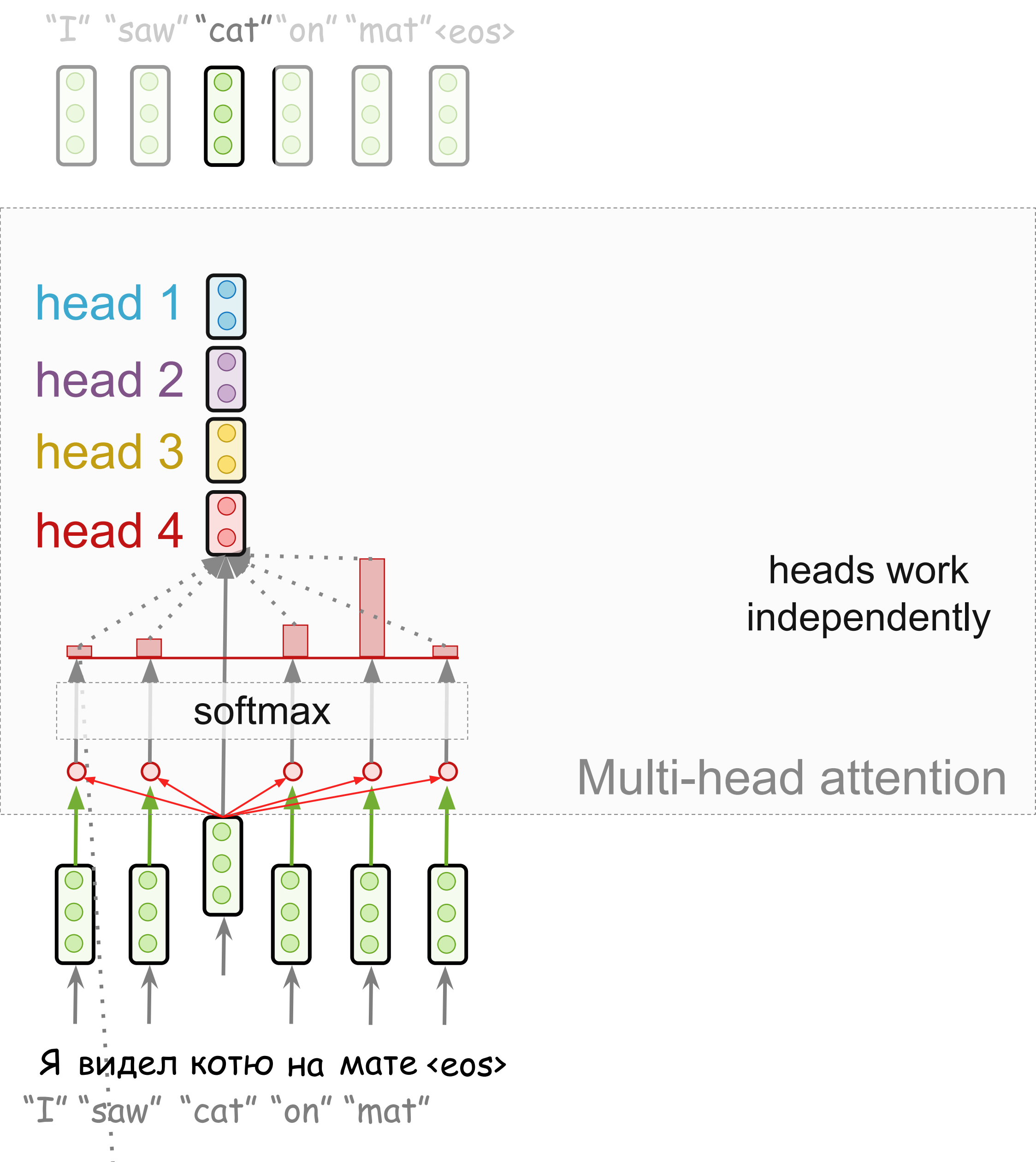
Multi-Head Attention



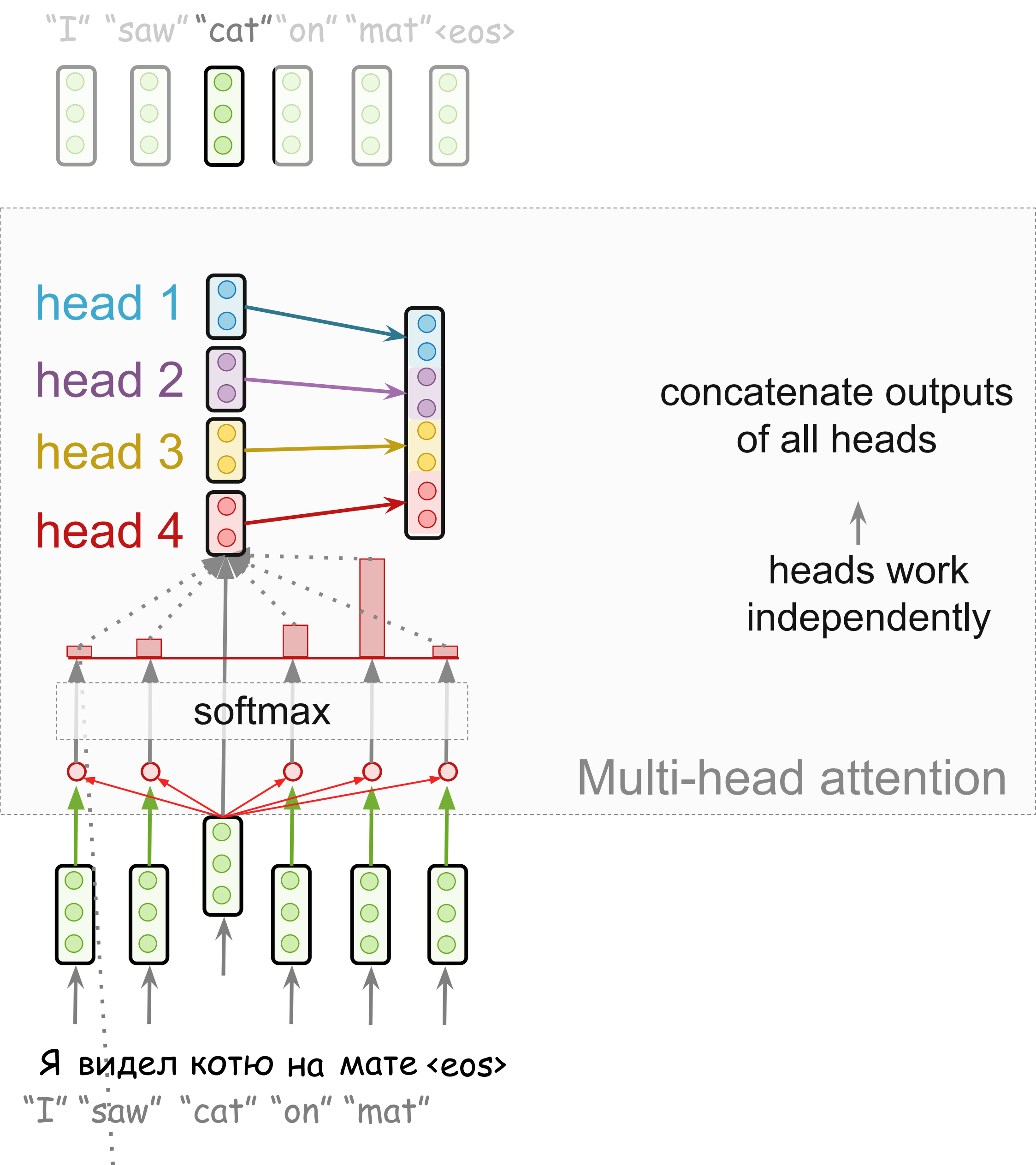
Multi-Head Attention



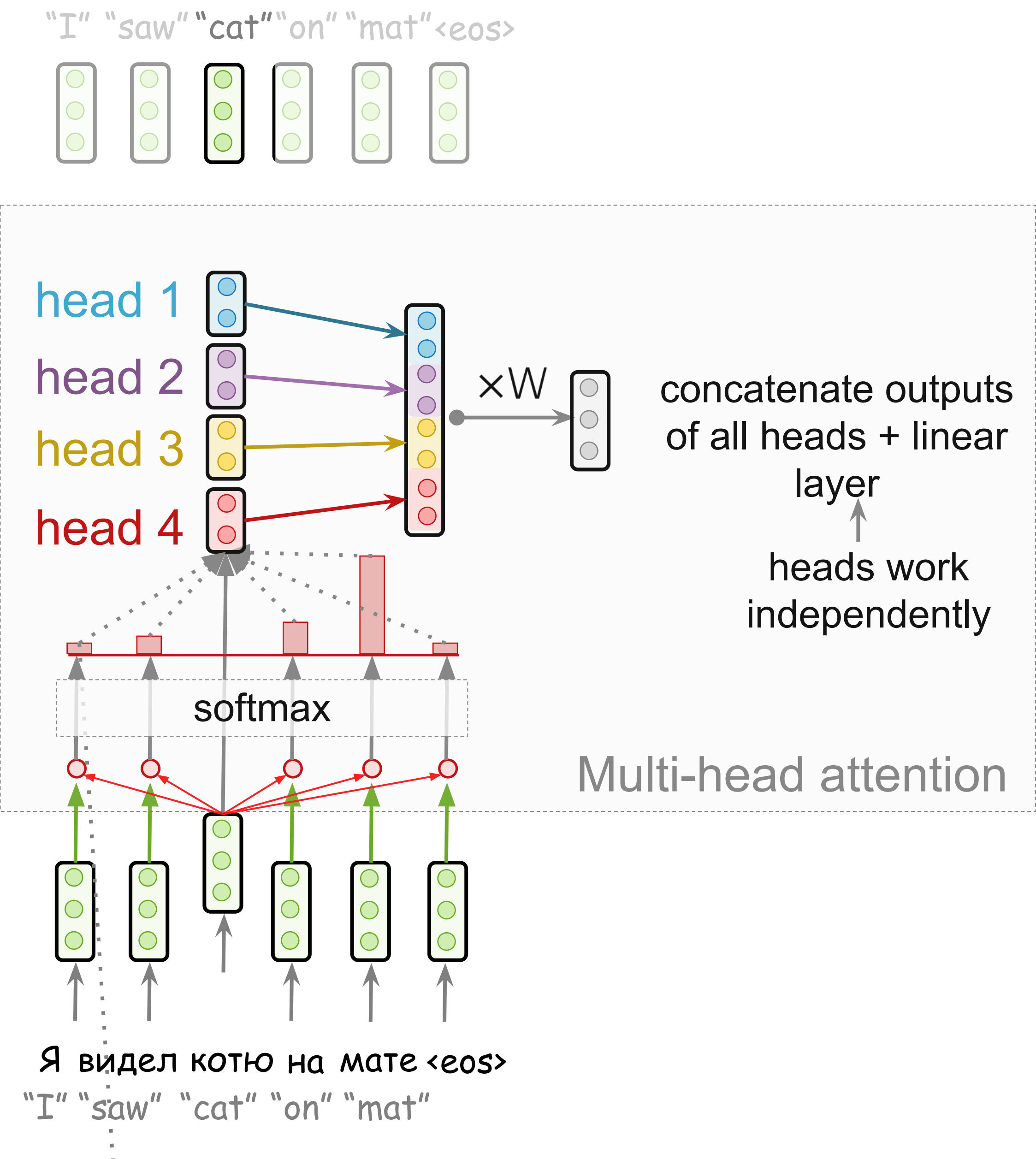
Multi-Head Attention



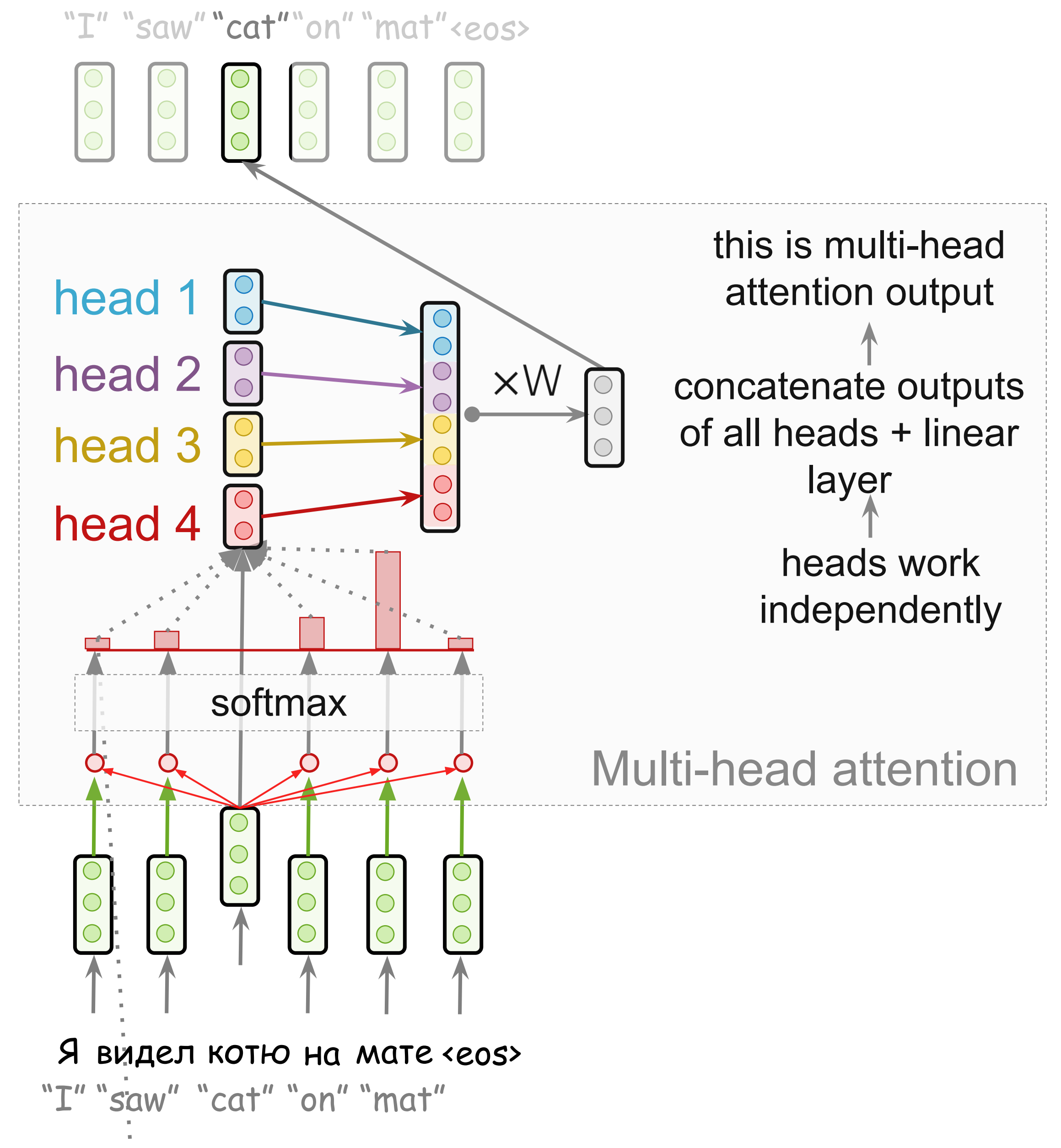
Multi-Head Attention



Multi-Head Attention



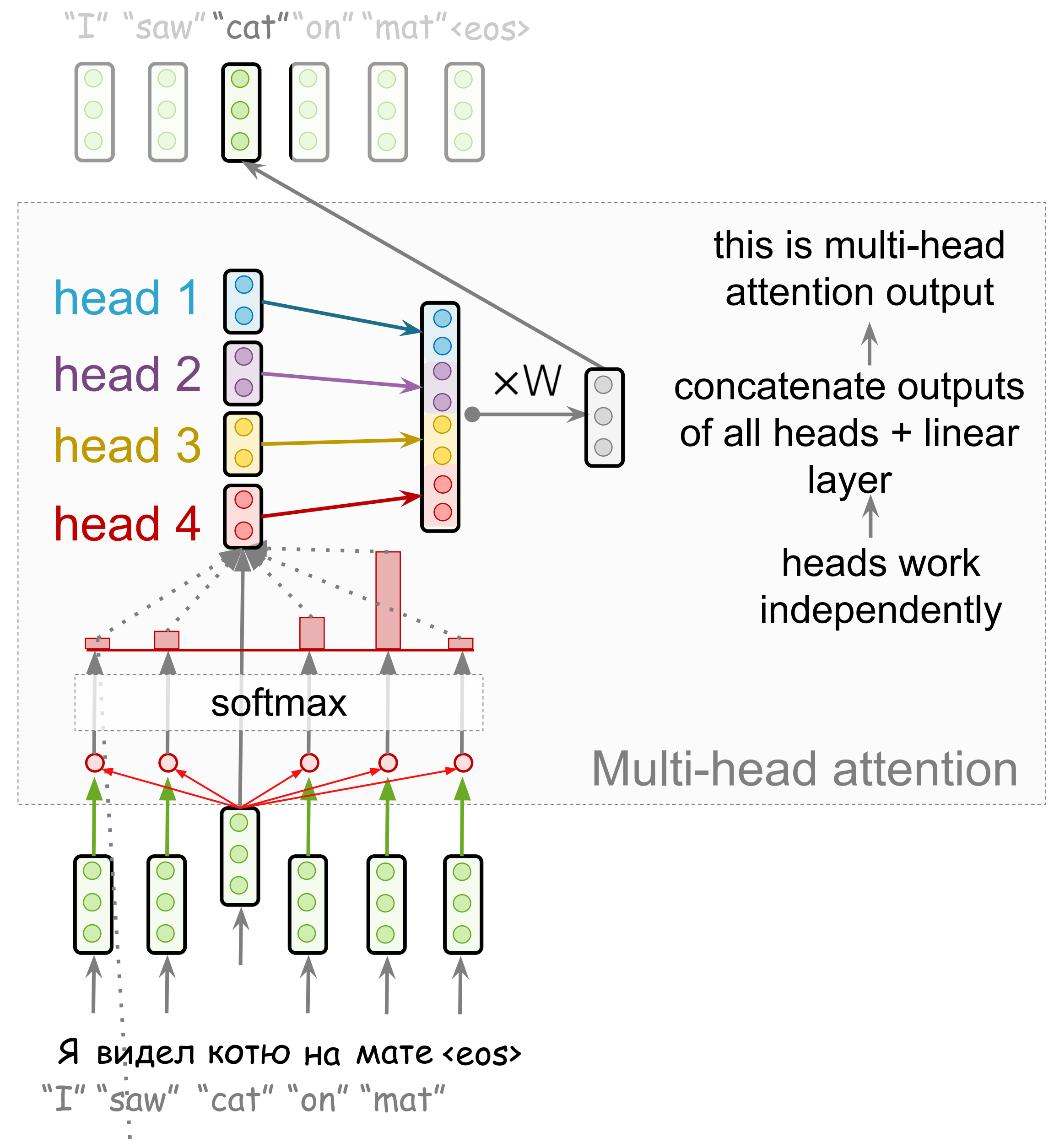
Multi-Head Attention



Multi-Head Attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_n)W_o,$$

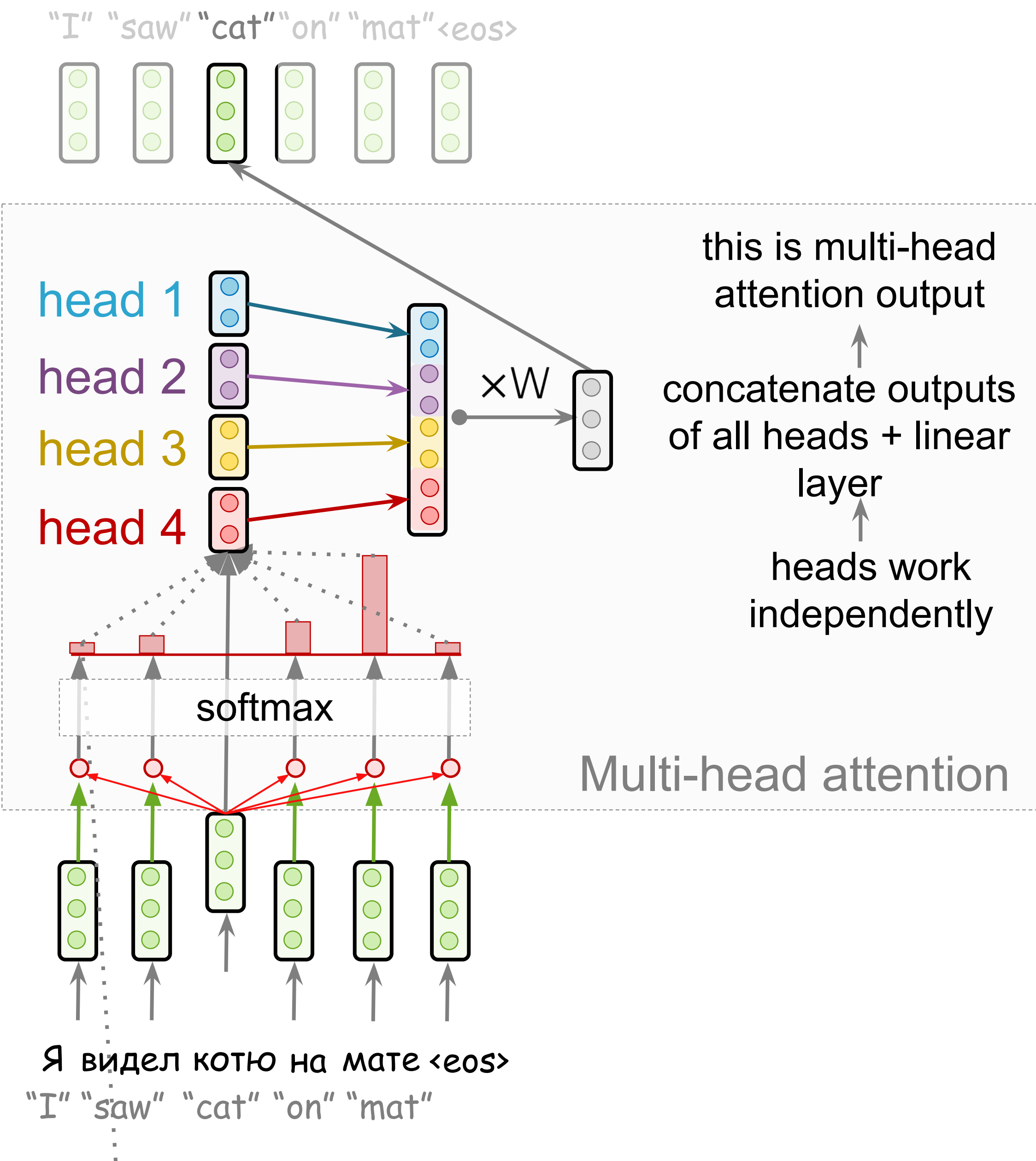
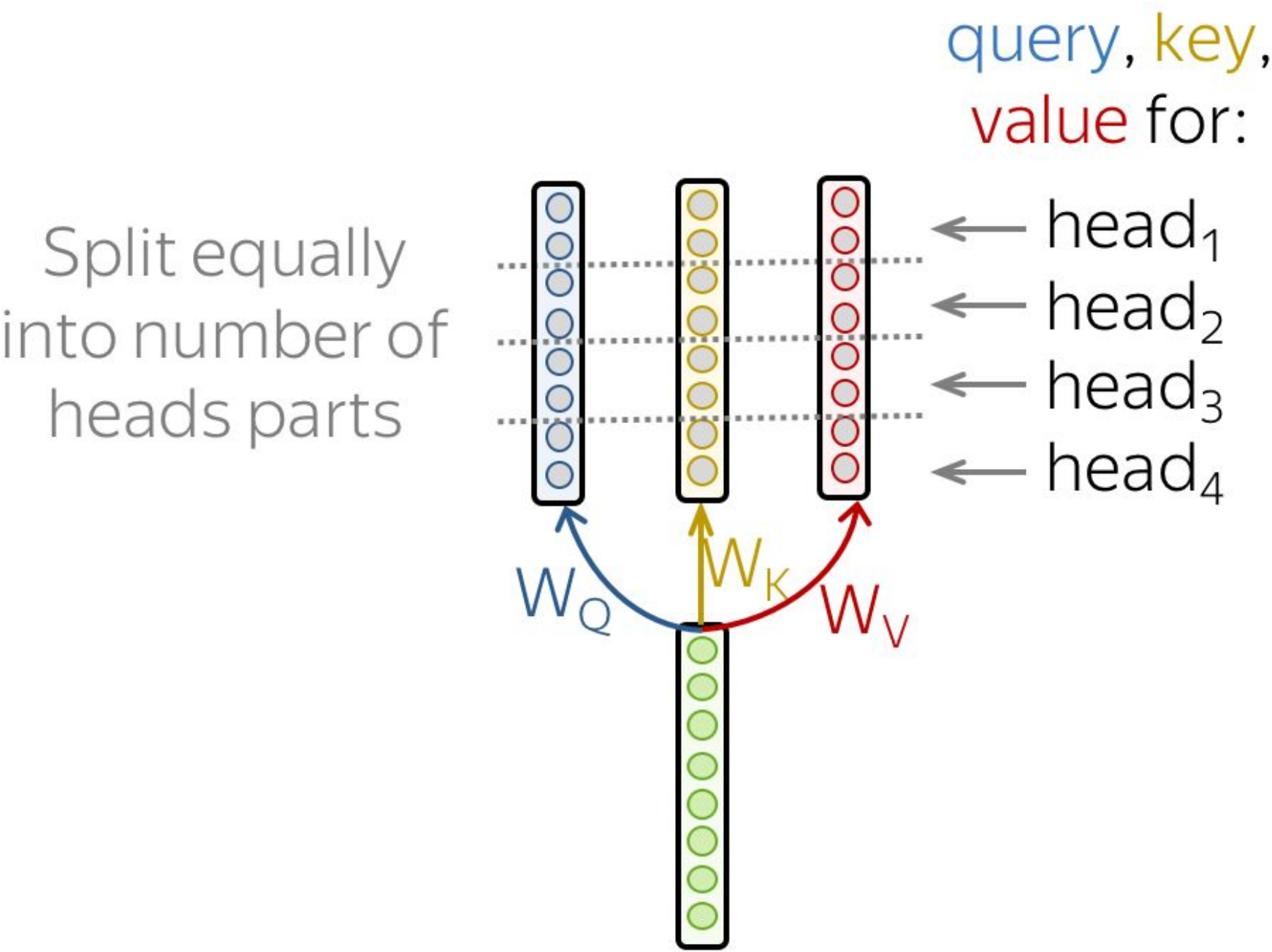
$$\text{head}_i = \text{Attention}(QW_Q^i, KW_K^i, VW_V^i)$$



Multi-Head Attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_n)W_o,$$

$$\text{head}_i = \text{Attention}(QW_Q^i, KW_K^i, VW_V^i)$$



What is going to happen:

- Seq2seq Basics
- Attention
- Transformer
- Transformer LMs
- BERT and family



- Idea
- Self-Attention
- Masked Self-Attention
- Multi-Head Attention
- KV caching
- Model architecture

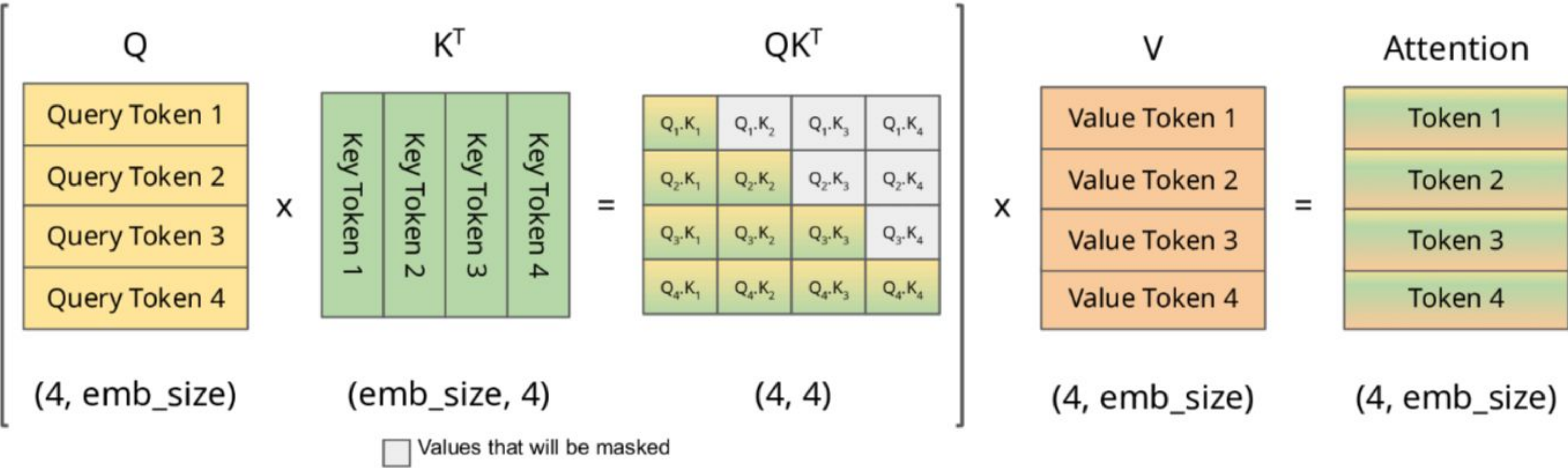
What is going to happen:

- Seq2seq Basics
- Attention
- Transformer
- Transformer LMs
- BERT and family

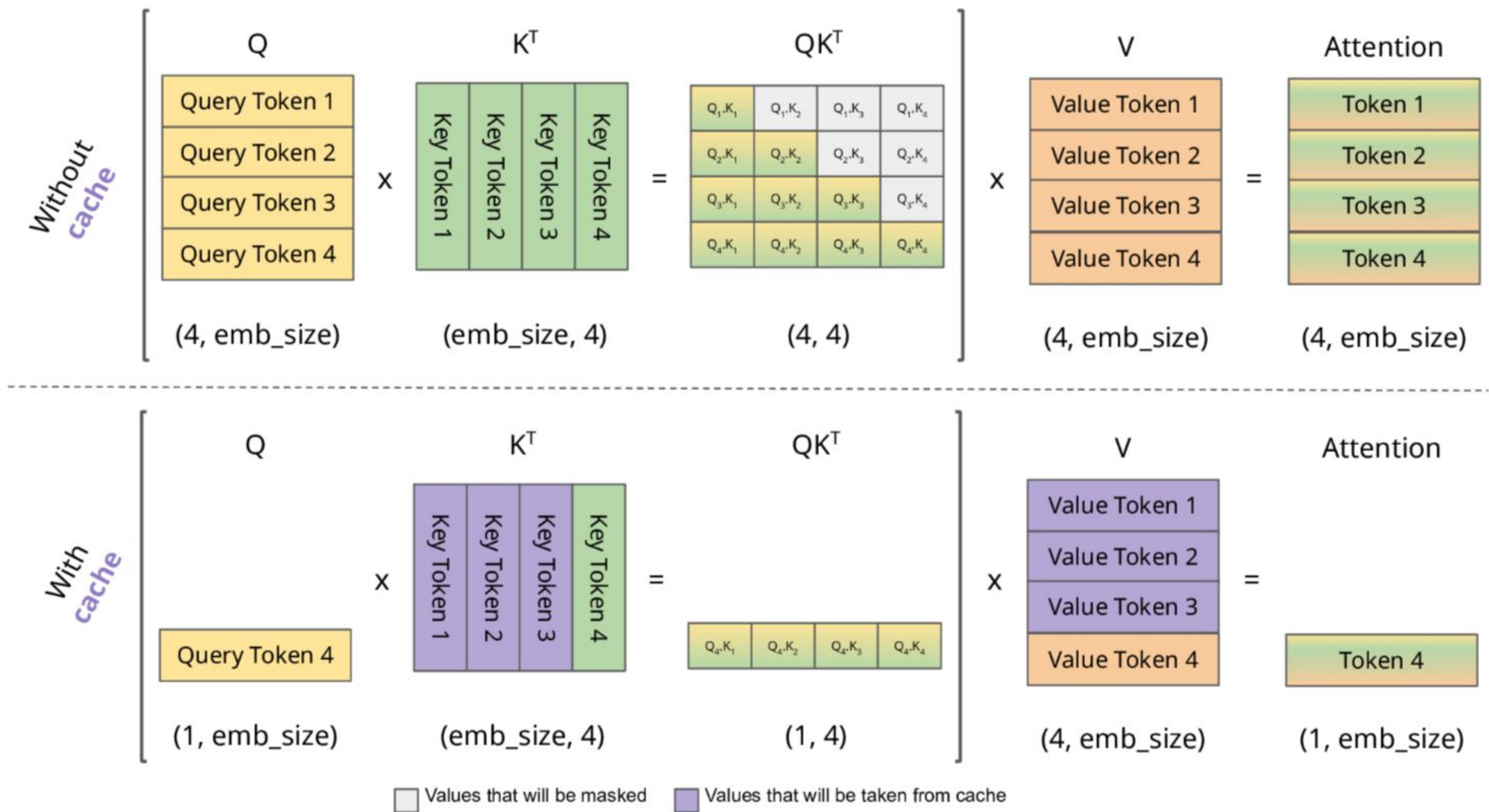


- Idea
- Self-Attention
- Masked Self-Attention
- Multi-Head Attention
- KV caching
- Model architecture

KV caching



KV caching



What is going to happen:

- Seq2seq Basics
- Attention
- Transformer
- Transformer LMs
- BERT and family



- Idea
- Self-Attention
- Masked Self-Attention
- Multi-Head Attention
- KV caching
- Model architecture

What is going to happen:

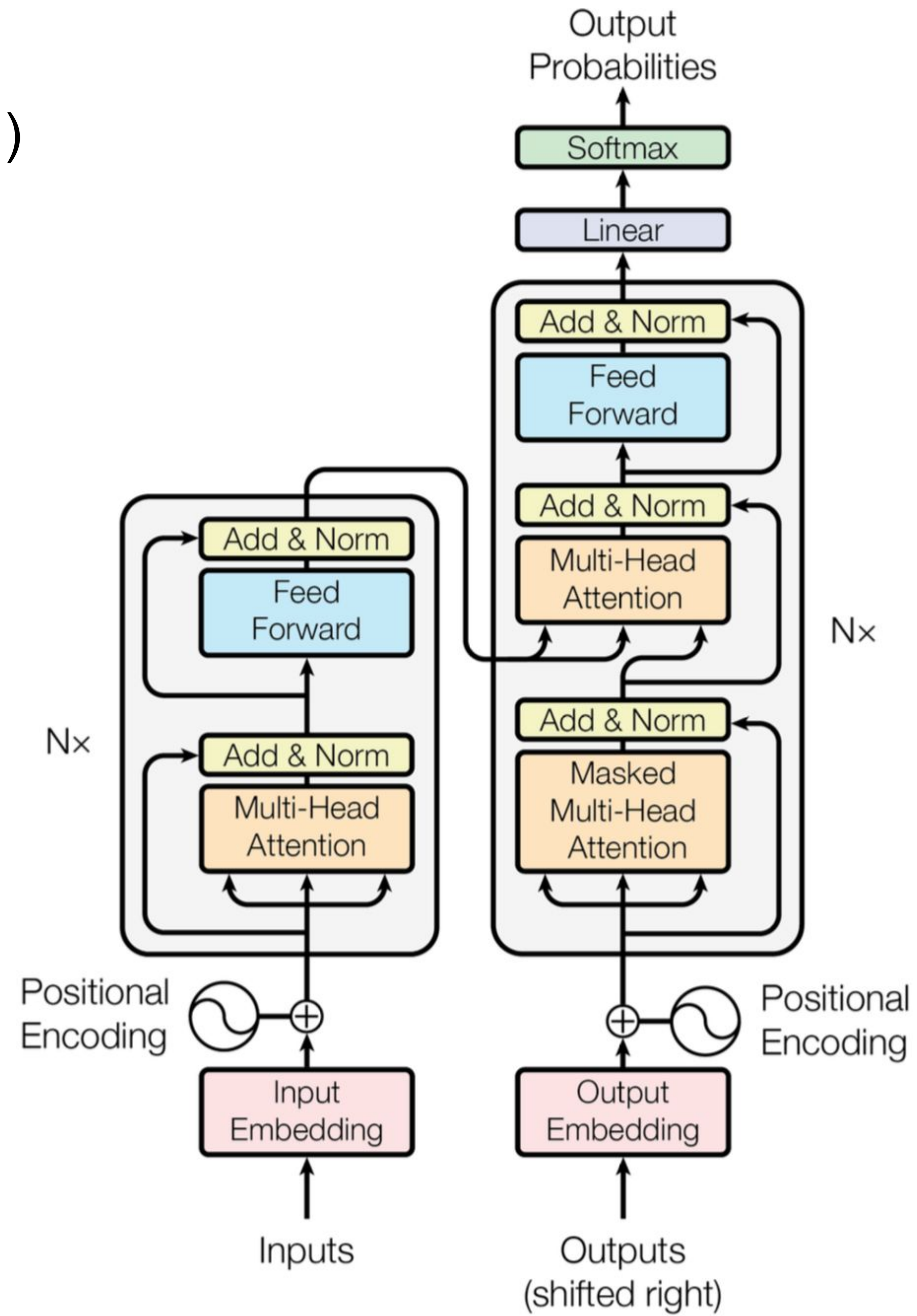
- Seq2seq Basics
- Attention
- Transformer
- Transformer LMs
- BERT and family



- Idea
- Self-Attention
- Masked Self-Attention
- Multi-Head Attention
- KV caching
- Model architecture

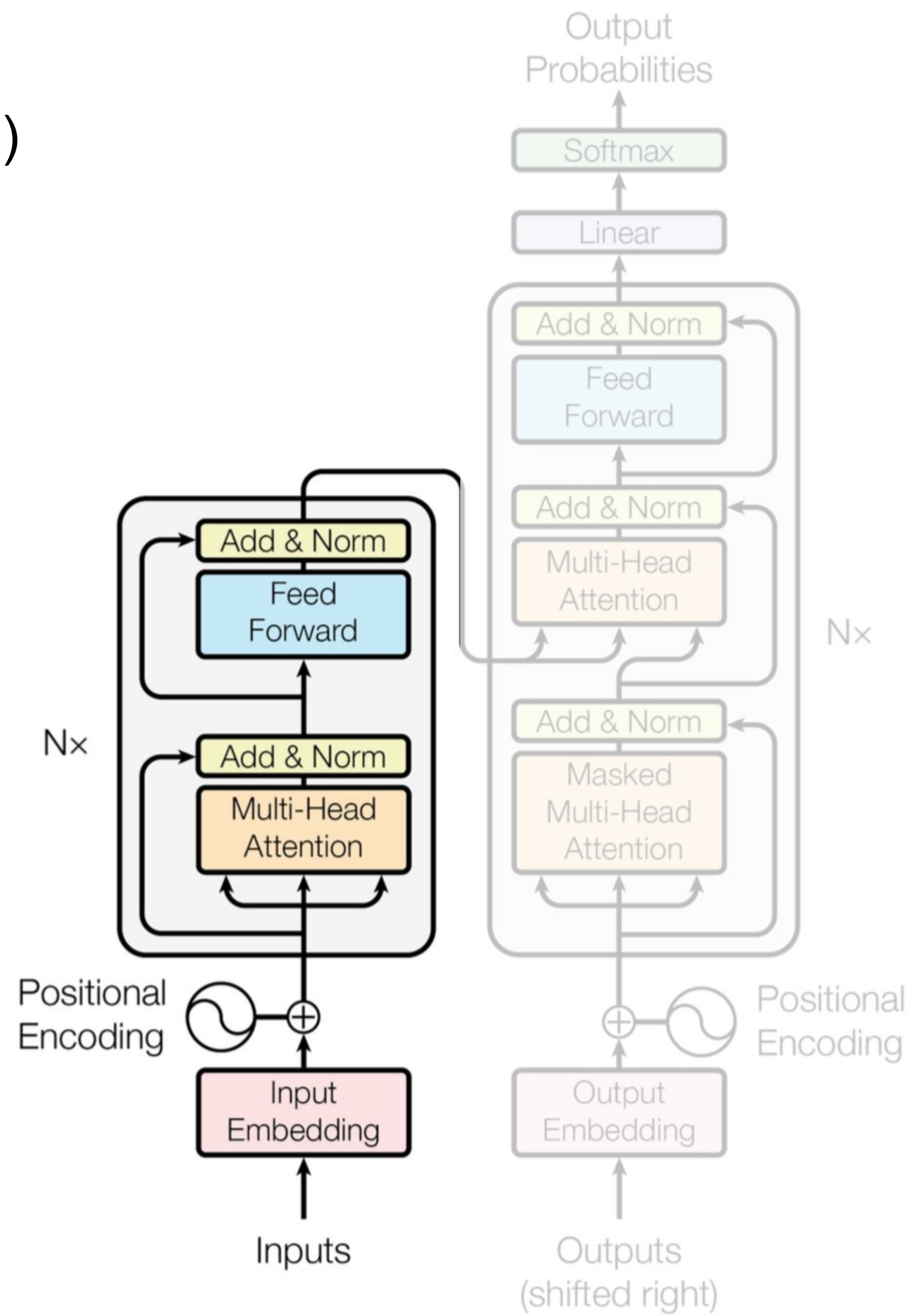
Transformer

(original model, seq2seq)



Transformer

(original model, seq2seq)

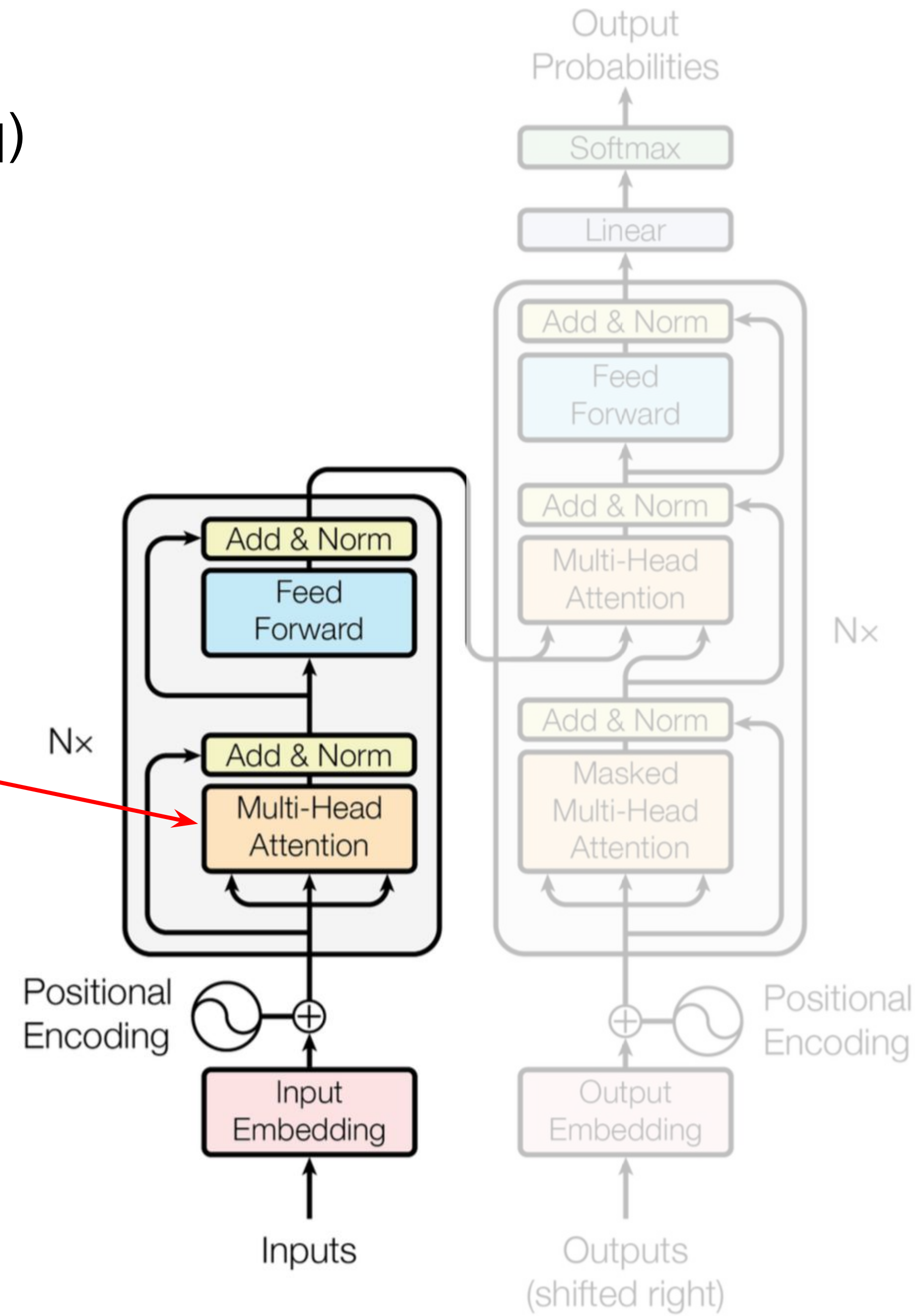


Transformer

(original model, seq2seq)

Encoder self-attention:
tokens look at each other

queries, keys, values
are computed from
encoder states



Transformer

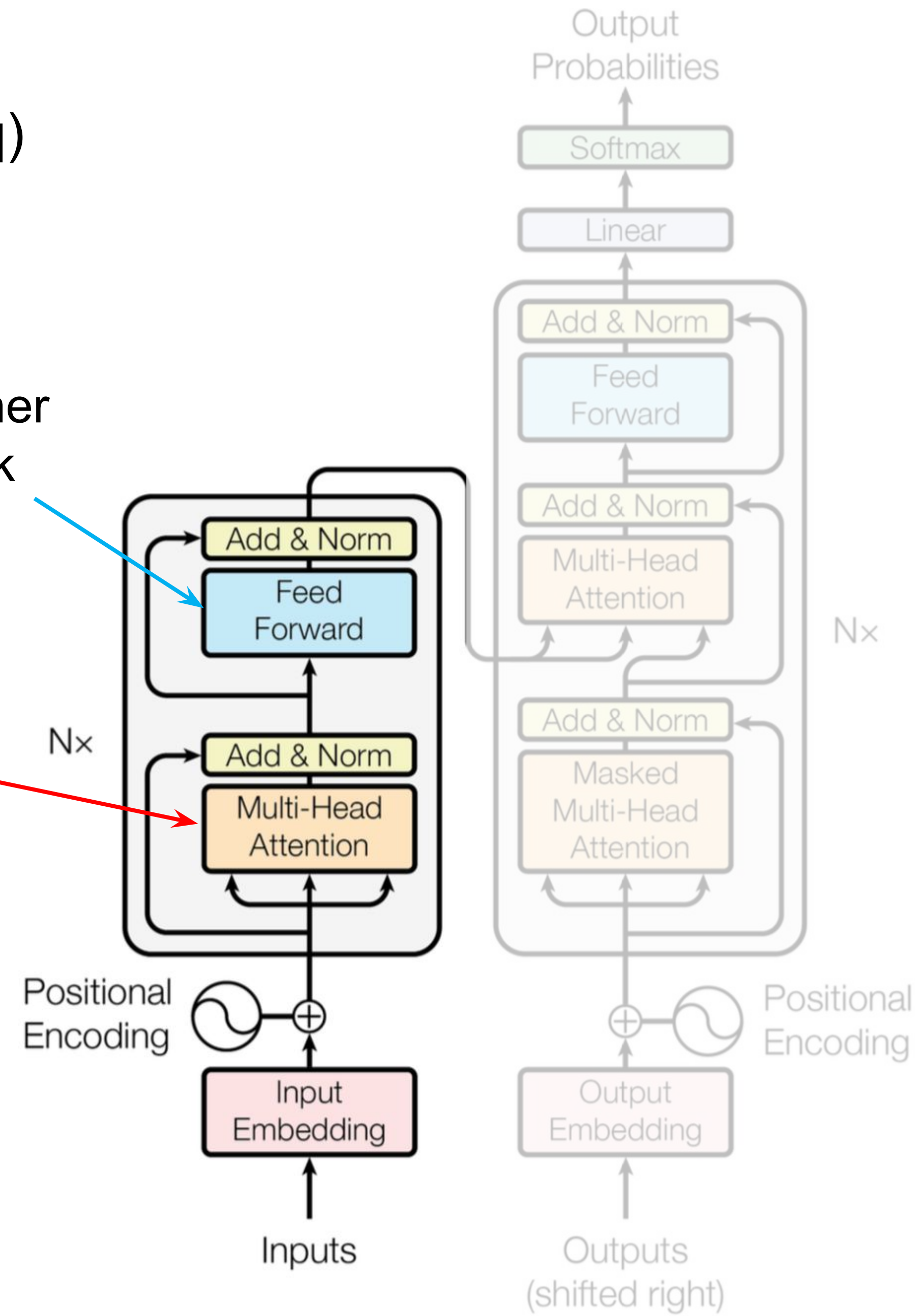
(original model, seq2seq)

Feed-forward network:

after taking information from other tokens, take a moment to think and process this information

Encoder self-attention:
tokens look at each other

queries, keys, values
are computed from
encoder states



Transformer

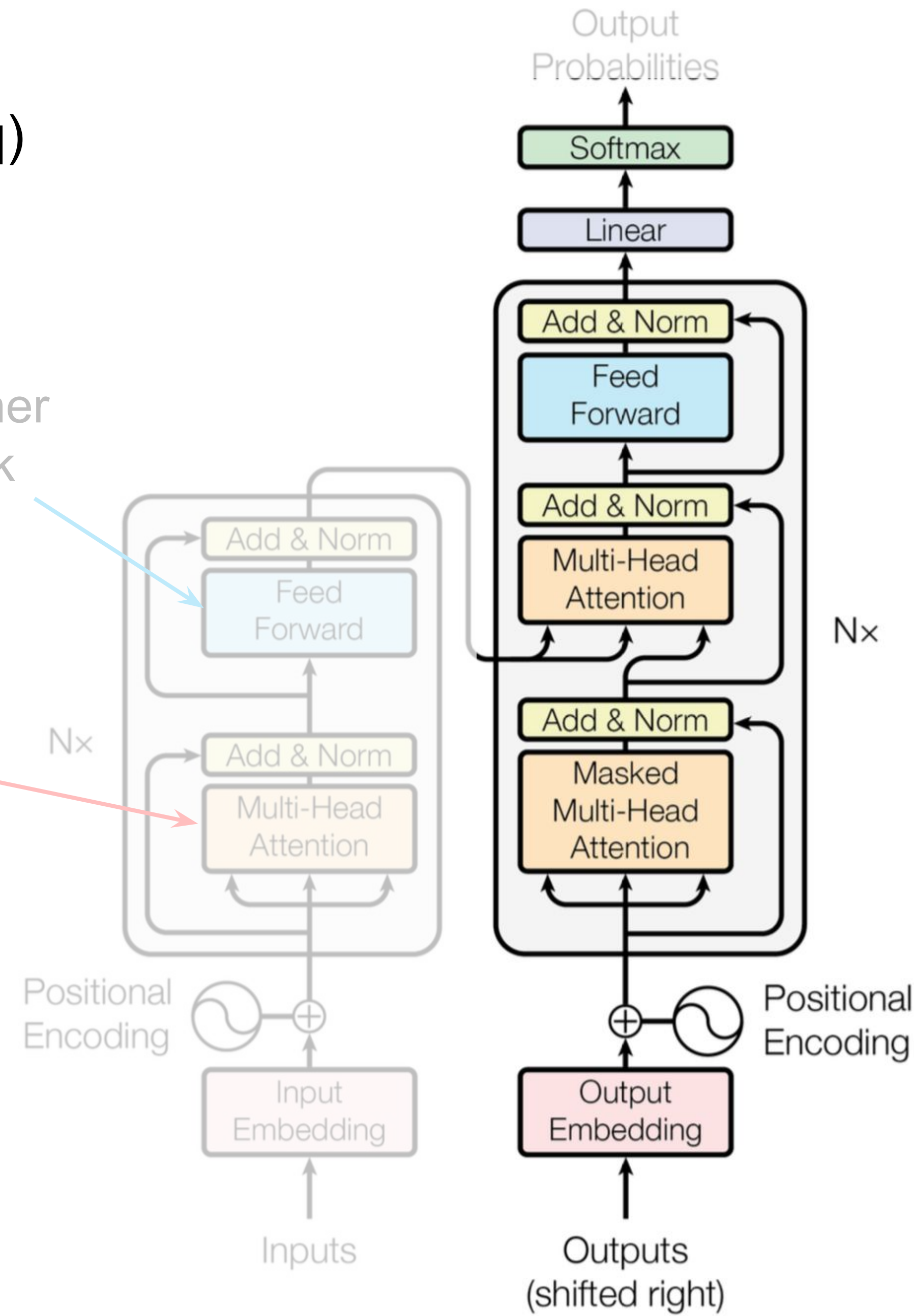
(original model, seq2seq)

Feed-forward network:

after taking information from other tokens, take a moment to think and process this information

Encoder self-attention:
tokens look at each other

queries, keys, values
are computed from
encoder states



Transformer

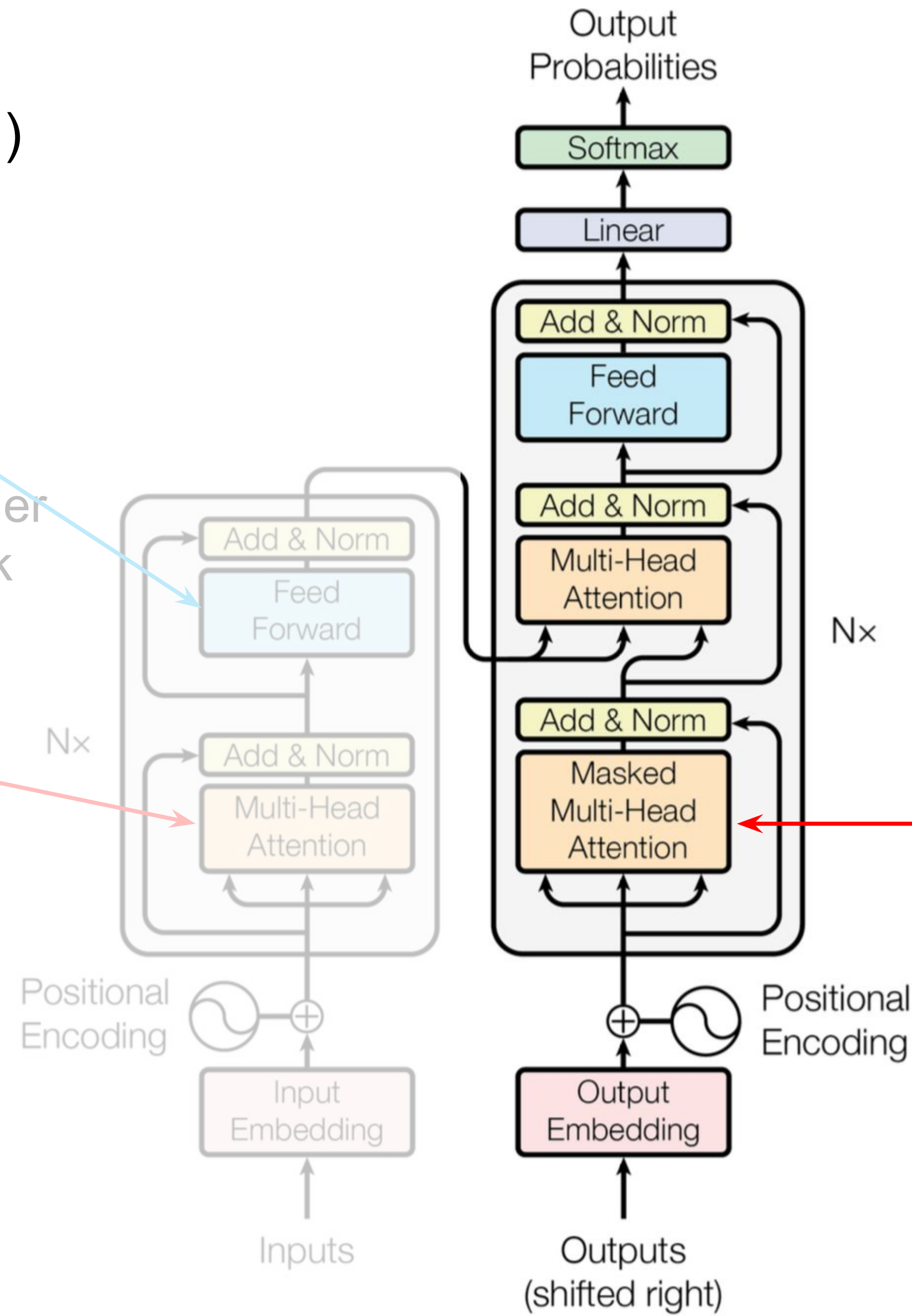
(original model, seq2seq)

Feed-forward network:

after taking information from other tokens, take a moment to think and process this information

Encoder self-attention:
tokens look at each other

queries, keys, values
are computed from
encoder states



Decoder self-attention (masked):
tokens look at the previous tokens

queries, keys, values are
computed from decoder states

Transformer

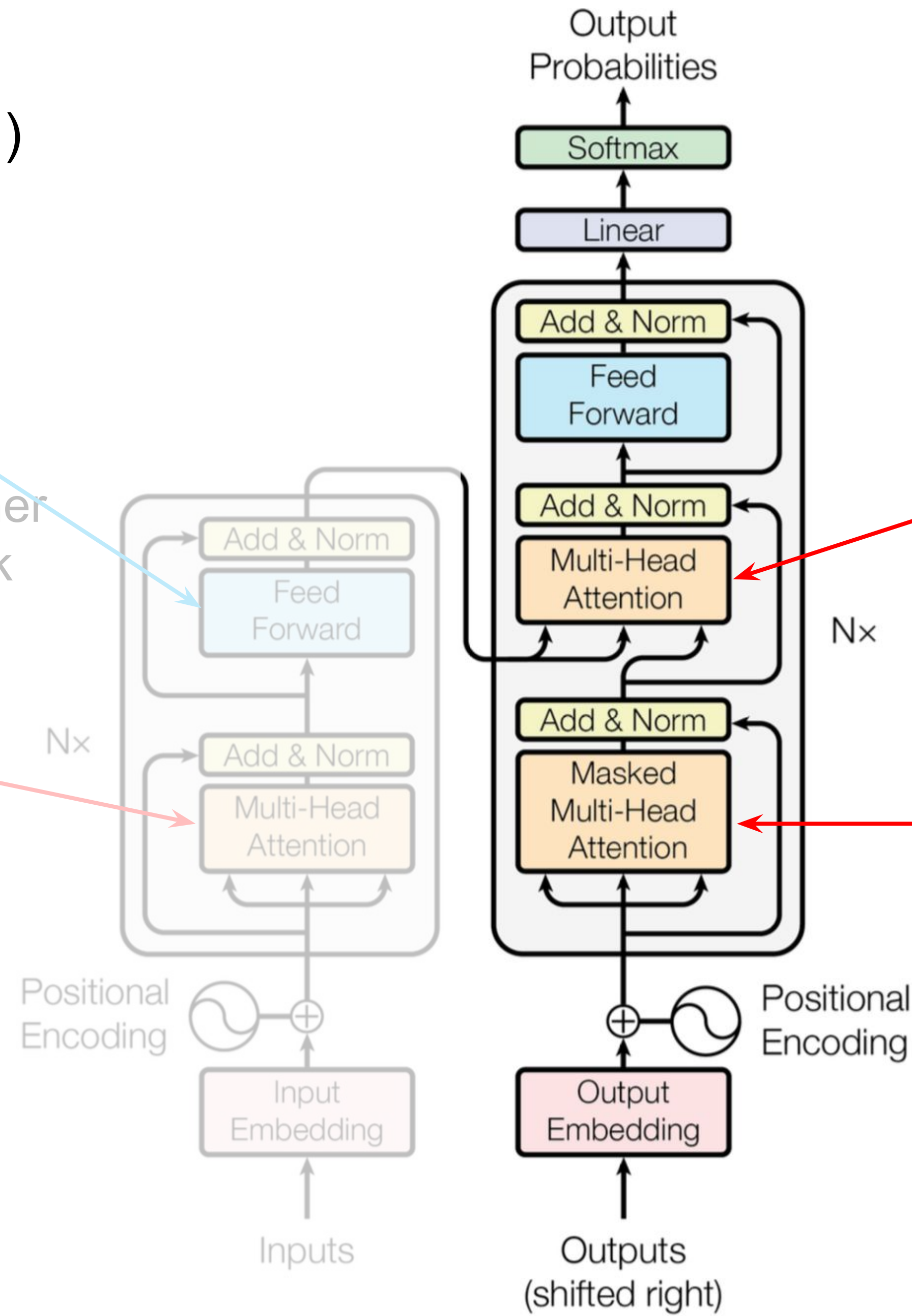
(original model, seq2seq)

Feed-forward network:

after taking information from other tokens, take a moment to think and process this information

Encoder self-attention:
tokens look at each other

queries, keys, values
are computed from
encoder states



Decoder-encoder attention:
target token looks at the source
queries – from decoder states; keys
and values from encoder states

Decoder self-attention (masked):
tokens look at the previous tokens
queries, keys, values are
computed from decoder states

Transformer

(original model, seq2seq)

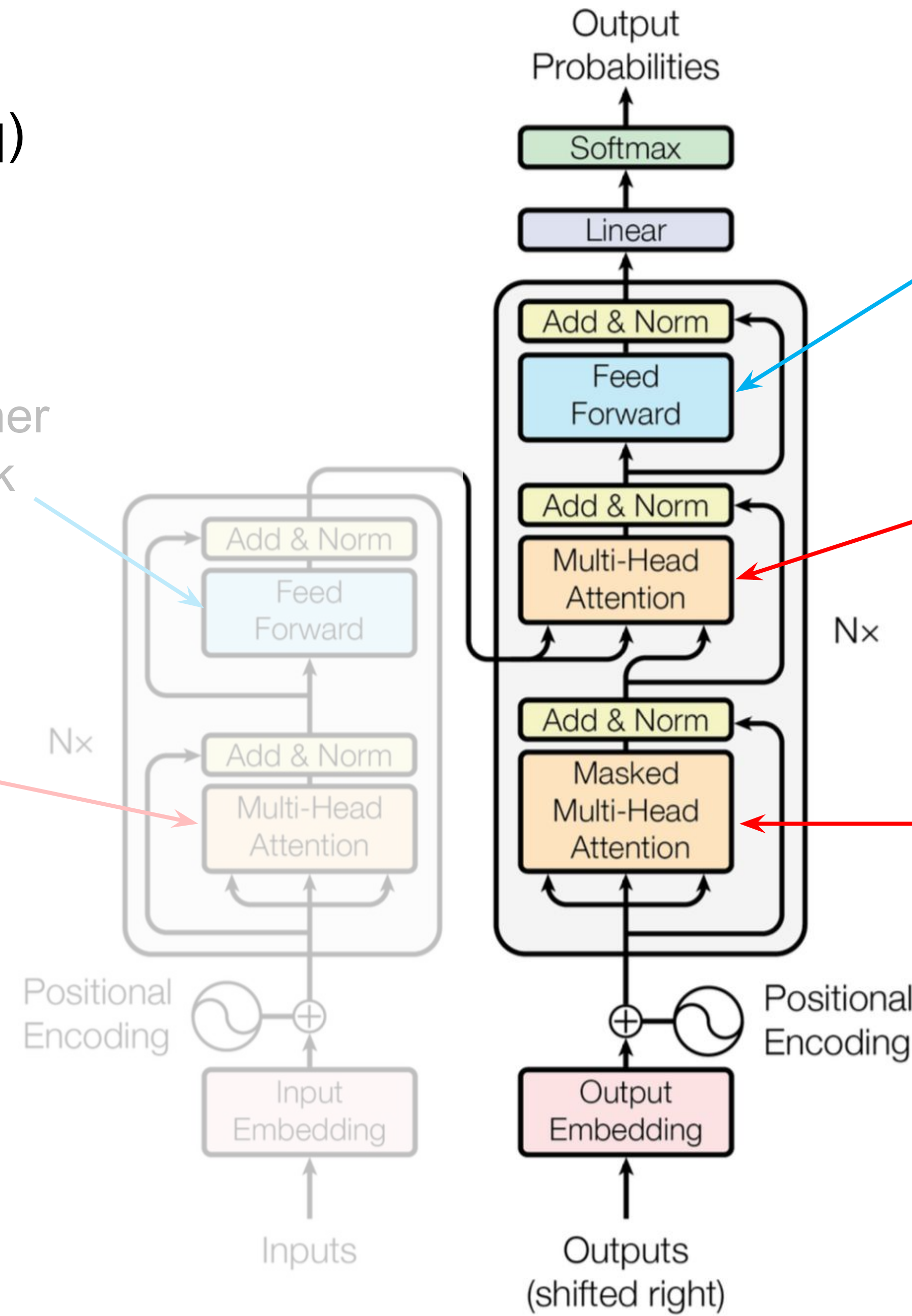
Feed-forward network:

after taking information from other tokens, take a moment to think and process this information

Encoder self-attention:

tokens look at each other

queries, keys, values are computed from encoder states



Feed-forward network:

after taking information from other tokens, take a moment to think and process this information

Decoder-encoder attention:

target token looks at the source queries – from decoder states; keys and values from encoder states

Decoder self-attention (masked):

tokens look at the previous tokens

queries, keys, values are computed from decoder states

Transformer

(original model, seq2seq)

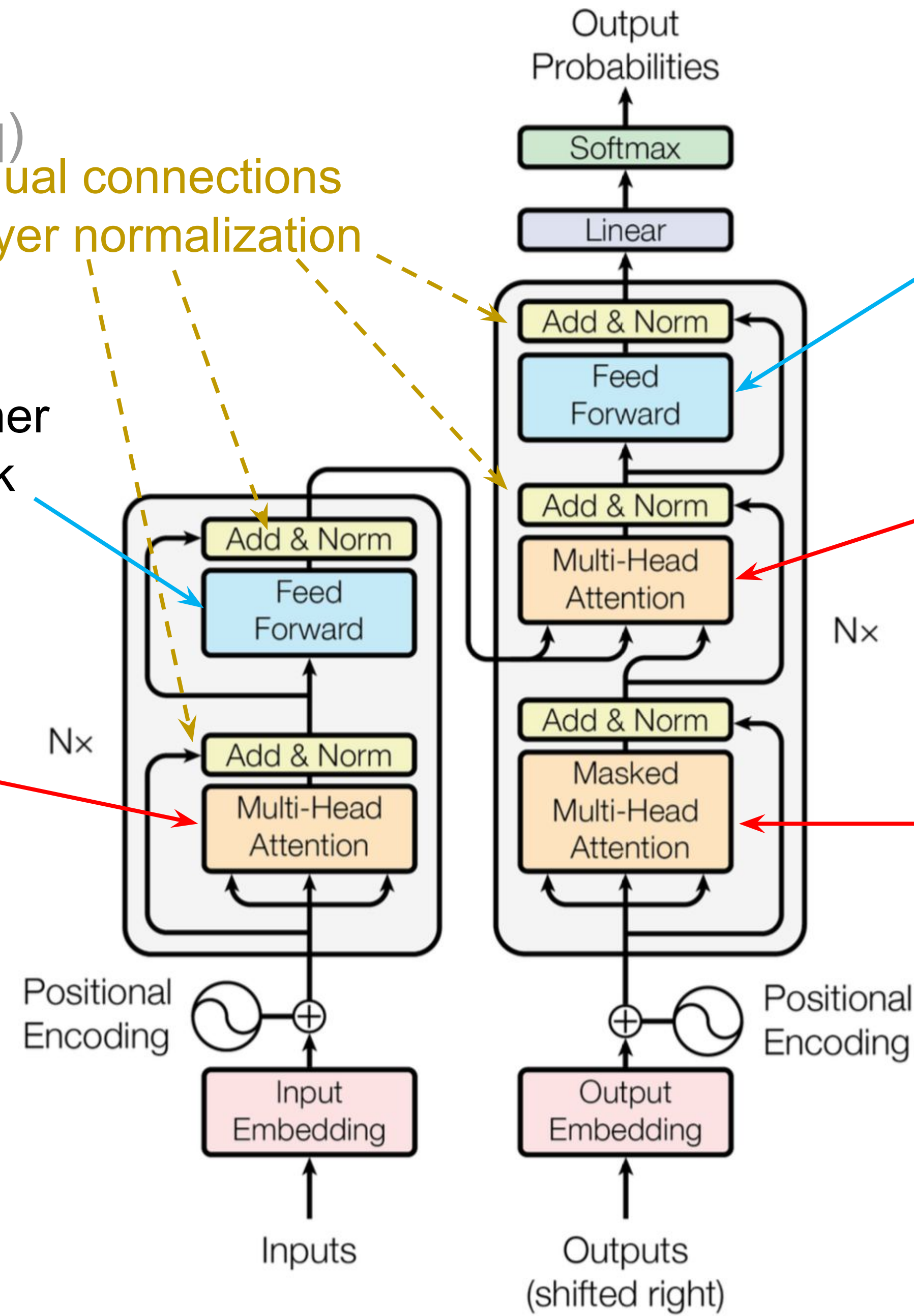
Residual connections
and layer normalization

Feed-forward network:

after taking information from other
tokens, take a moment to think
and process this information

Encoder self-attention:
tokens look at each other

queries, keys, values
are computed from
encoder states



Feed-forward network:

after taking information from other
tokens, take a moment to think
and process this information

Decoder-encoder attention:

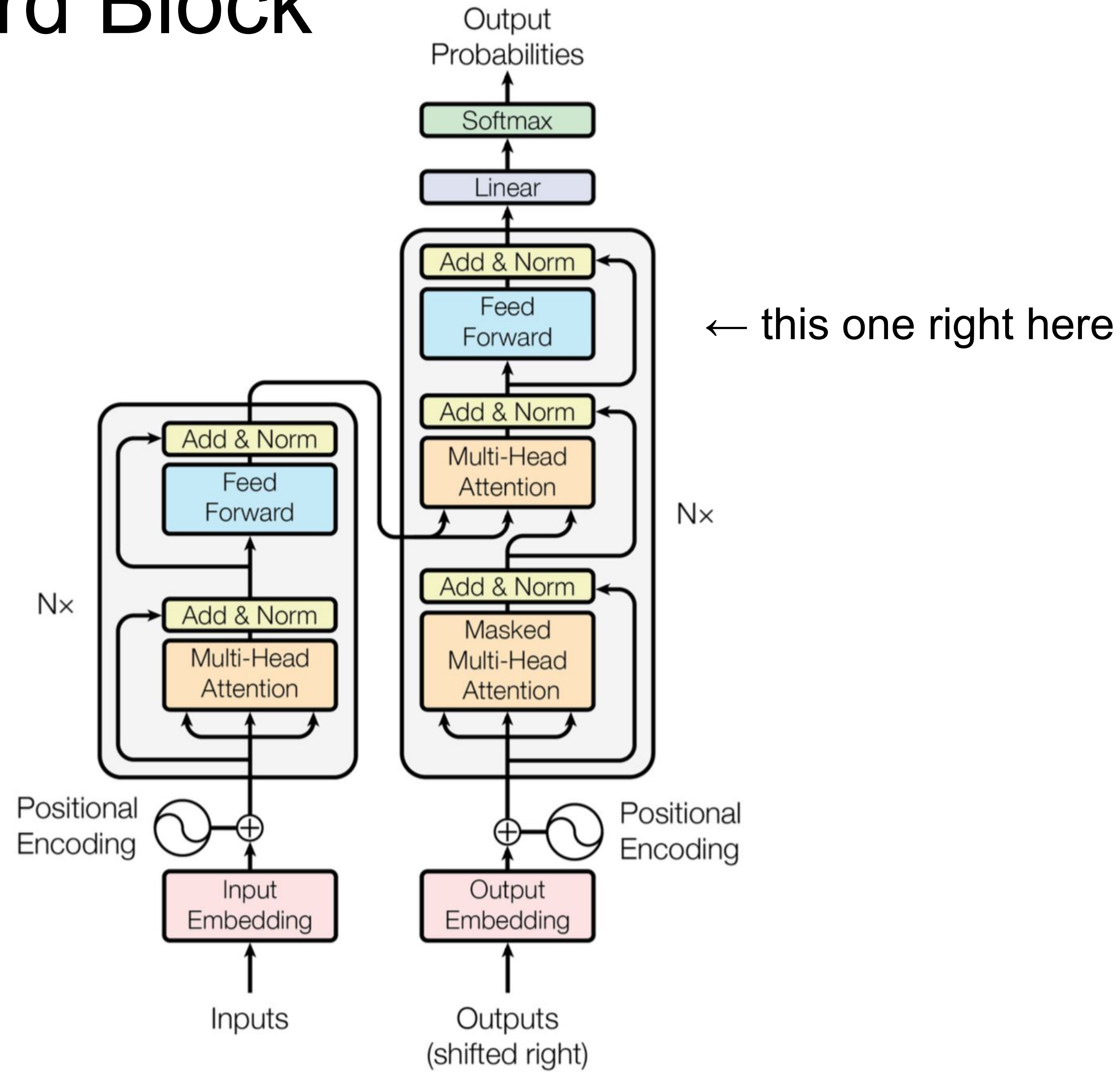
target token looks at the source
queries – from decoder states; keys
and values from encoder states

Decoder self-attention (masked):

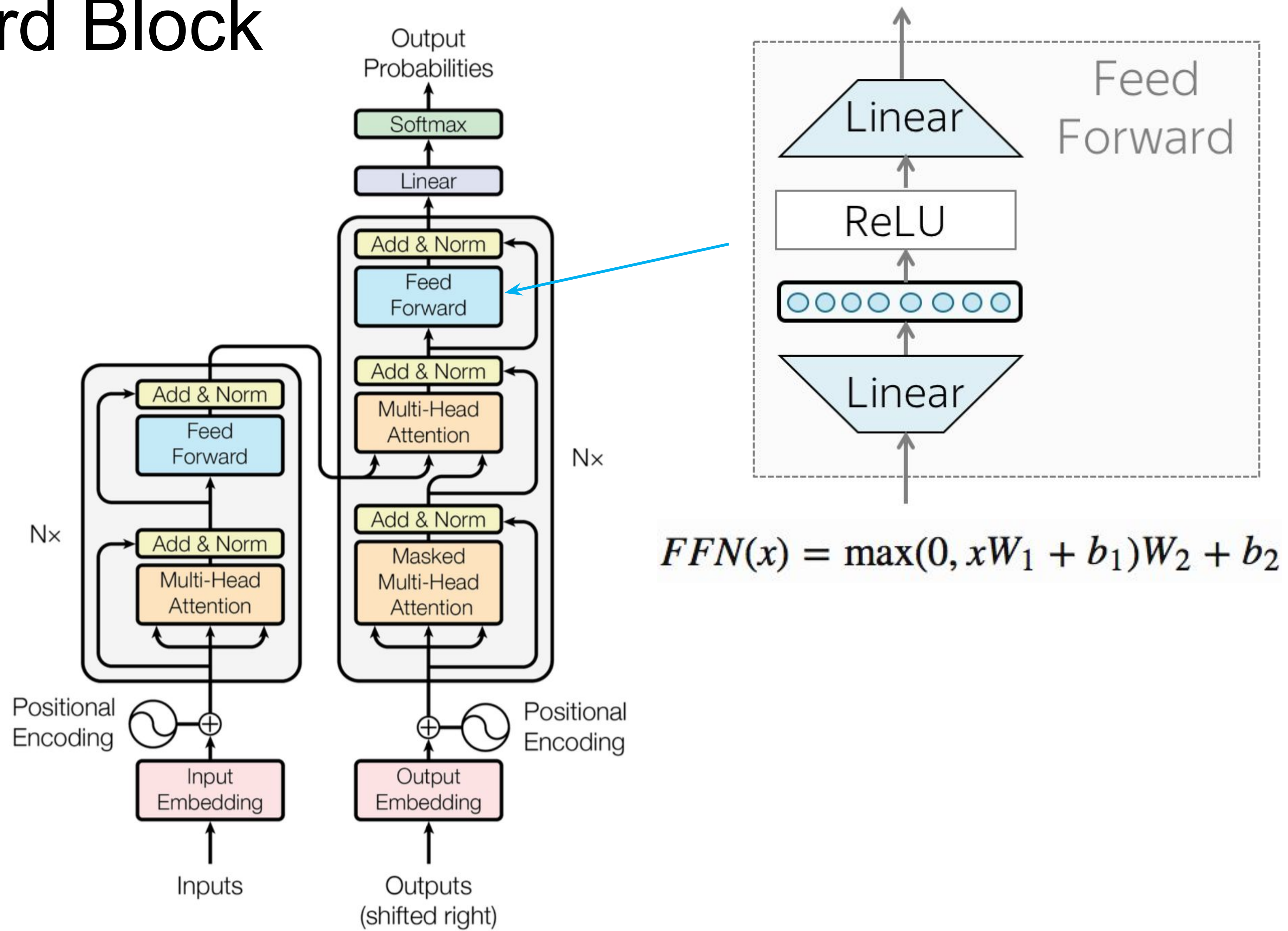
tokens look at the previous tokens

queries, keys, values are
computed from decoder states

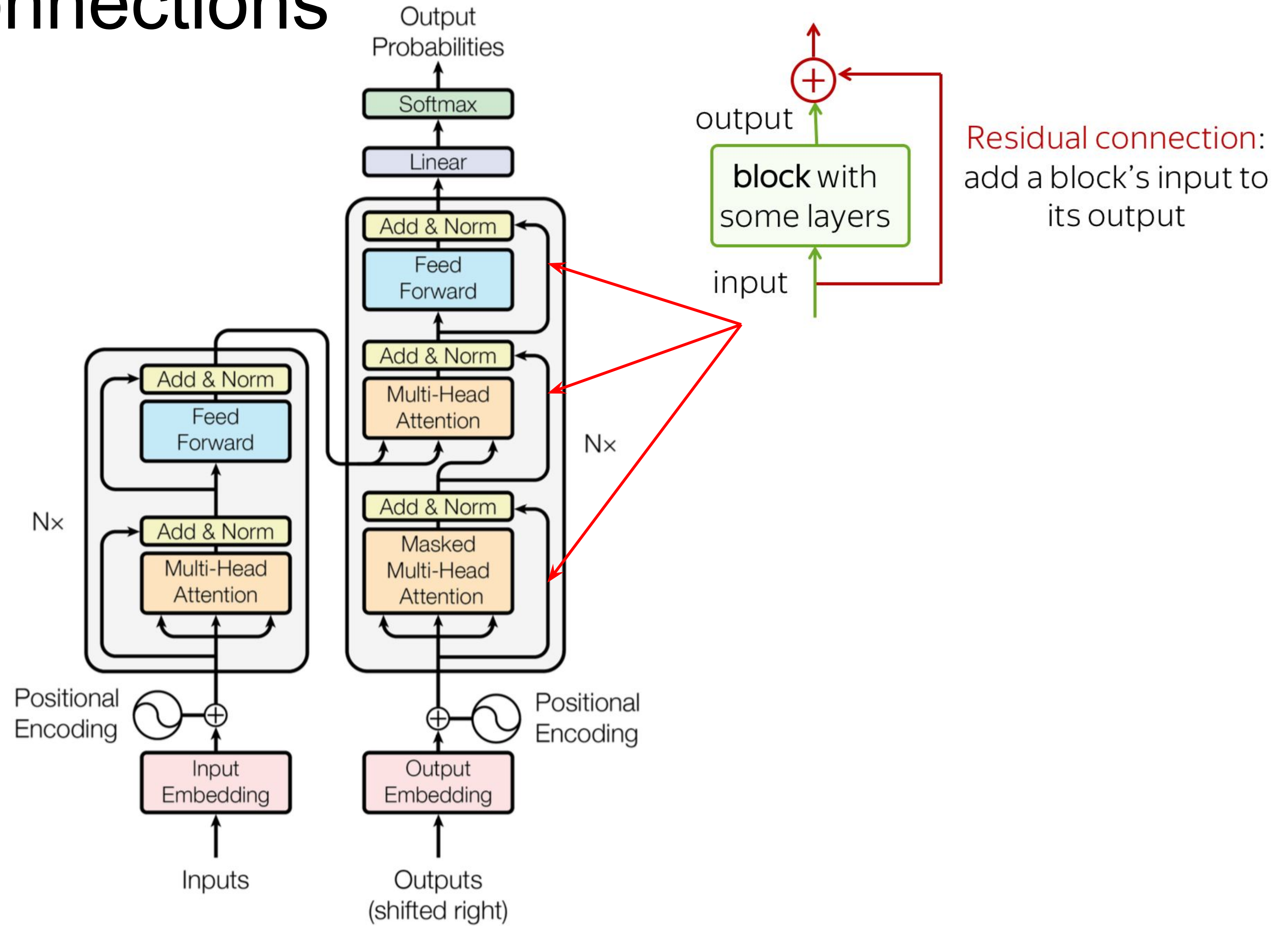
Feed Forward Block



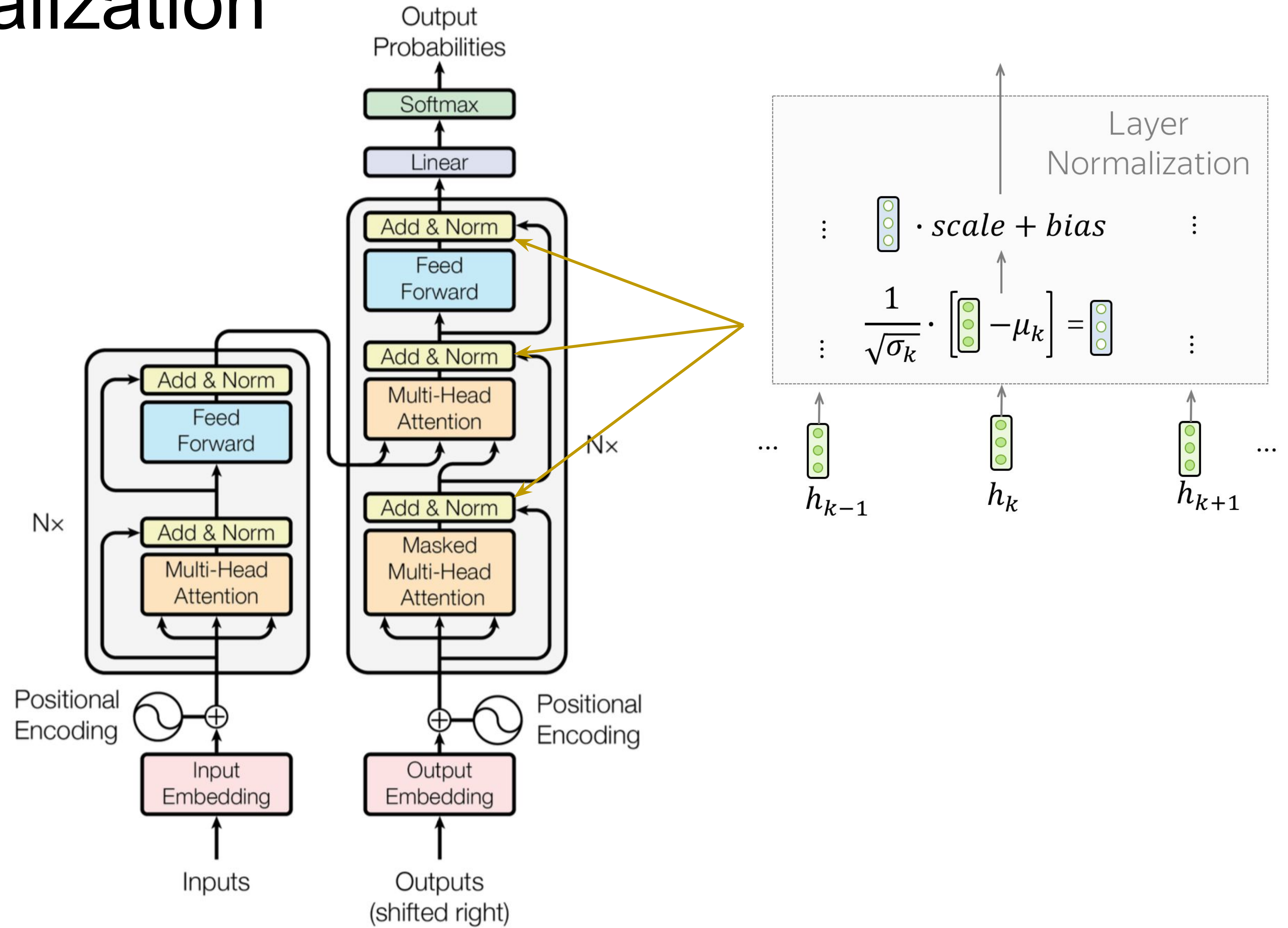
Feed Forward Block



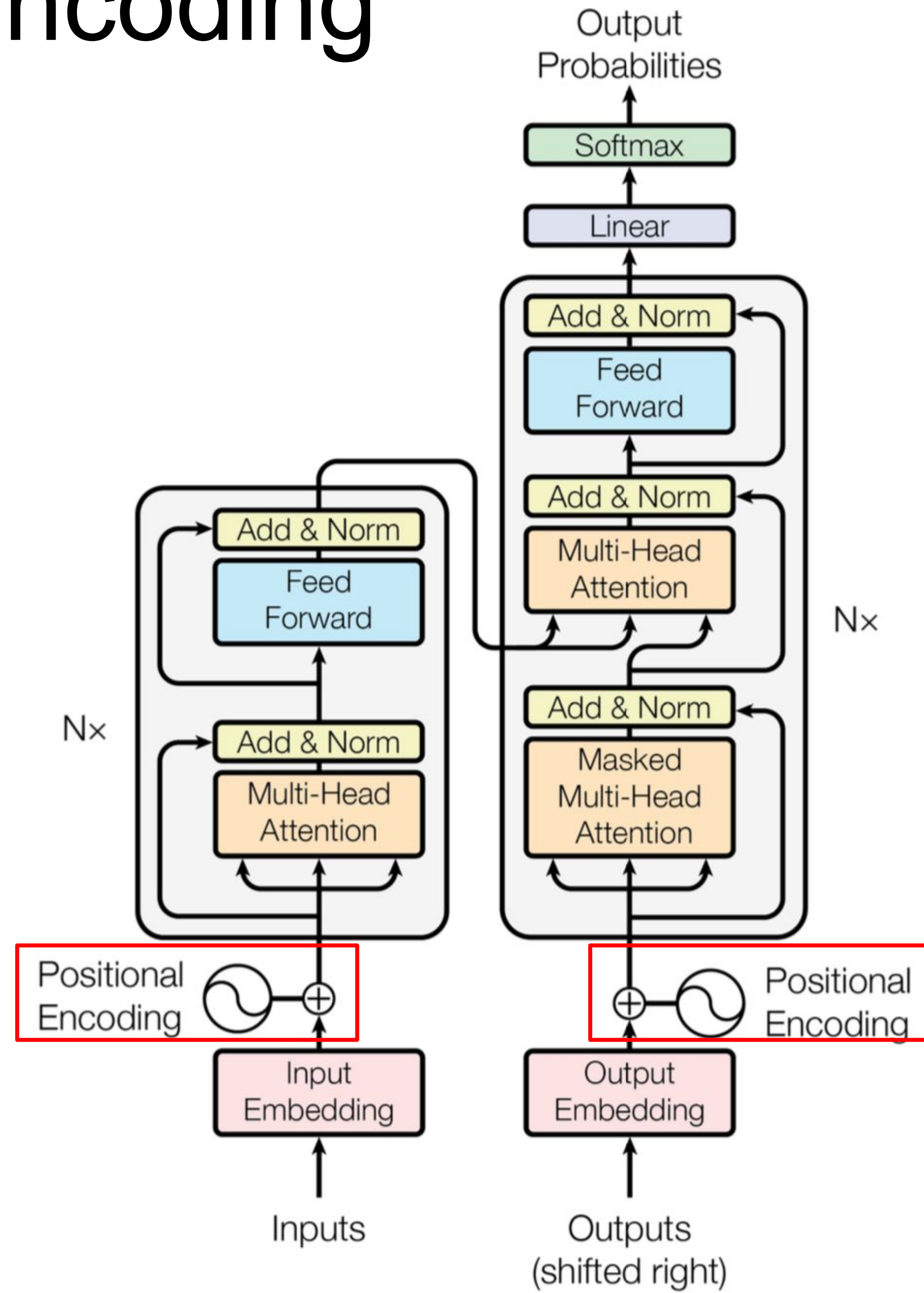
Residual Connections



Layer Normalization



Positional Encoding



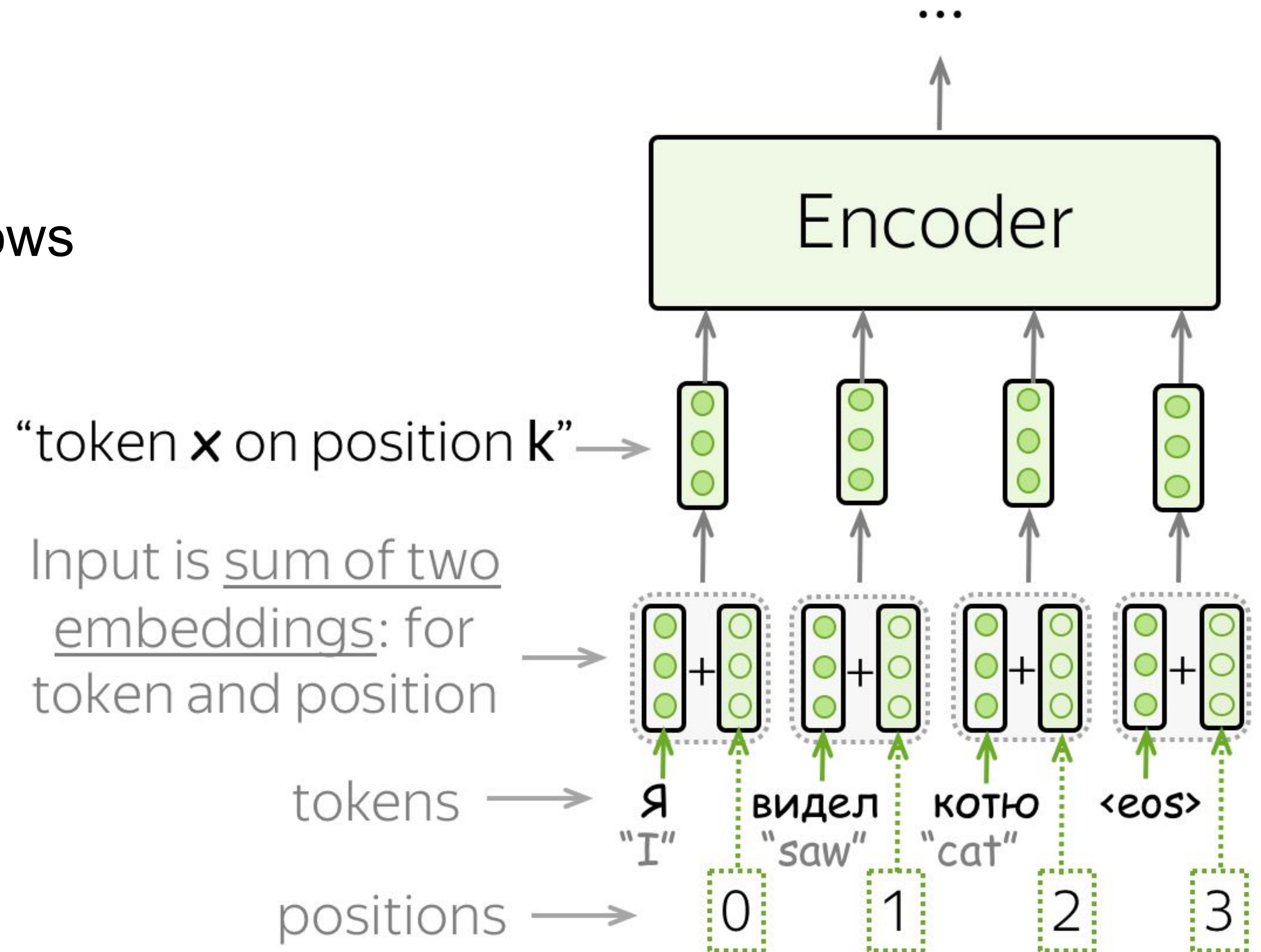
Positional Encoding

Problem:

- without recurrence or convolution, the model knows nothing about position

Solution:

- encode position explicitly and add



Positional Encoding

Fixed encodings:

$$PE_{pos,2i} = \sin(pos/10000^{2i/d_{model}}),$$

$$PE_{pos,2i+1} = \cos(pos/10000^{2i/d_{model}})$$

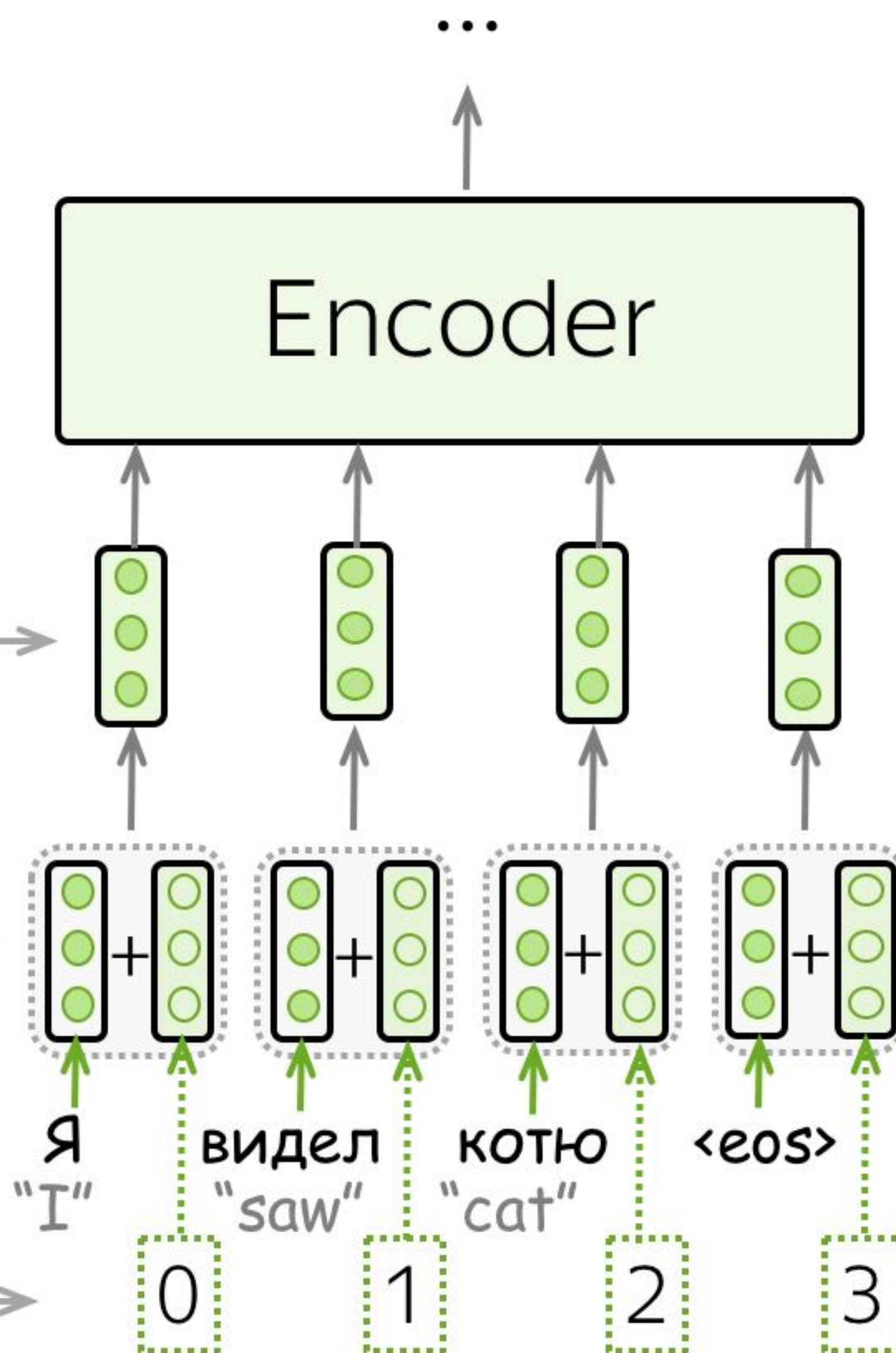
pos – position, i – dimension

“token x on position k ”

Input is sum of two embeddings: for token and position

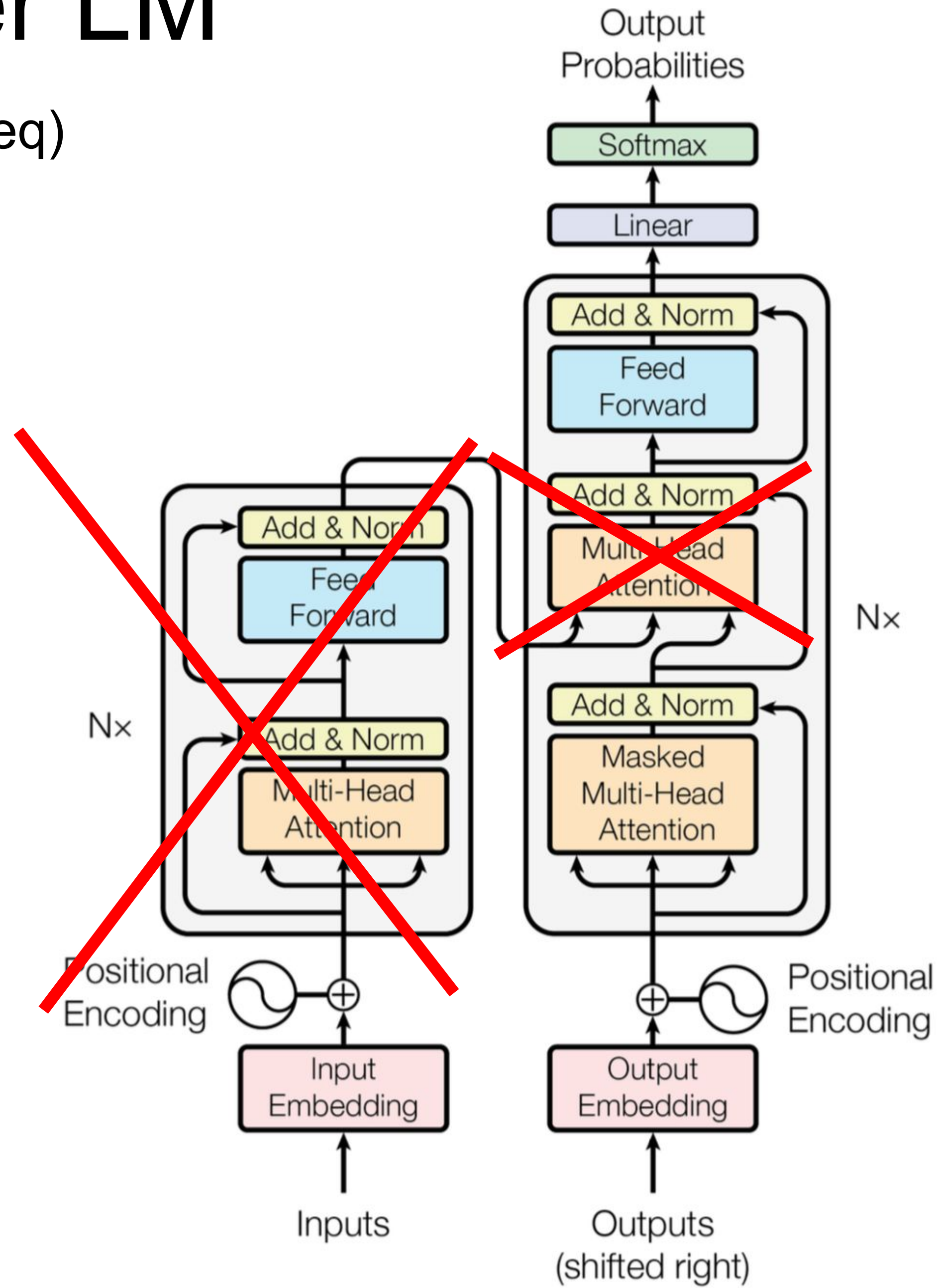
tokens

positions



Transformer LM

(original model, seq2seq)



Complexity of inference

n - length of input sequence, m - length of output sequence

d - size of embedding

RNN: $O(nd^2 + md^2)$

Transformer without caching: $O(n^2d + m^3d)$

Transformer with caching: $O(n^2d + m^2d)$

A detour...

- Seq2seq Basics
- Attention
- Transformer
- Transformer LMs
- BERT and family



- Idea
- Self-Attention
- Masked Self-Attention
- Multi-Head Attention
- KV caching
- Model architecture
-  analysys & interpretability

Analysis Methods

Model-specific:

- Looking at model components
- ...

Model-agnostic:

Analysis Methods

Model-specific:

- Looking at model components
- ...

In the previous lectures:

- Convolutional filters of classifiers
- Neurons in RNN/CNN LMs

Model-agnostic:

Analysis Methods

Model-specific:

- Looking at model components
- ...

Model-agnostic:

In the previous lectures:

- Convolutional filters of classifiers
- Neurons in RNN/CNN LMs

Today:

- Heads in Multi-Head Attention

Analysis Methods

Model-specific:

- Looking at model components
- ...

In the previous lectures:

- Convolutional filters of classifiers
- Neurons in RNN/CNN LMs

Today:

- Heads in Multi-Head Attention

Model-agnostic:

In the previous lecture:

- Look at the predictions: contrastive evaluation of specific phenomena

Analysis Methods

Model-specific:

- Looking at model components
- ...

In the previous lectures:

- Convolutional filters of classifiers
- Neurons in RNN/CNN LMs

Today:

- Heads in Multi-Head Attention

Model-agnostic:

In the previous lecture:

- Look at the predictions: contrastive evaluation of specific phenomena

Today:

- Probing: What do representations capture?

Analysis Methods

Model-specific:

- Looking at model components
- ...

In the previous lectures:

- Convolutional filters of classifiers
- Neurons in RNN/CNN LMs

Today:

- Heads in Multi-Head Attention

Model-agnostic:

In the previous lecture:

- Look at the predictions: contrastive evaluation of specific phenomena

Today:

- Probing: What do representations capture?

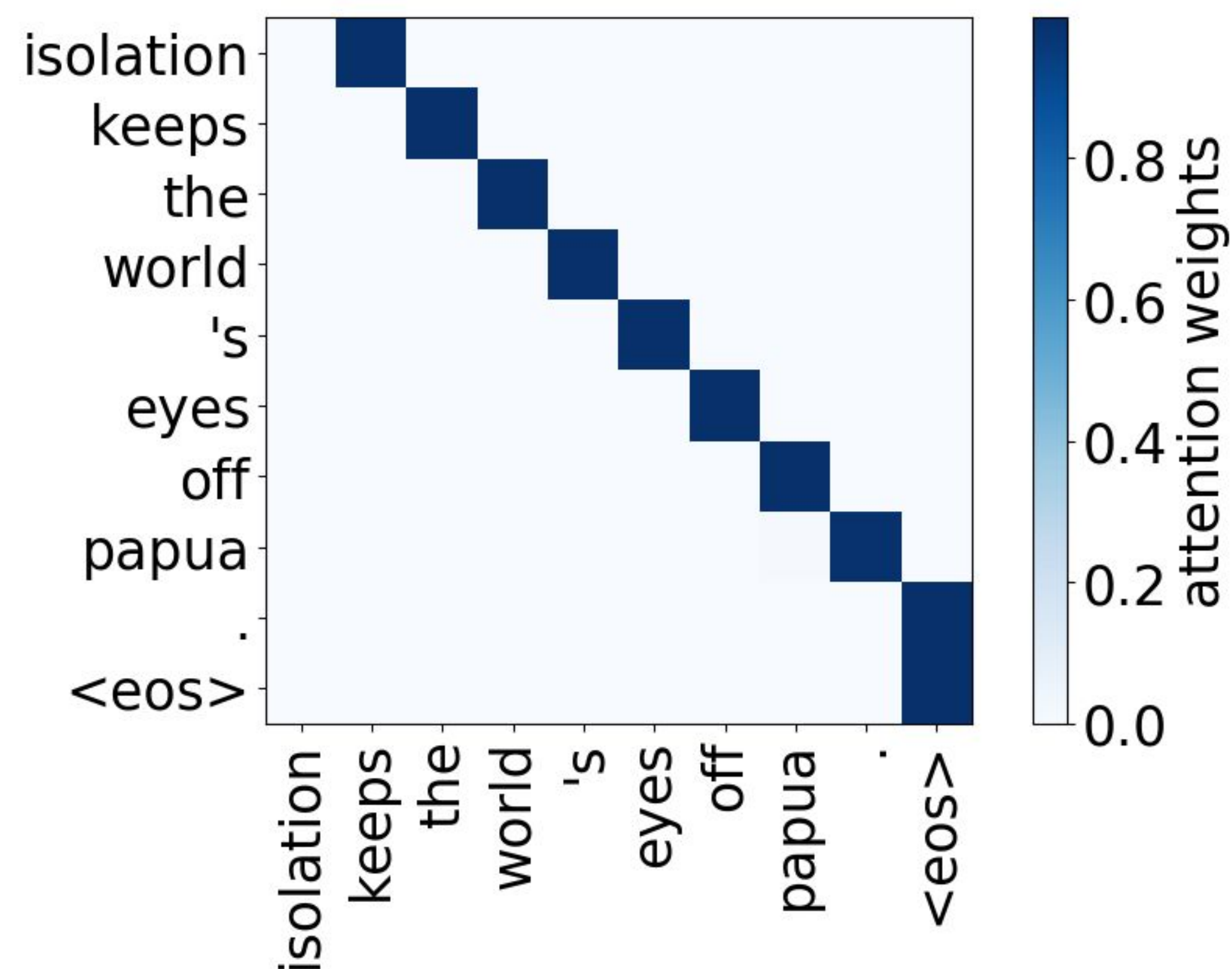
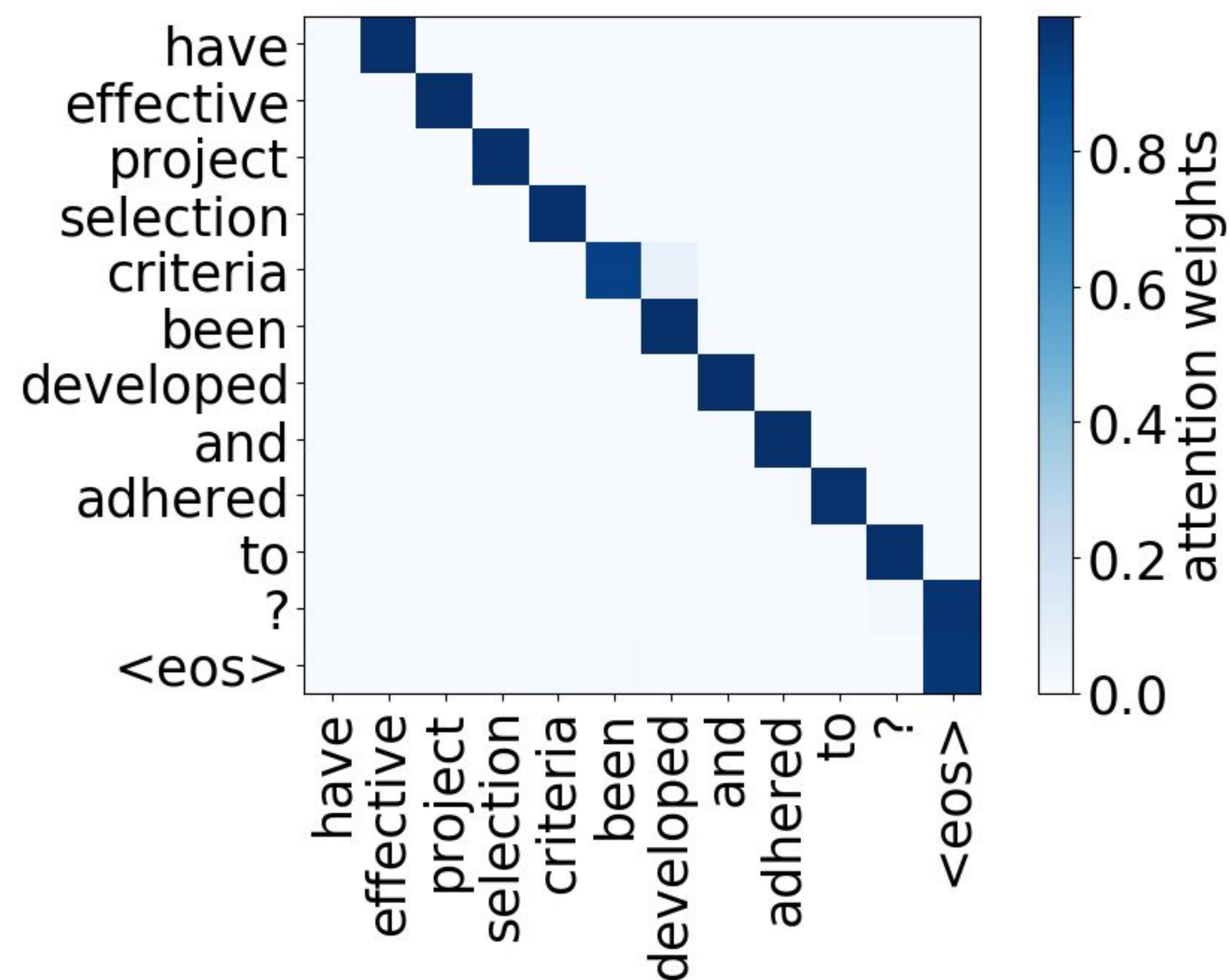
Multi-Head Attention: What are the Heads Doing?

Heads which on average contribute more to generated translations (“important heads”) play interpretable roles:

- Positional
- Syntactic
- Attention to rare tokens

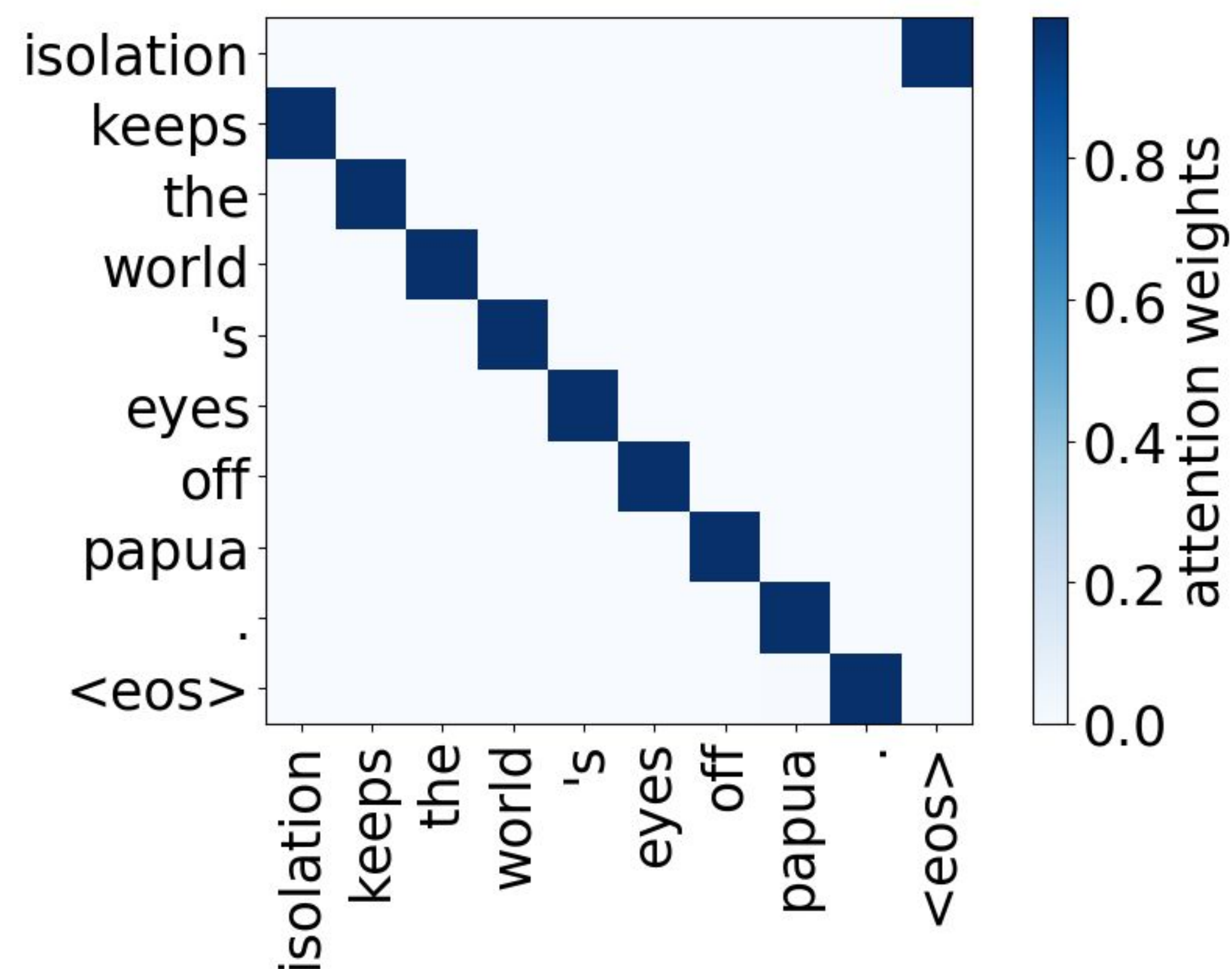
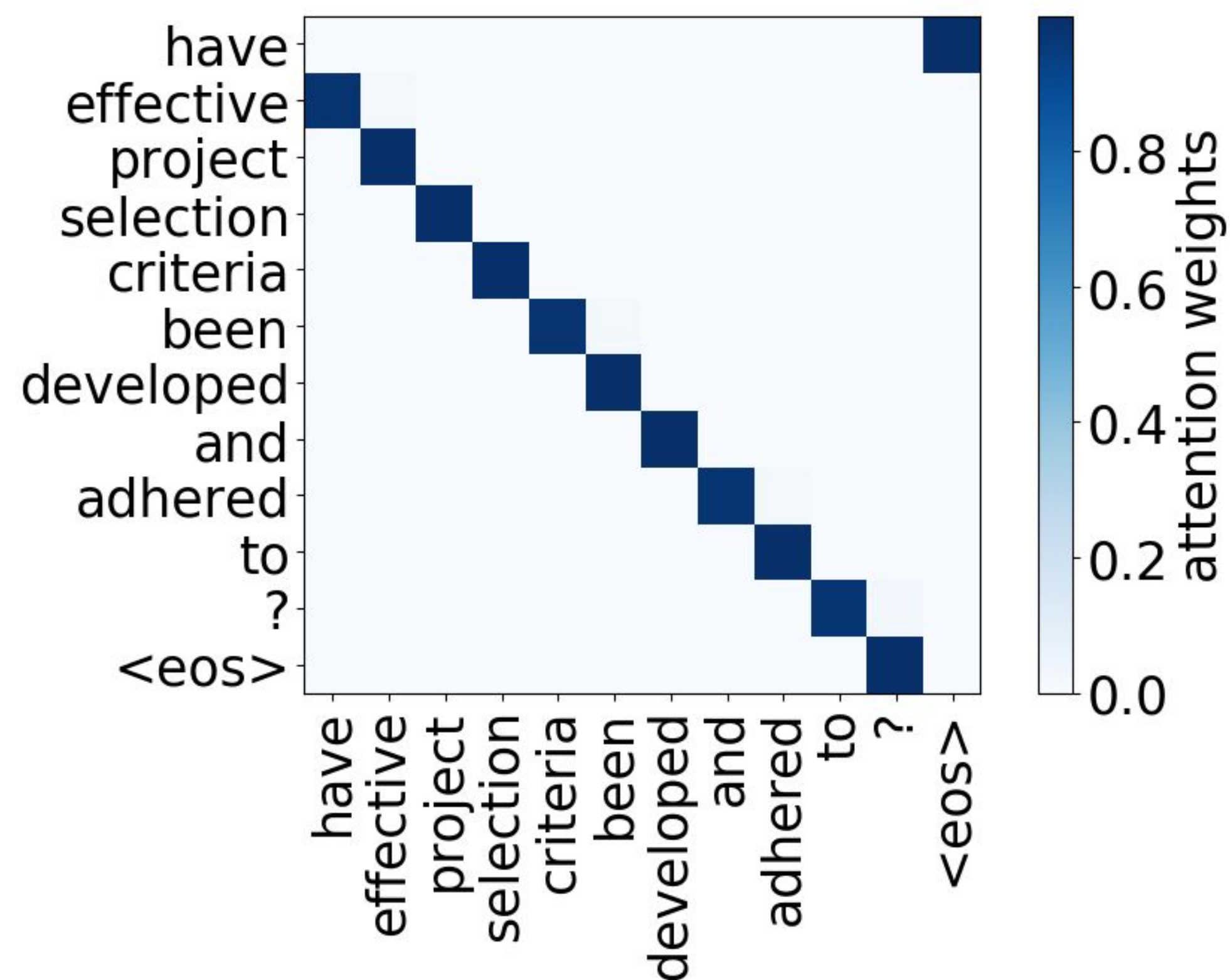
Paper: Analyzing Multi-Head Self-Attention:
Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned

Positional Heads: Attention to Neighbors



Paper: Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned

Positional Heads: Attention to Neighbors



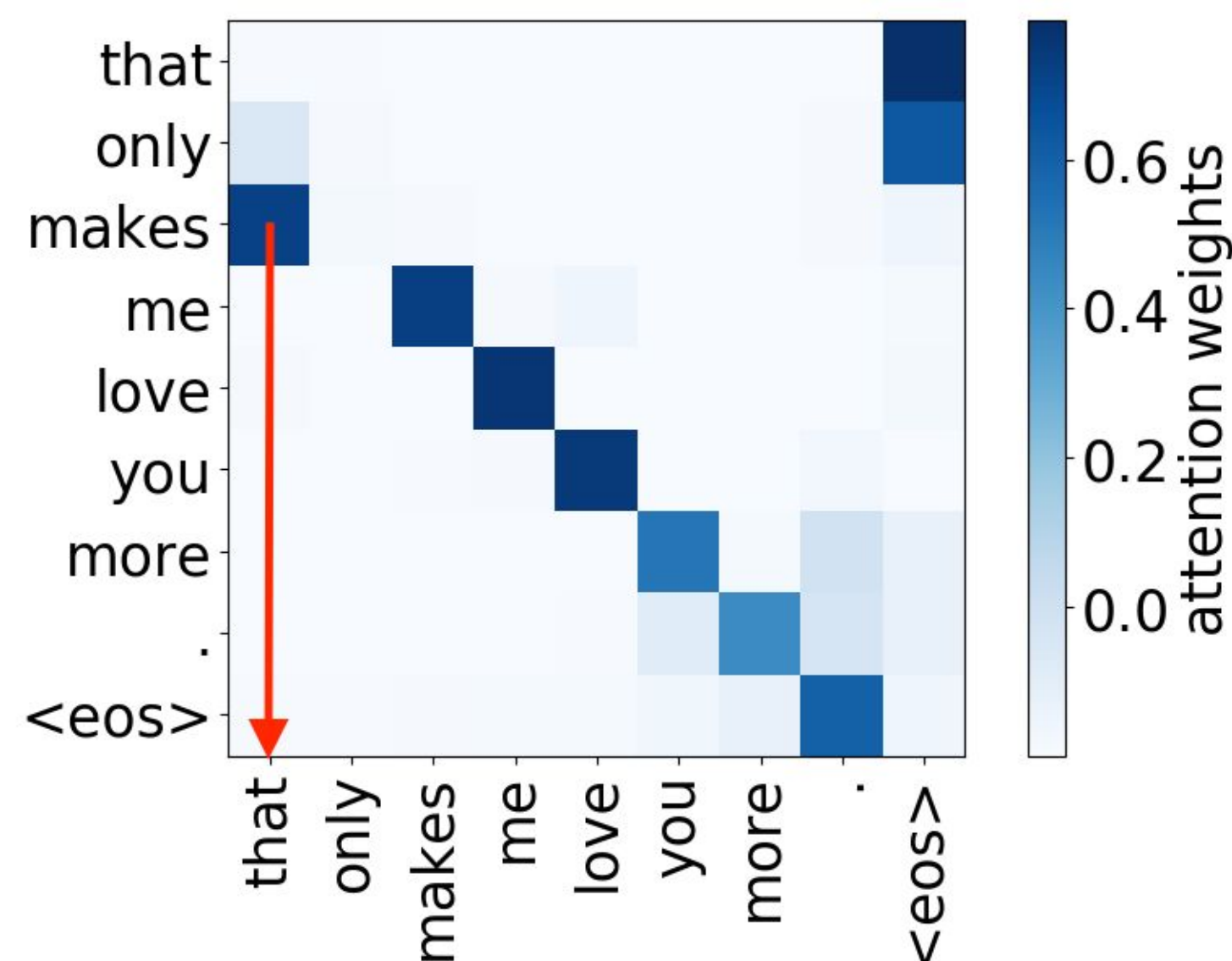
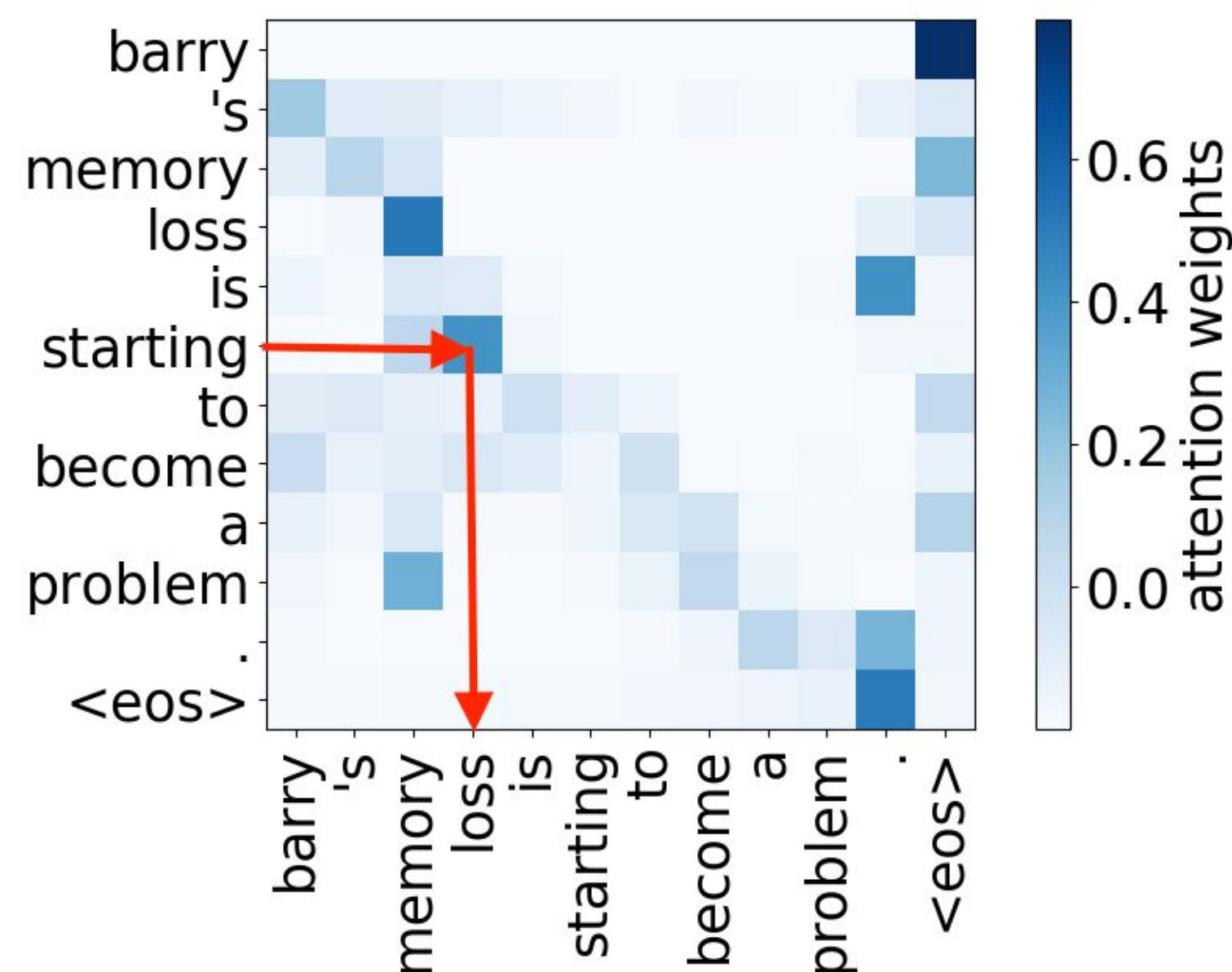
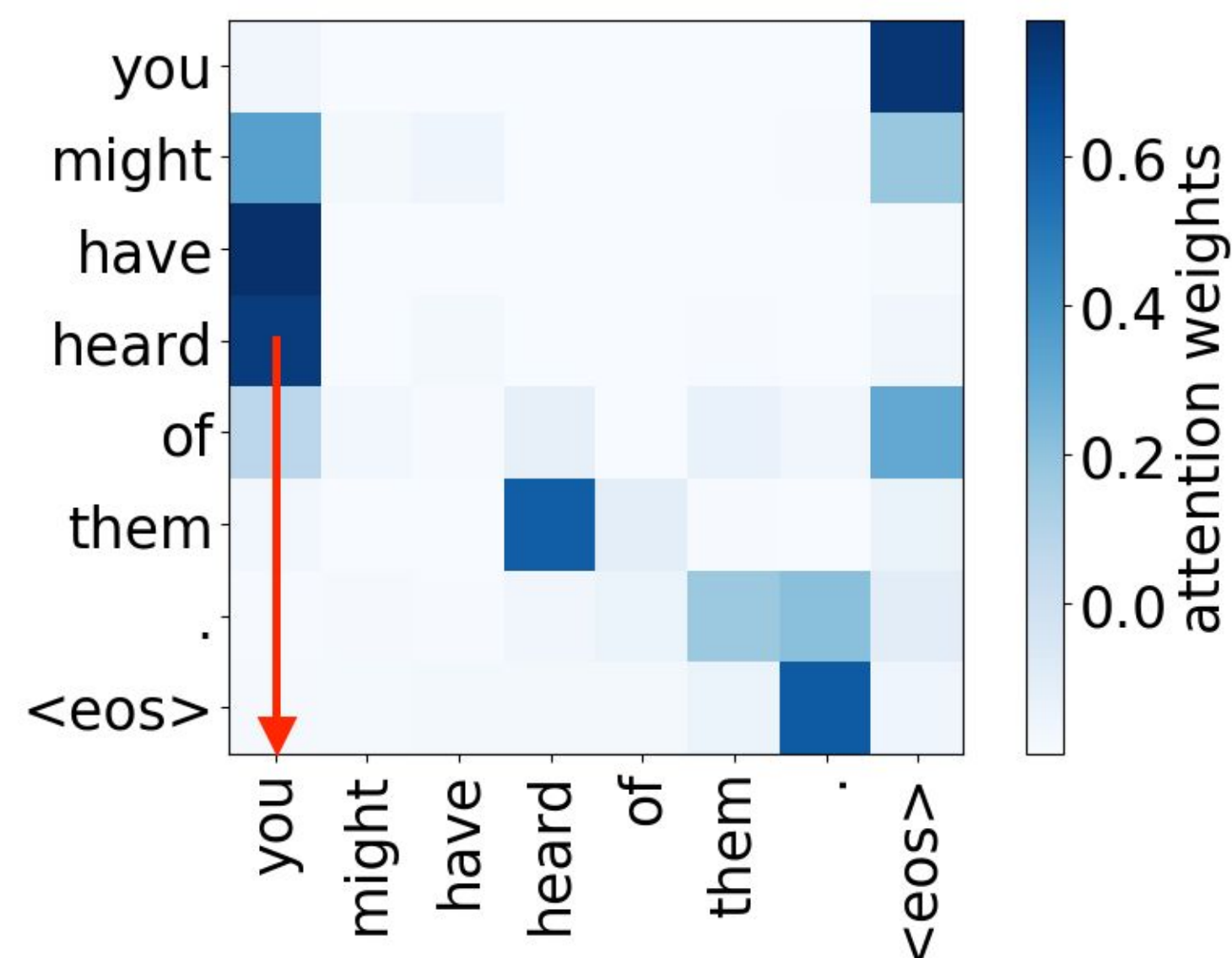
Paper: Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be

Syntactic Heads: Track Dependencies

- verb->subject

Она руководит **новым** проектом

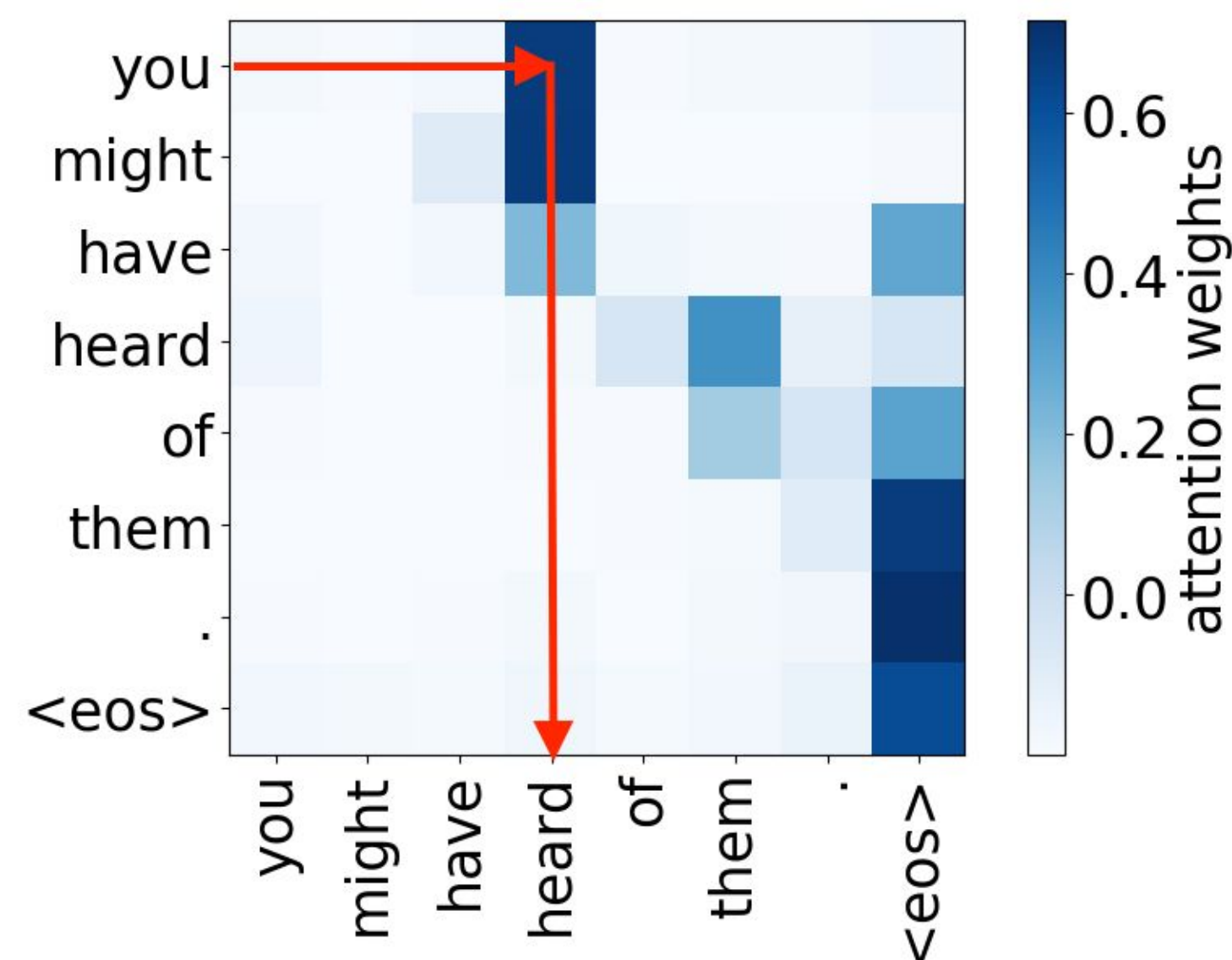
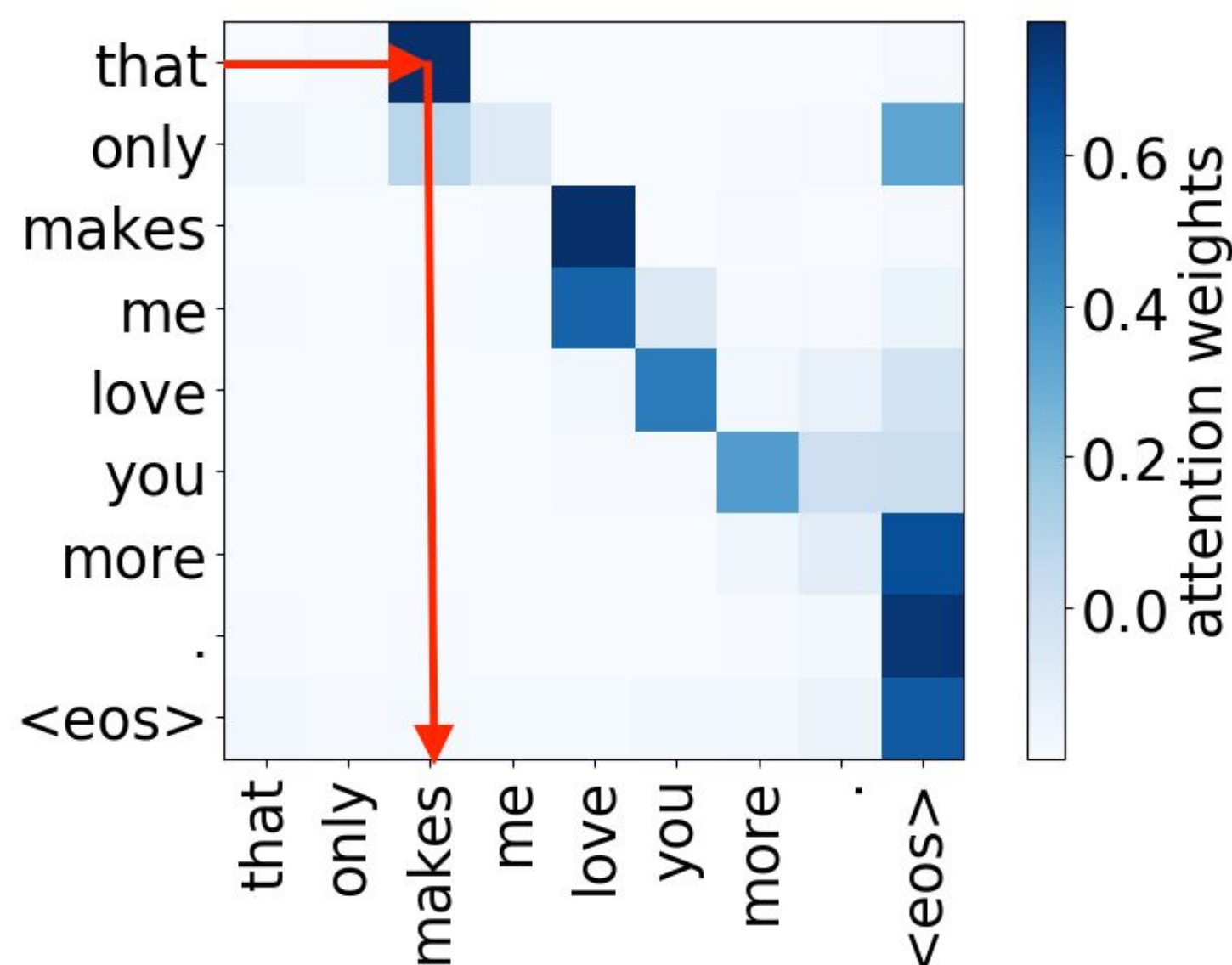
- Gender agreement
- Case government
- Lexical preferences
- ...



Paper: Analyzing Multi-Head Self-Attention:
Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned

Syntactic Heads: Track Dependencies

- subject->verb



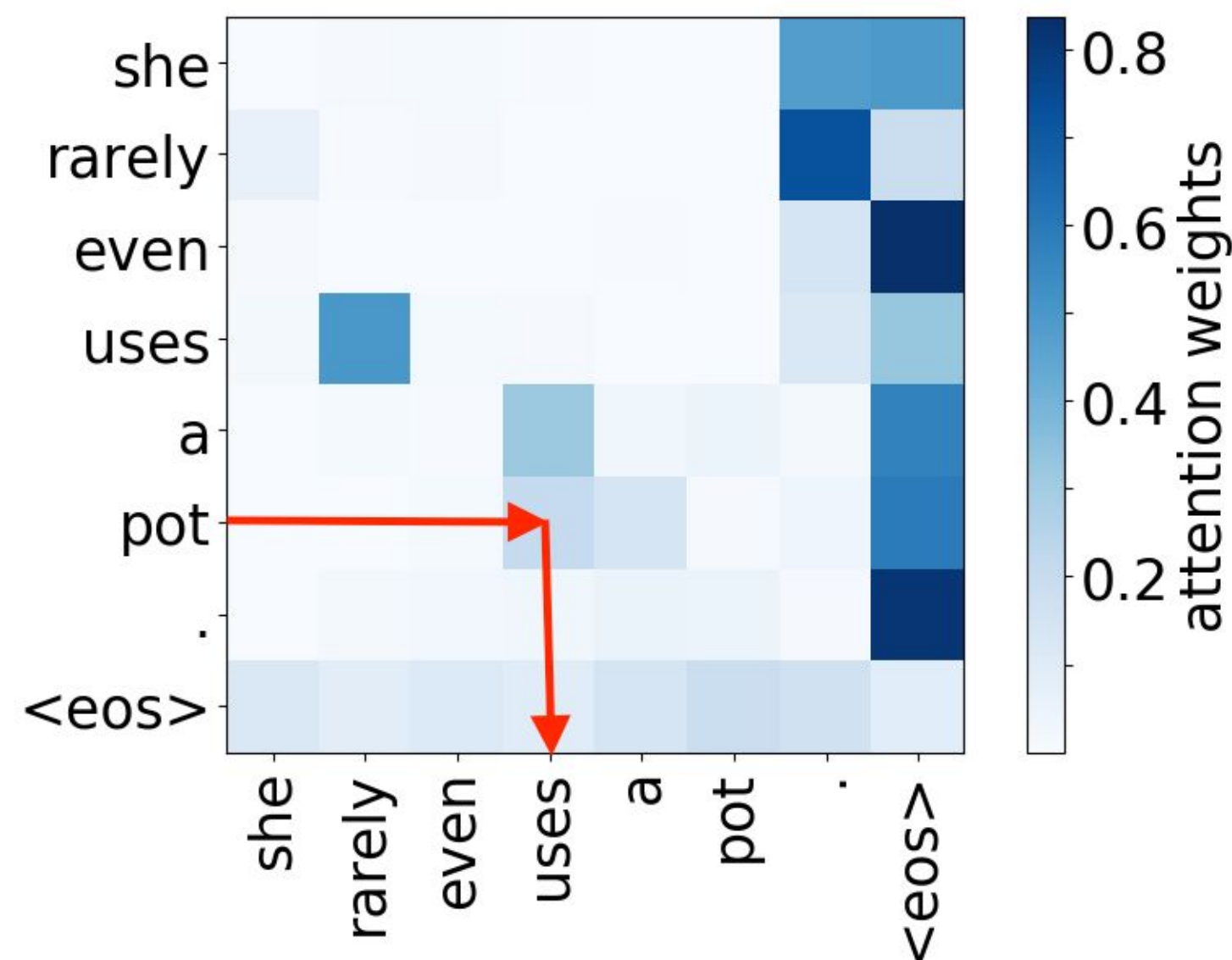
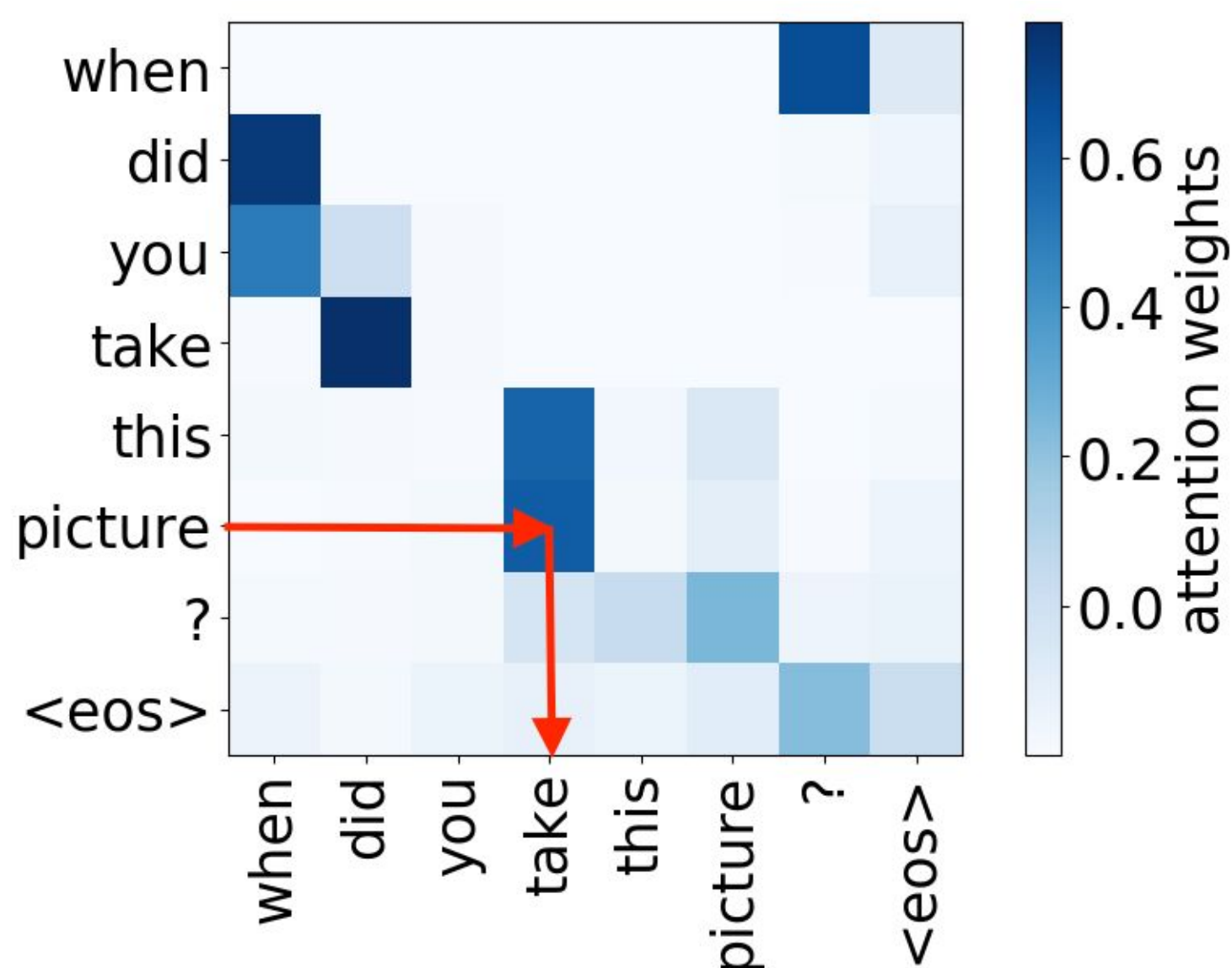
Она руководит **новым** проектом

- Gender agreement
- Case government
- Lexical preferences
- ...

Paper: Analyzing Multi-Head Self-Attention:
Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned

Syntactic Heads: Track Dependencies

- object->verb

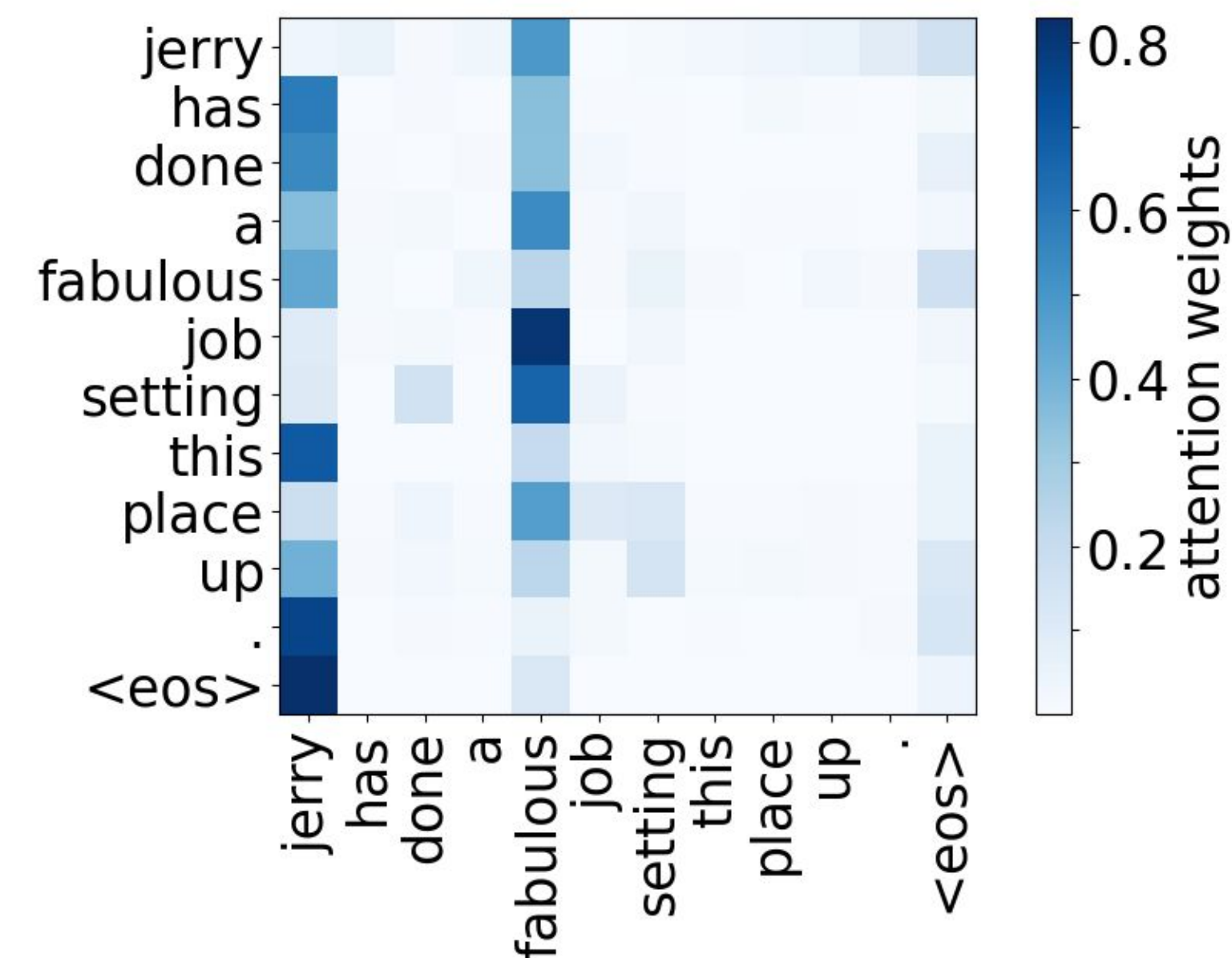
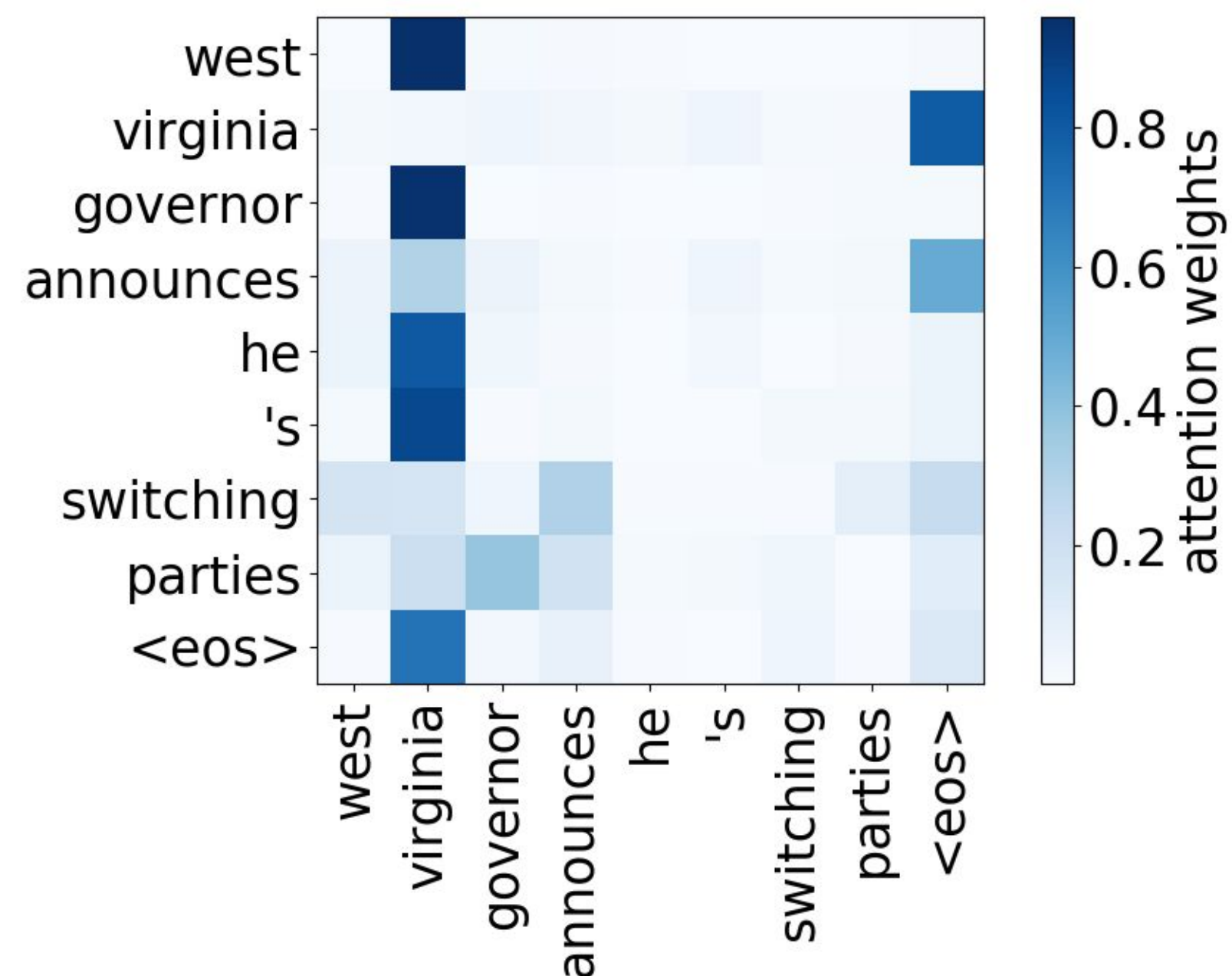
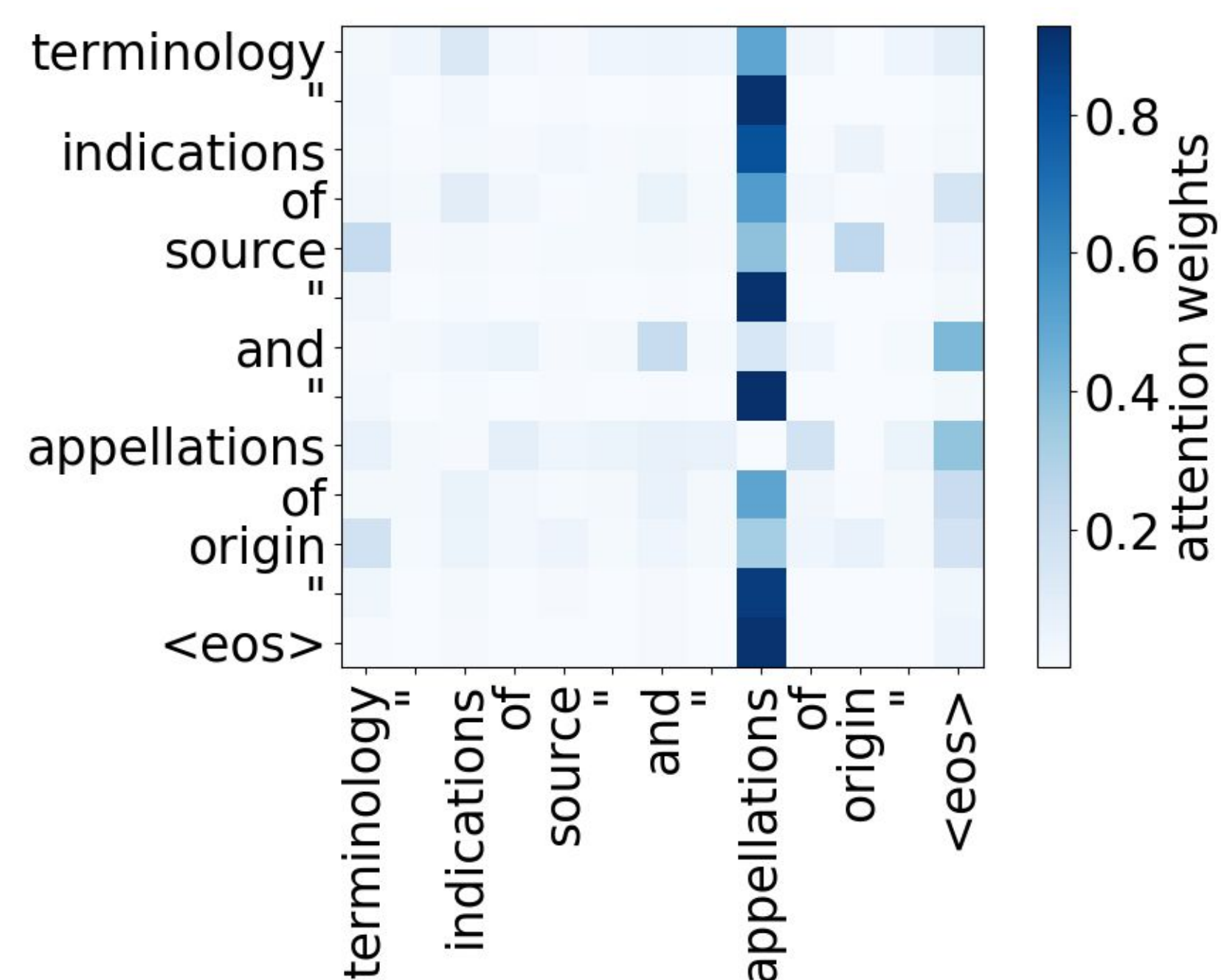


Она руководит **новым** проектом

- Gender agreement
- Case government
- Lexical preferences
- ...

Paper: Analyzing Multi-Head Self-Attention:
Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned

Attention to Rare Tokens



Paper: Analyzing Multi-Head Self-Attention:
Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned

Analysis Methods

Model-specific:

- Looking at model components
- ...

In the previous lectures:

- Convolutional filters of classifiers
- Neurons in RNN/CNN LMs

Today:

- Heads in Multi-Head Attention

Model-agnostic:

In the previous lecture:

- Look at the predictions: contrastive evaluation of specific phenomena

Today:

- Probing: What do representations capture?

Analysis Methods

Model-specific:

- Looking at model components
- ...

In the previous lectures:

- Convolutional filters of classifiers
- Neurons in RNN/CNN LMs

Today:

- Heads in Multi-Head Attention

Model-agnostic:

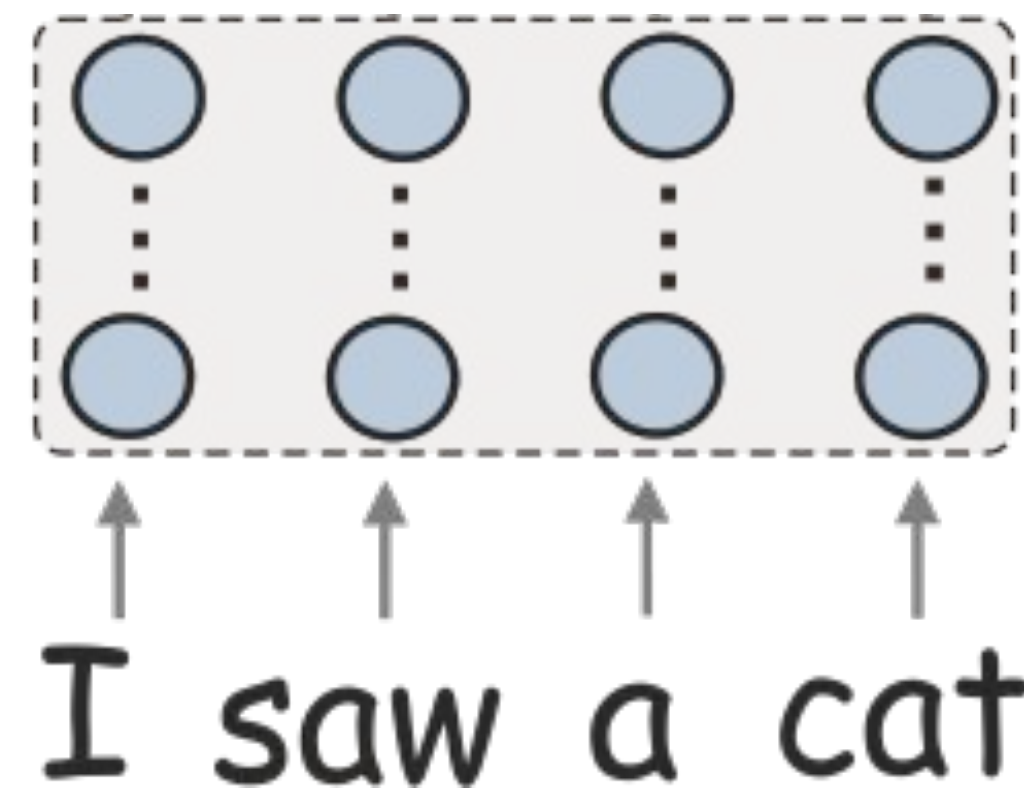
In the previous lecture:

- Look at the predictions: contrastive evaluation of specific phenomena

Today:

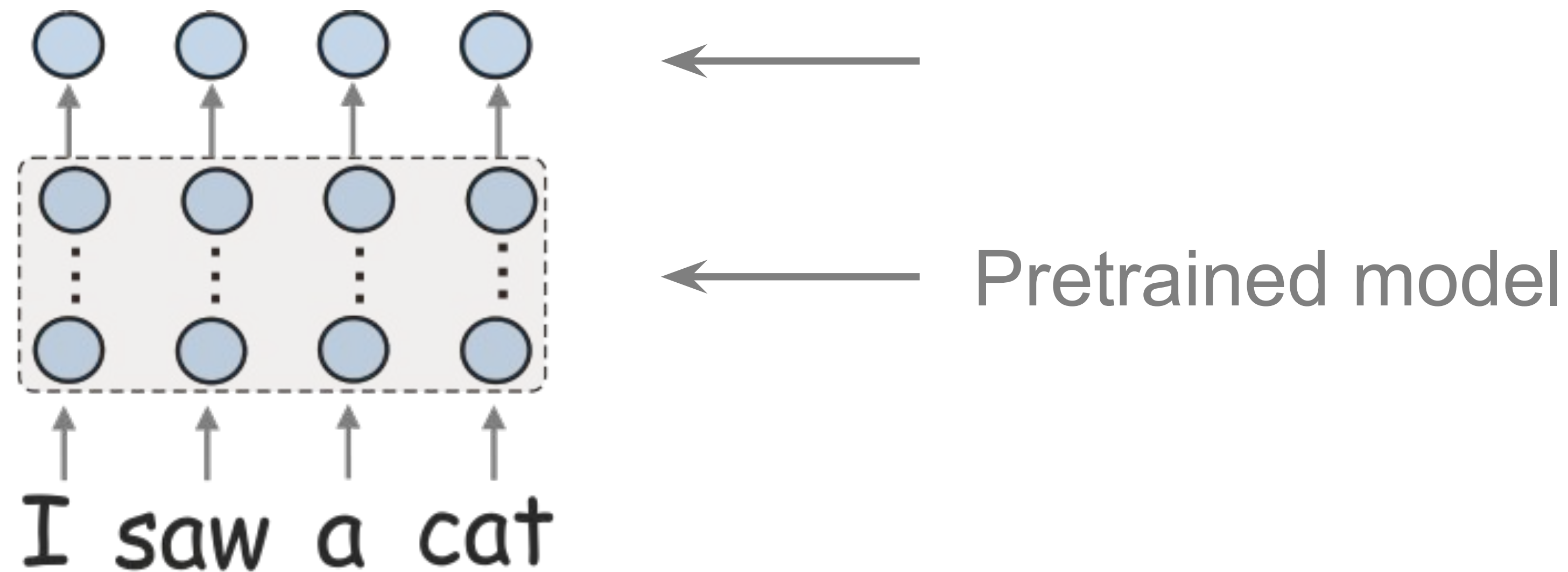
- Probing: What do representations capture?

How to understand if a model captures a linguistic property?

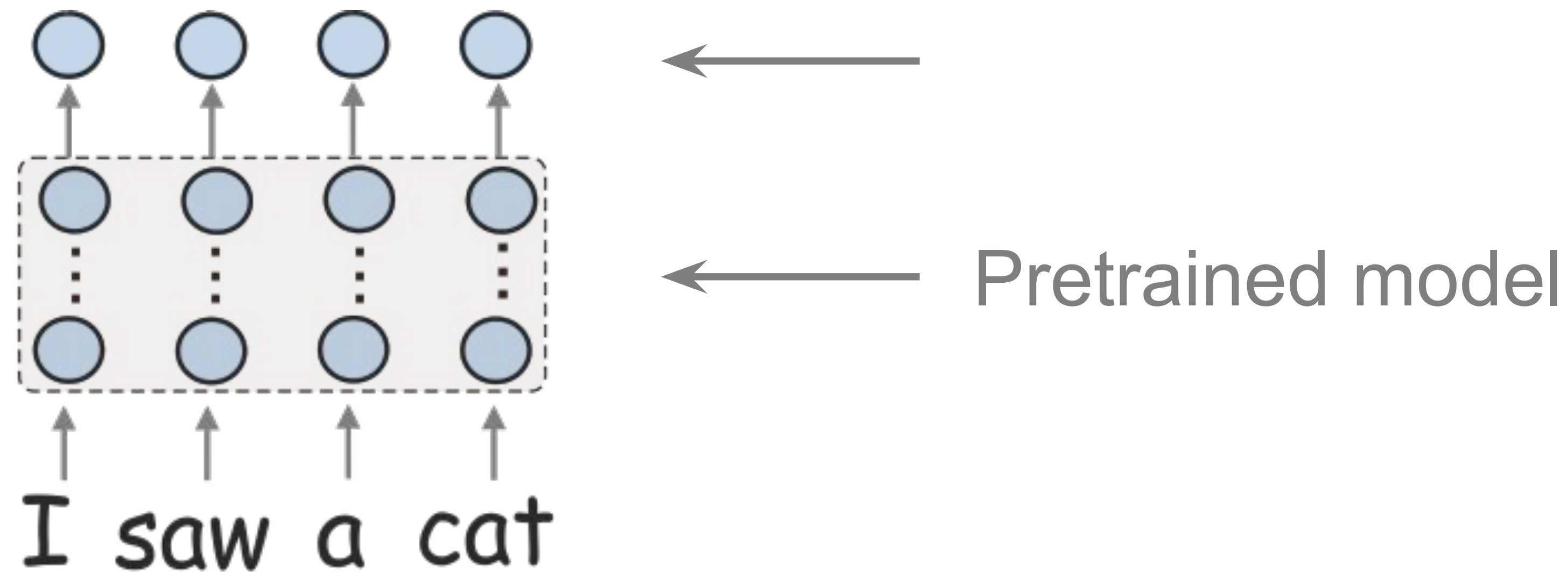
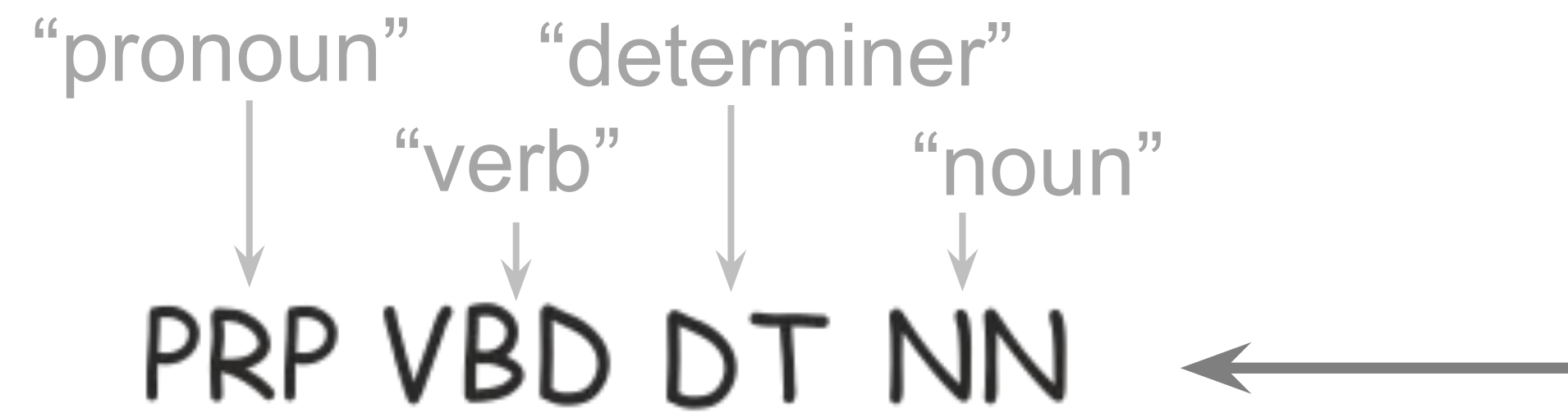


← Pretrained model

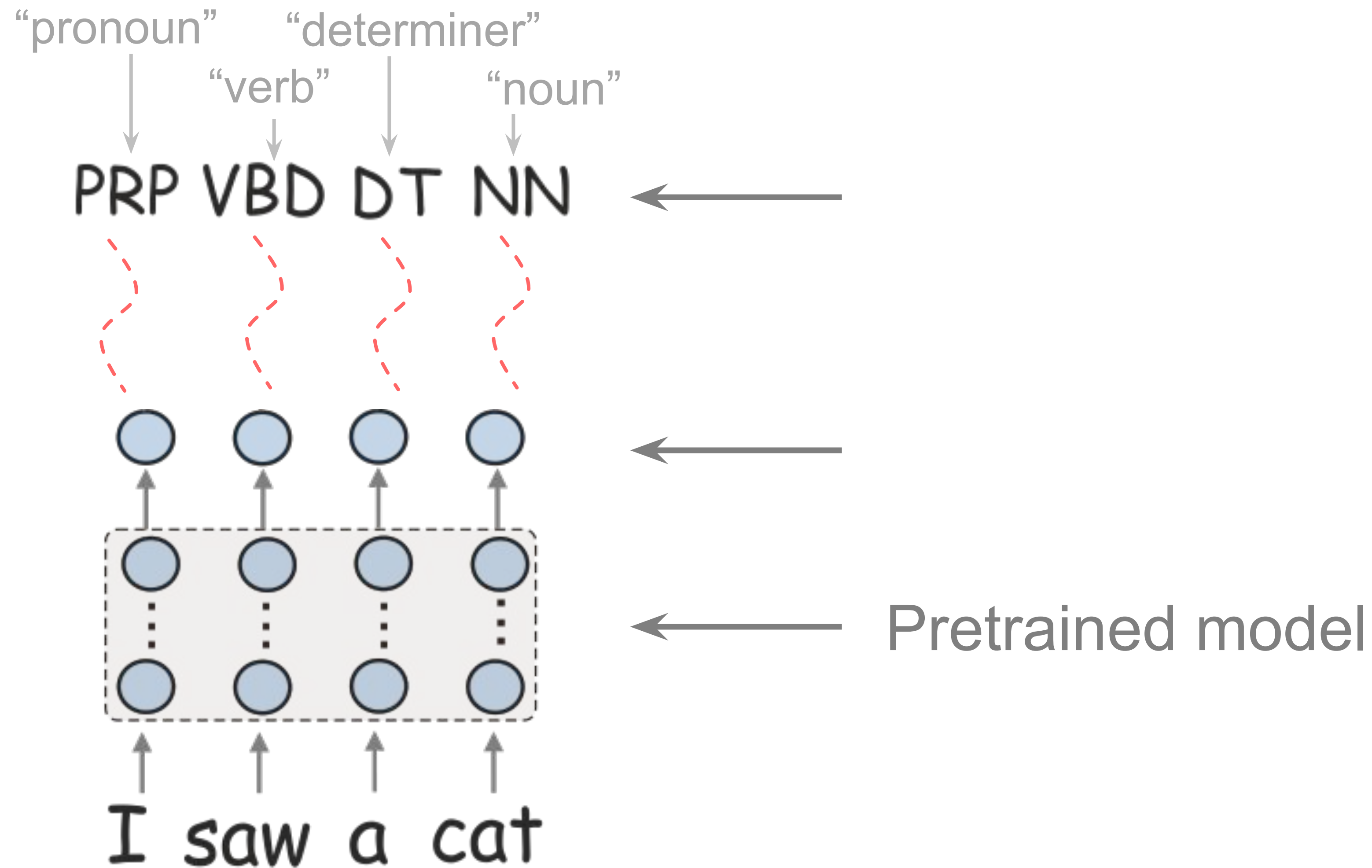
How to understand if a model captures a linguistic property?



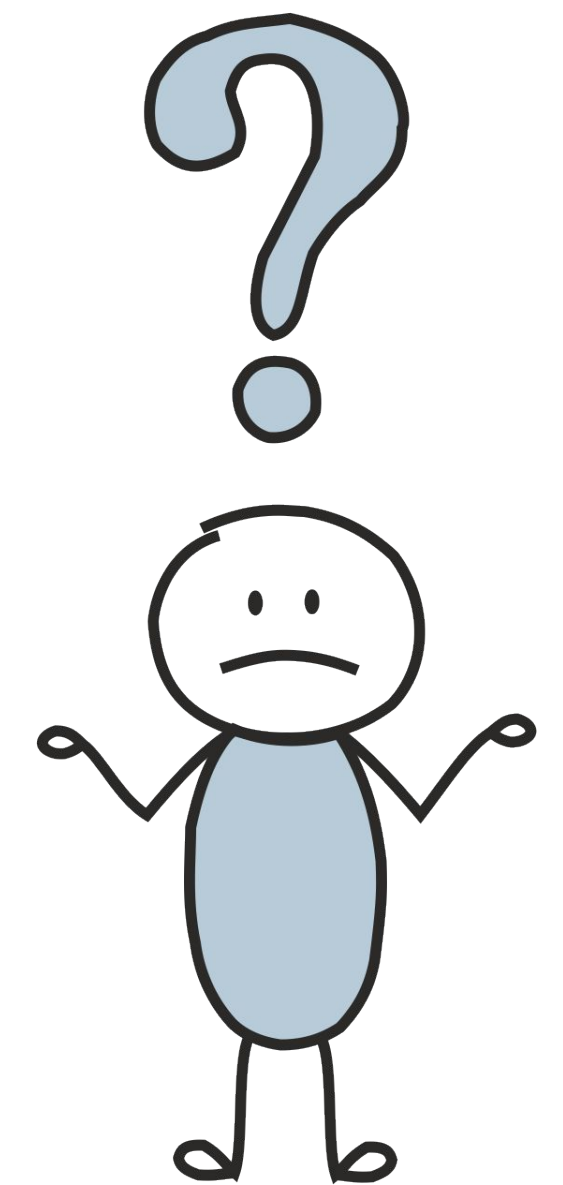
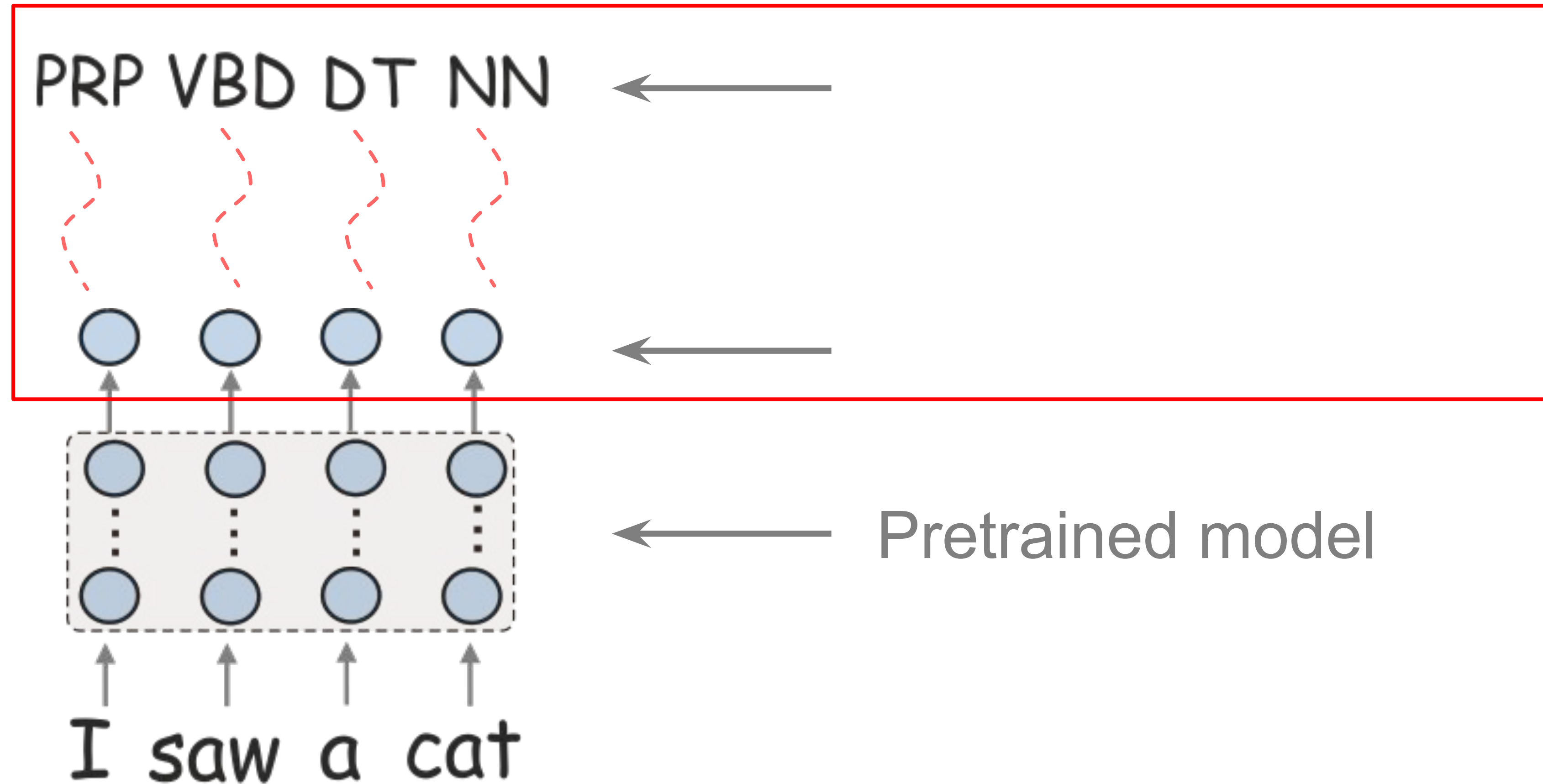
How to understand if a model captures a linguistic property?



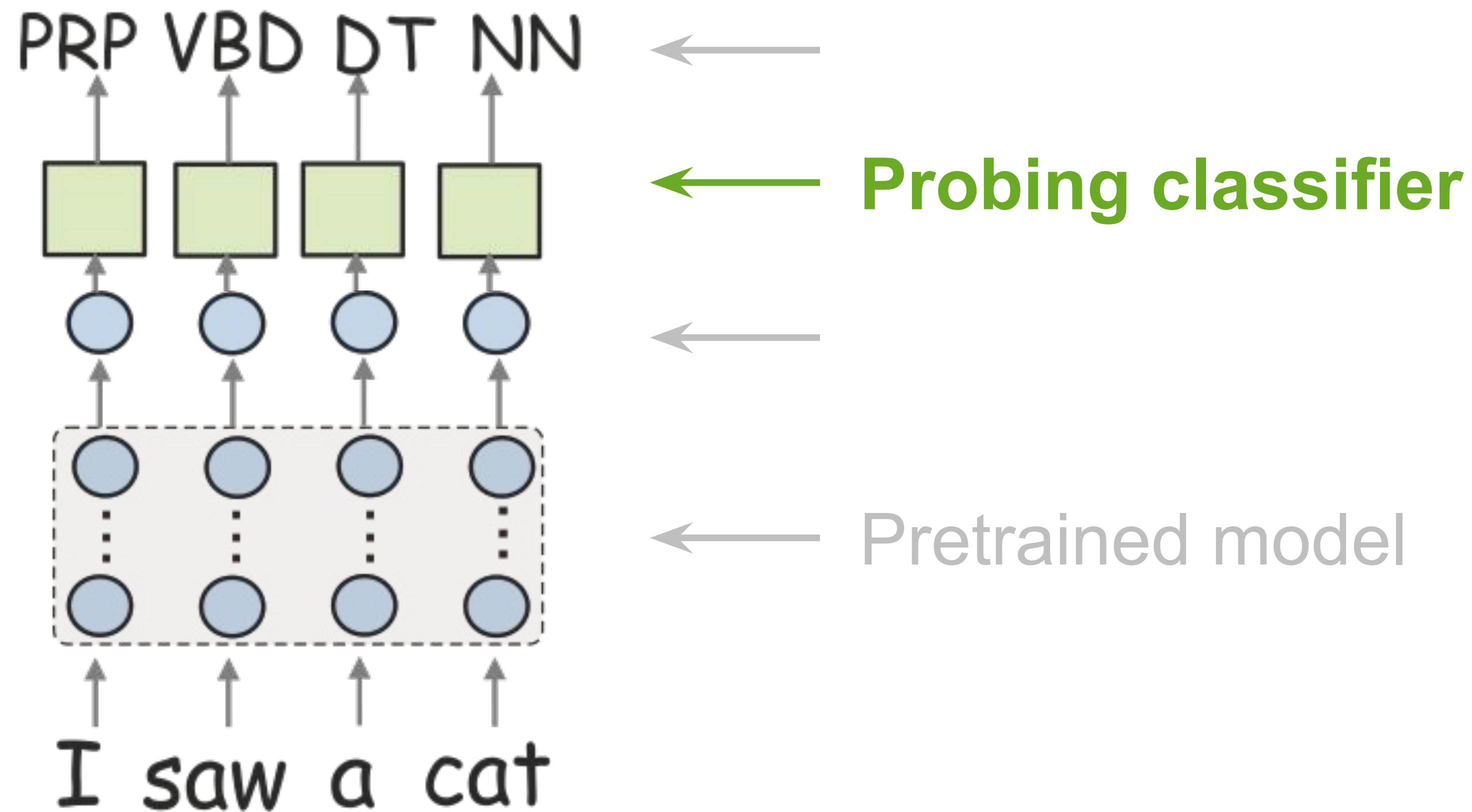
How to understand if a model captures a linguistic property?



How to understand if a model captures a linguistic property?

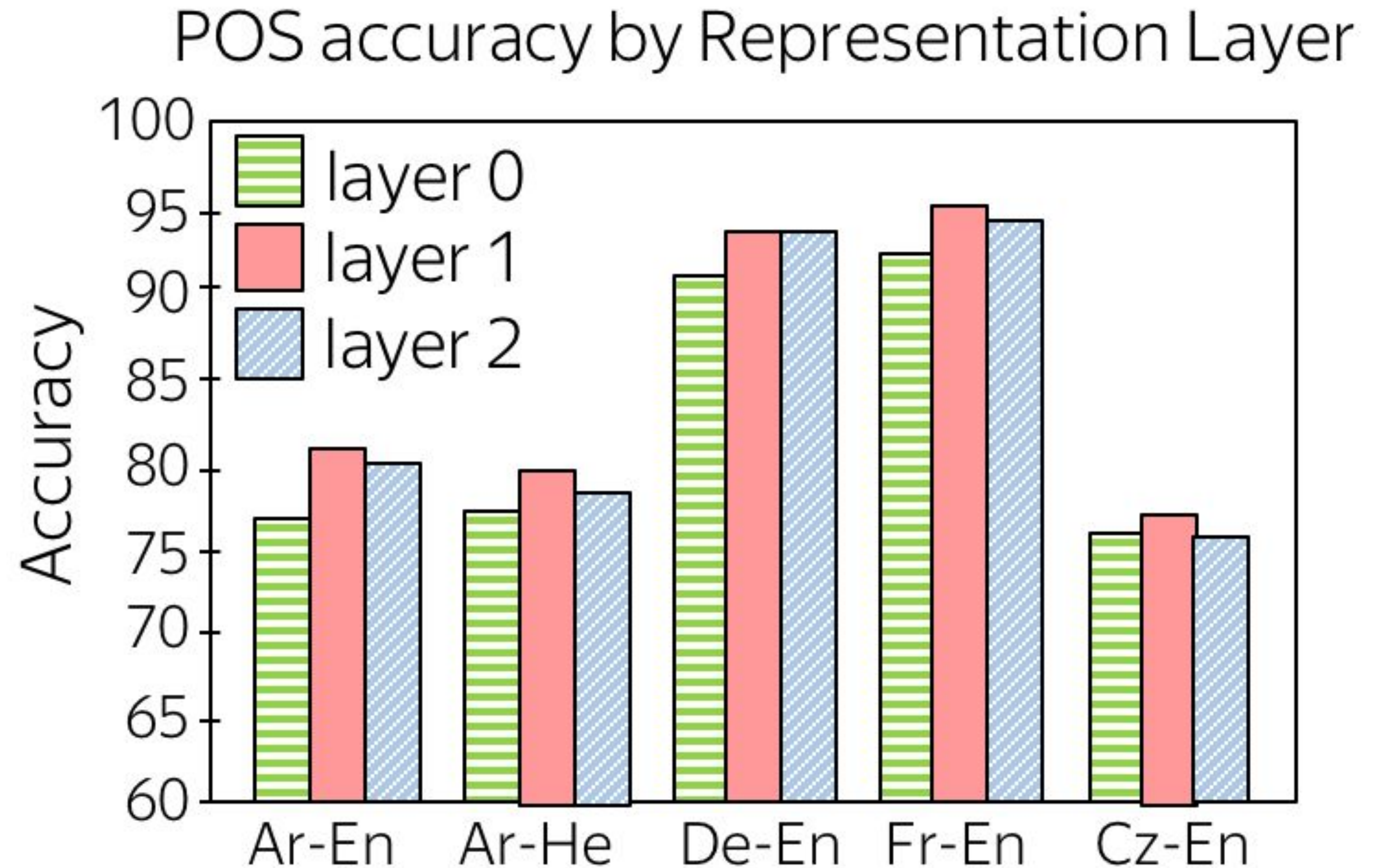


Standard Probing: train a classifier, use its accuracy



What Do NMT Models Learn About Morphology?

- Take NMT models for different language pairs
- Look at the encoder layers



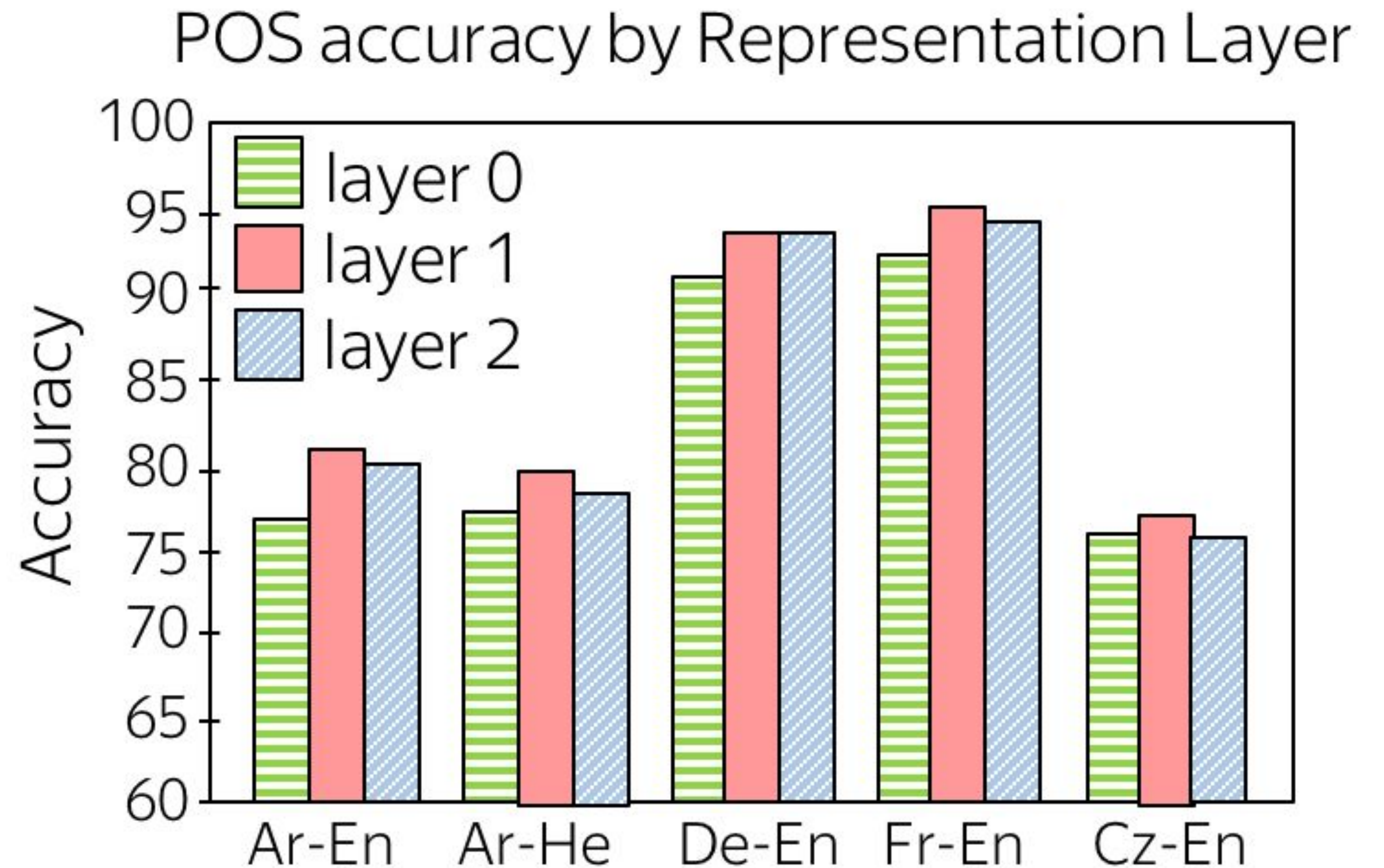
Paper: [What do Neural Machine Translation Models Learn about Morphology?](#)

What Do NMT Models Learn About Morphology?

- Take NMT models for different language pairs
- Look at the encoder layers

Results:

- Encoding helps: layer 0 (word embeddings) is the worst



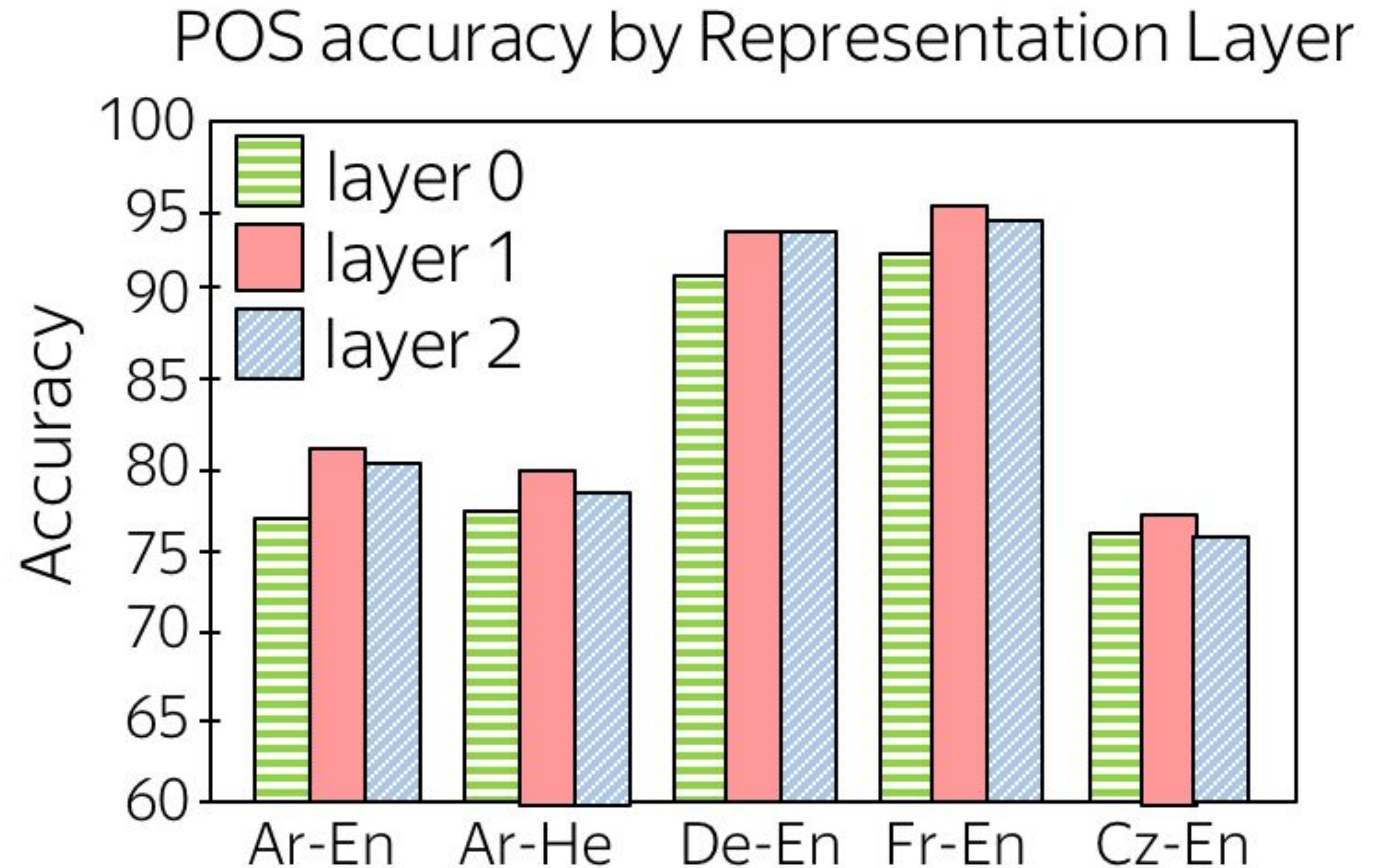
Paper: What do Neural Machine Translation Models Learn about Morphology?

What Do NMT Models Learn About Morphology?

- Take NMT models for different language pairs
- Look at the encoder layers

Results:

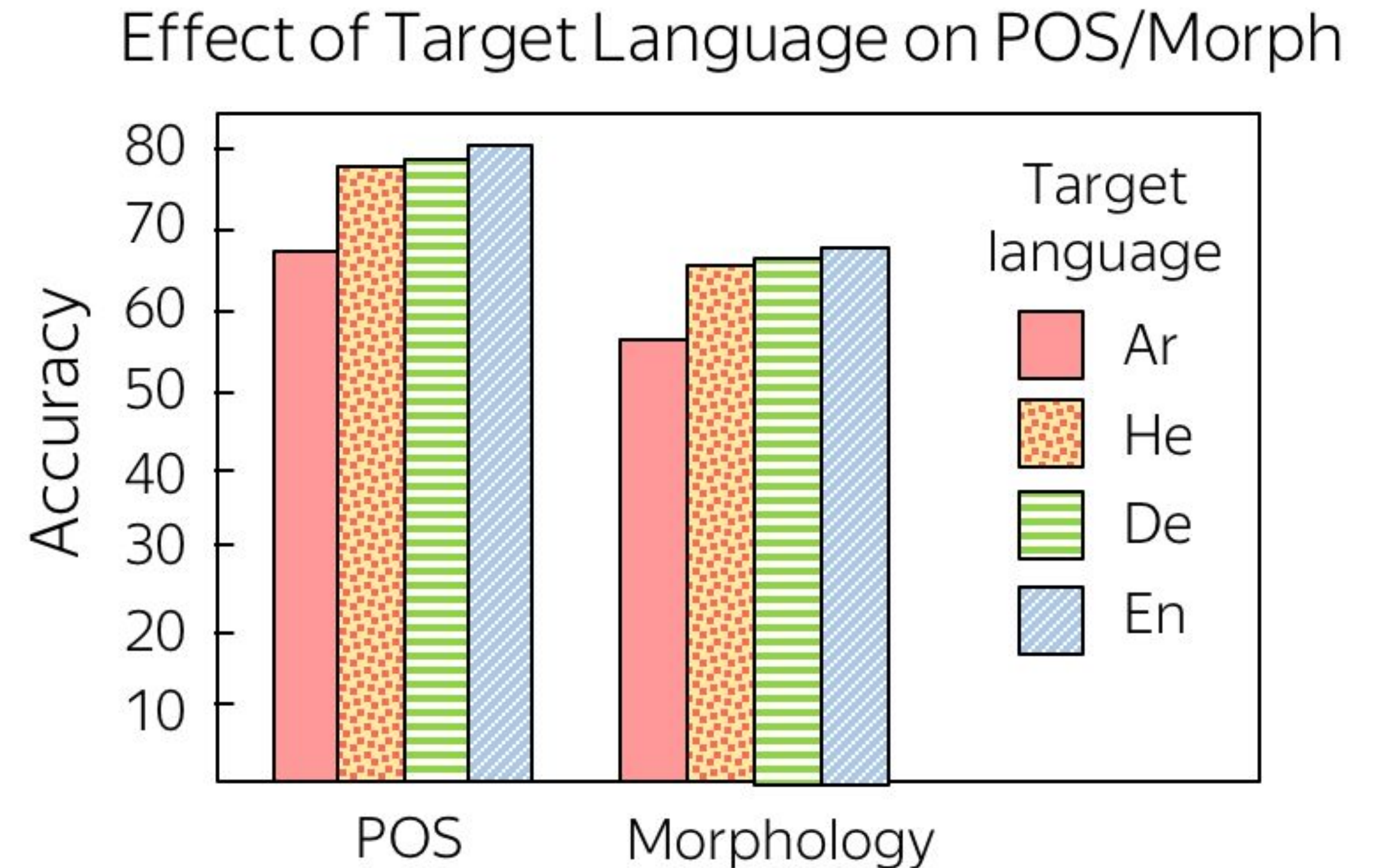
- Encoding helps: layer 0 (word embeddings) is the worst
- Layer 1 is better than layer 2



Paper: What do Neural Machine Translation Models Learn about Morphology?

What Do NMT Models Learn About Morphology?

- Take NMT models with the same source language and different target languages
- Look at the encoder



Paper: [What do Neural Machine Translation Models Learn about Morphology?](#)

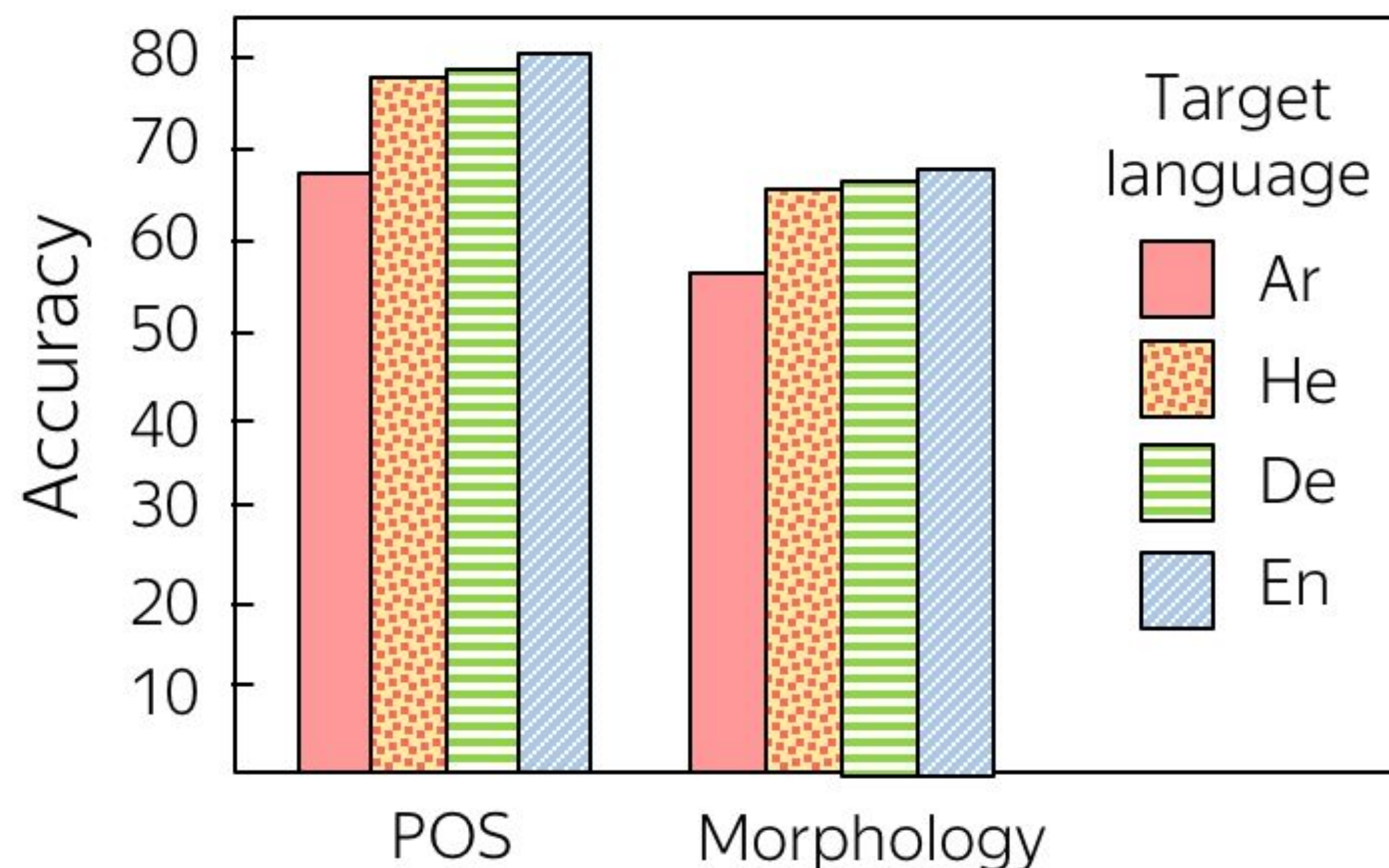
What Do NMT Models Learn About Morphology?

- Take NMT models with the same source language and different target languages
- Look at the encoder

Results:

- Surprising: weaker target morphology leads to stronger encoder

Effect of Target Language on POS/Morph



Learn more in the NLP Course **For**

You

→ This is up to You!

Paper: What do Neural Machine Translation Models Learn about Morphology?

What is going to happen:

- Seq2seq Basics
- Attention
- Transformer
- Transformer LMs
- BERT and family

What is going to happen:

- Seq2seq Basics
- Attention
- Transformer
- Transformer LMs
- BERT and family